OMNICODE: VOLUME 4: APPLICATIONS, INTEROP & INFRASTRUCTURE

Track 3 C/Lua Character Bindings, OmniStudio Interfaces & Track 5 Ops/CI

CORPUS VOLUME
VOL 4

NOTEBOOKLM WORD COUNT
29,265 words (Limit: 500k words)

CATALOGED FILES
48 files (8,255 lines)

GENERATION DATE
2026-10-14 10:52:51

EPISTEMIC COVENANT BEDROCK & IMMUTABLE LANDMARKS

"And he gave unto Moses... two tables of testimony, tables of stone, written with the finger of God." — Exodus 31:18

"Prepare thy work without, and make it fit for thyself in the field; and afterwards build thine house." — Proverbs 24:27

"Remove not the ancient landmark, which thy fathers have set." — Proverbs 22:28

GOOGLE NOTEBOOKLM (AI PRO) GROUNDING PROMPTS

- **Balanced Ternary Quad-State:** *"How does the T4 algebra {-1:Rollback, 0:Await, +1:Proceed, NULL:Halt} enforce symmetry in this volume?"*
- **4-Block Structure:** *"Explain the boundaries of Block 1 (Metadata/Scripture), Block 2 (Opening), Block 3 (Body), and Block 4 (Closing)."*
- **Character Lifecycle & SVO Verbs:** *"Trace the Character lifecycle through INSCRIBE, TRANSDUCE, PROJECT, and SEQUENCE across terminal ports 0x03F8 (FROM) and 0x000B8000 (INTO)."*

VOLUME SOURCE MANIFEST & INDEX (48 FILES)

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
1	**<TRACK 3> : {Applications & OmniStudio Interfaces}/TRACK.md**	MD	10	52	0.4
2	**<TRACK 3> : {Applications & OmniStudio Interfaces}/examples/01_boundaries_and_asides.omni**	OMNI	168	1,456	10.4
3	**<TRACK 3> : {Applications & OmniStudio Interfaces}/examples/02_logic_and_quad_state.omni**	OMNI	60	302	2.3
4	**<TRACK 3> : {Applications & OmniStudio Interfaces}/examples/03_cardinal_directions.omni**	OMNI	62	328	2.5
5	**<TRACK 3> : {Applications & OmniStudio Interfaces}/examples/04_turing_track_tape.omni**	OMNI	55	348	2.4
6	**<TRACK 3> : {Applications & OmniStudio Interfaces}/examples/05_control_flow_and_loops.omni**	OMNI	146	792	6.4
7	**<TRACK 3> : {Applications & OmniStudio Interfaces}/examples/canonical_notation.omni**	OMNI	75	476	3.2
8	**<TRACK 3> : {Applications & OmniStudio Interfaces}/examples/if_else_statement.omni**	OMNI	64	336	2.6
9	**<TRACK 3> : {Applications & OmniStudio Interfaces}/examples/if_then_statement.omni**	OMNI	60	275	2.3
10	**<TRACK 3> : {Applications & OmniStudio Interfaces}/examples/nested_loop.omni**	OMNI	79	375	2.9
11	**<TRACK 3> : {Applications & OmniStudio Interfaces}/examples/simple_loop.omni**	OMNI	66	316	2.4
12	**<TRACK 3> : {Applications & OmniStudio Interfaces}/examples/while_statement.omni**	OMNI	67	351	2.8
13	**<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/editors/vscode/README.md**	MD	79	379	2.8
14	**<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/editors/vscode/extension.js**	JS	93	306	3.9
15	**<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/editors/vscode/language-configuration.json**	JSON	30	76	0.5
16	**<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/editors/vscode/omnicode-language-0.1.0.vsix**	VSIX	1	5	2.9
17	**<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/editors/vscode/omnicode-language-0.1.1.vsix**	VSIX	1	5	4.0
18	**<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/editors/vscode/omnicode-language-0.1.2.vsix**	VSIX	1	5	6.4
19	**<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/editors/vscode/package.json**	JSON	54	92	1.1
20	**<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/editors/vscode/syntaxes/omnicode.tmLanguage.json**	JSON	321	529	8.7
21	**<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/editors/vscode/snippets/omnicode.json**	JSON	163	611	6.5
22	**<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/gui/app.js**	JS	509	1,537	20.4
23	**<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/gui/cloud.html**	HTML	201	718	10.8
24	**<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/gui/desktop_app.py**	PY	1,182	3,250	44.9
25	**<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/gui/download.html**	HTML	140	659	8.3
26	**<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/gui/index.html**	HTML	136	610	7.7
27	**<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/gui/lexicon.html**	HTML	114	508	6.3
28	**<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/gui/model.html**	HTML	152	719	8.9
29	**<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/gui/scrolls.html**	HTML	146	568	7.8
30	**<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/gui/specs.html**	HTML	155	798	9.6
31	**<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/gui/studio.html**	HTML	136	610	7.7
32	**<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/gui/style.css**	CSS	1,747	3,804	37.7

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
33	**<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/gui/terminal.html**	HTML	191	782	10.8
34	**<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/gui/terminal.js**	JS	305	885	11.9
35	**<TRACK 5> : {Infrastructure, Operations & Archives}/TRACK.md**	MD	12	57	0.5
36	**<TRACK 5> : {Infrastructure, Operations & Archives}/infra/systemd/omnicode-web.service**	SERVICE	22	34	0.5
37	**<TRACK 5> : {Infrastructure, Operations & Archives}/infra/docker/Dockerfile**	TEXT	178	947	7.8
38	**<TRACK 5> : {Infrastructure, Operations & Archives}/infra/docker/docker-compose.yml**	YML	113	650	5.7
39	**<TRACK 5> : {Infrastructure, Operations & Archives}/infra/scripts/check_toolchain.sh**	SH	57	227	2.5
40	**<TRACK 5> : {Infrastructure, Operations & Archives}/infra/scripts/deploy_oracle_cloud.sh**	SH	160	721	7.0
41	**<TRACK 5> : {Infrastructure, Operations & Archives}/infra/scripts/oci_security_list_helper.sh**	SH	87	474	4.1
42	**<TRACK 5> : {Infrastructure, Operations & Archives}/infra/scripts/omnicode_gui.sh**	SH	16	53	0.5
43	**<TRACK 5> : {Infrastructure, Operations & Archives}/infra/scripts/package_native.sh**	SH	125	395	4.2
44	**<TRACK 5> : {Infrastructure, Operations & Archives}/infra/scripts/setup_cloudflare_tunnel.sh**	SH	194	713	6.8
45	**<TRACK 5> : {Infrastructure, Operations & Archives}/infra/scripts/sync_oracle_vm.sh**	SH	133	577	5.6
46	**<TRACK 5> : {Infrastructure, Operations & Archives}/infra/dns/README.md**	MD	71	380	2.9
47	**<TRACK 5> : {Infrastructure, Operations & Archives}/infra/dns/omnicodestudio.com.zone**	ZONE	49	249	2.6
48	**<TRACK 5> : {Infrastructure, Operations & Archives}/infra/dns/zone_manager.py**	PY	269	925	10.7

<TRACK 3> : {Applications & OmniStudio Interfaces}/TRACK.md MD 10 LINES 52 WORDS 0.4 KB

```
# <TRACK 3> : {Applications & OmniStudio Interfaces}

## Scope & Invariant

User-facing tooling, visual studio interfaces, IDE integration, and canonical demonstration tapes.

- `apps/gui/`: OmniStudio HUD web interface & desktop application.
- `apps/editors/`: Visual Studio Code extension, grammar syntaxes, and language tooling.
- `apps/examples/`: Canonical verification reels and language showcase tapes.
```

<TRACK 3> : {Applications & OmniStudio Interfaces}/examples/01_boundaries_and_asides.omni

OMNI | 168 LINES | 1456 WORDS | 10.4 KB

```
# ============================================================================
# BLOCK 1: METADATA BLOCK
# ( Ranked Morphological Boundaries & Preserved Intent Asides )
# ============================================================================
# ARCHITECTURAL TOPOLOGY:
#   Foundational Showcase: The 6 Ranked Morphological Boundary Types.
#   - Rank 1: < ... > Invariant Axiom      (Immutable Law & Ontological Truth)
#   - Rank 2: [ ... ] Denotation           (Concrete Definition & Mutable State)
#   - Rank 3: { ... } Connotation          (Context, Environment & Payload)
#   - Rank 4: [ ... } Inside-Out Emergence (Entity Projects into Context)
#   - Rank 5: { ... ] Outside-In Imposition(Context Shapes Entity)
#   - Rank 6: ( ... ) Intent Aside         (First-Class Pronunciation & Teleology)
# ============================================================================

= [Module]: < Foundations { Boundaries } > & [ Example { Boundaries and Asides } ] ( Demonstrates the six ranked morphological boundaries and first-class intent asides ) {
    [ Version ]:   { "1.0.0" } ( The Version is in the context of 1.0.0 ),
    [ Created ]:   { "2026-10-11" } ( Created in the context of October 11, 2026 ),
    [ Updated ]:   { "2026-10-12" } ( Last updated in the context of October 12, 2026 ),
    [ Author ]:    { "CreativeWorkzStudio LLC / ProfessorSeanEX" } ( Author in the context of CreativeWorkzStudio LLC and ProfessorSeanEX ),
    [ System ]:    { OmniCode { Core Foundations } } ( System is in the context of OmniCode Core Foundations ),
    [ Witness ]:   < Scripture { Proverbs 8:29 KJV } > { "When he gave to the sea his decree, that the waters should not pass his commandment: when he appointed the foundations of the earth." } ( Invariant Scripture Witness Proverbs 8:29 in the context: When he appointed the foundations of the earth ),
    [ Nature ]:    < FoundationalExample > & { BaseBlock } ( Nature is a Foundational Example in the context of Base Block ),
    [ Semantics ]: { BoundaryPhysics :: MorphologicalClosure :: IntentPreservation } ( Semantics in the context of Boundary Physics, Morphological Closure, and Intent Preservation )
}

# Freestanding Foundational Axiom Declaration
# Rank 1: < ... > Invariant Axiom (Immutable Law)
< Axiom Truth > = < +1 > ( Ground truth axiom: immutably sealed in positive unity ) {
    [ Law ]:       < +1 > ( The immutable truth law is sealed in affirmative Proceed ),
    [ Geometry ]:  ENCLOSE [ Truth ] WITHIN < Invariant Space > ( Truth is bounded by immutable axiom walls )
}

# ============================================================================
# BLOCK 2: OPENING BLOCK
# ============================================================================
Opening {
    # Rank 1: Invariant Axiom (Inherited law)
    [ Ground Law ]: < Axiom Truth > ( Rank 1: Immutable axiom law ),

    # Rank 2: [ ... ] Denotation (Mutable Definition / State Container)
    [ State Alpha ]: [ +1 ] ( Rank 2: Concrete discrete definition with mutable state initialized to Proceed ),

    # Rank 3: { ... } Connotation (Ambient Universe / Environmental Context)
    { Universe Omega }: { Ground Law } ( Rank 3: Surrounding environmental context holding ground truth ),

    # Rank 4: [ ... } Inside-Out Emergence (Entity Projects Outward into Universe)
    [ Emergent Node }: [ State Alpha } ( Rank 4: Internal definition emerges outward into ambient context ),

    # Rank 5: { ... ] Outside-In Imposition (Environmental Context Shapes Internal Entity)
    { Imposed Node ]: { Universe Omega ] ( Rank 5: Surrounding context actively imposes constraints on internal entity ),

    # Rank 6: ( ... ) Intent Aside (Teleological Purpose & Cognitive Voice)
    ( Preserved Intent ): ( Rank 6: First-class teleological intent preserved in AST for human-AI shared reasoning ),

    # Hardware & Memory Preconditions
    [ Alignment ]: [ Memory Aligned 64 ] ( Memory aligned to 64-bit boundary ),
    [ Preconditions ]: [ (+1) plus (-1) <=> 0 ] & [ Status = +1:PROCEED ] ( ALU equilibrium holds and the machine status is armed in Proceed )
}

# ============================================================================
# BLOCK 3: BODY BLOCK
# ============================================================================
Body [
    # ------------------------------------------------------------------------
    # ROLL 1: RANK 1 INVARIANT VERIFICATION
    # Verify immutable ground truth identity
```

```
    # ------------------------------------------------------------------------
    [ ROLL 1 ] : [ Verify Axiom ] : < Axiom Truth > <=> VERIFY [ +1 ] = [ Truth` ]
        ( Rank 1: Verify immutable ground truth axiom against positive unity ),

    # ------------------------------------------------------------------------
    # ROLL 2: RANK 2 DENOTATIVE STATE MUTATION
    # Advance mutable state within defined boundary
    # ------------------------------------------------------------------------
    [ ROLL 2 ] : [ Mutate State ] : [ State Alpha ] --> ADVANCE [ +1 ] = [ State Alpha` ]
        ( Rank 2: Advance discrete denotative state yielding new state State-Alpha-prime ),

    # ------------------------------------------------------------------------
    # ROLL 3: RANK 3 CONNOTATIVE CONTEXT EXPANSION
    # Expand ambient contextual universe with verified state
    # ------------------------------------------------------------------------
    [ ROLL 3 ] : [ Expand Context ] : { Universe Omega } <-- ABSORB { State Alpha` } = { Universe Omega` }
        ( Rank 3: Ambient context absorbs new denotative state expanding environmental scope ),

    # ------------------------------------------------------------------------
    # ROLL 4: RANK 4 INSIDE-OUT ASYMMETRIC EMERGENCE
    # Project internal discrete state outward into ambient context
    # ------------------------------------------------------------------------
    [ ROLL 4 ] : [ Project Emergence ] : [ State Alpha` } --> EMERGE { Universe Omega` } = [ Emergent Node` }
        ( Rank 4: Discrete state emerges through asymmetrical boundary into ambient universe ),

    # ------------------------------------------------------------------------
    # ROLL 5: RANK 5 OUTSIDE-IN ASYMMETRIC IMPOSITION
    # Impose ambient contextual constraints onto internal discrete entity
    # ------------------------------------------------------------------------
    [ ROLL 5 ] : [ Impose Context ] : { Universe Omega` ] --> IMPOSE [ State Alpha` ] = { Imposed Node` ]
        ( Rank 5: Ambient universe imposes environmental boundaries onto internal discrete state ),

    # ------------------------------------------------------------------------
    # ROLL 6: RANK 6 FIRST-CLASS INTENT ASIDE PRESERVATION
    # Preserve teleological intent as a first-class AST node for human-AI cognition
    # ------------------------------------------------------------------------
    [ ROLL 6 ] : [ Preserve Intent ] : ( Preserved Intent ) --> RECORD [ AST ] = ( Teleology Verified` )
        ( Rank 6: Intent aside preserved as live AST node for human-AI cognitive continuity )
]

# ============================================================================
# BLOCK 4: CLOSING BLOCK
# ============================================================================
Closing ( Sealing Gate: Reconcile, validate, tripwire, debug, log, and affirm permanent boundary execution ) {
    # ------------------------------------------------------------------------
    # 1. VERIFICATION OF OPERATIONAL ROLLS
    # ------------------------------------------------------------------------
    [ Verify ]: [
        [ ROLL 1 ] = PROCEED ::
        [ ROLL 2 ] = PROCEED ::
        [ ROLL 3 ] = PROCEED ::
        [ ROLL 4 ] = PROCEED ::
        [ ROLL 5 ] = PROCEED ::
        [ ROLL 6 ] = PROCEED
    ] ( Verify that all six boundary operational rolls achieved affirmative consensus in PROCEED ),

    # ------------------------------------------------------------------------
    # 2. VALIDATION OF INVARIANT INTEGRITY & CLOSURE
    # ------------------------------------------------------------------------
    [ Validate ]: { Truth` = +1 } & { State Alpha` = +1 } & { Emergent Node` } & { Imposed Node` } & ( Teleology
Verified` )
        ( Validate that invariant truth holds, state remains affirmative, and all morphological closures and asides are
sealed ),

    # ------------------------------------------------------------------------
    # 3. ZERO-TOLERANCE TRIPWIRING
    # ------------------------------------------------------------------------
    [ Tripwire ]: { Status` = +1:PROCEED } --> [ ARMED ] | { Status` != +1:PROCEED } --> [ TRIPPED ] --> [ Commit Status
= -1:ROLLBACK ]
        ( Tripwire remains armed in Proceed or trips on anomaly to veto execution ),

    # ------------------------------------------------------------------------
    # 4. DIAGNOSTIC DEBUG TELEMETRY
    # ------------------------------------------------------------------------
    [ Debug ]: { [ Truth` ] :: [ State Alpha` ] :: [ Emergent Node` ] :: [ Imposed Node` ] :: ( Teleology Verified` ) ::
[ Status` ] }
        ( Expose diagnostic telemetry across all verified morphological boundaries and machine status ),

```

```
    # ------------------------------------------------------------------------
    # 5. EPISODIC INVENTORY LOGGING
    # ------------------------------------------------------------------------
    [ Log ]: [ Journal ]: INSCRIBE { [ Boundaries Sealed ] :: [ Rank 1 Through 6 ] :: [ Truth` ] }
        ( Inscribe the verified boundary physics demonstration into the permanent episodic system journal ),

    # ------------------------------------------------------------------------
    # 6. PERMANENT COVENANT EXECUTION & FOURFOLD COMMIT STATUS LOOP
    # ------------------------------------------------------------------------
    [ Commit Status ]: [
        # +1: PROCEED (+t: Advance forward along the causal tape)
        { [ Verify ] = PROCEED } & { [ Tripwire ] = ARMED }
            --> [ +1:PROCEED ] |

        #  0: AWAIT (t=t: Hold in equilibrium, pending resolution)
        { [ Status` ] = PENDING }
            --> [ 0:AWAIT ] :: REPEAT |

        # NULL: HALT (t=empty: Cessation of clock, absolute stop)
        { [ Status` ] = NULL }
            --> [ HALT:NULL ] |

        # -1: ROLLBACK (-t: Reverse causal tape to origin t=0)
        { [ Tripwire ] = TRIPPED } | { [ Verify ] != PROCEED }
            --> [ -1:ROLLBACK ]
    ] ( Fourfold state machine: Proceed advances forward, Await holds equilibrium, Halt ceases execution in Null,
Rollback reverts to origin )
}
```

<TRACK 3> : {Applications & OmniStudio Interfaces}/examples/02_logic_and_quad_state.omni

OMNI | 60 LINES | 302 WORDS | 2.3 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Balanced Ternary Logic & Micro/Macro Quad-State Truth Space )
# ================================================================================
Module < Logic.QuadStateEngine > ( Showcase Balanced Ternary and Micro/Macro Duality ) {
    version: < "1.0.0" >,
    system: < OmniCode.Core >,
    semantics: < BalancedTernary :: MicroMacroDuality :: CaveatPhysics >
}

AxiomAffirmation < yes >    ( Micro: yes == Macro: PROCEED == +1 )
AxiomRefutation  < no >     ( Micro: no  == Macro: ROLLBACK == -1 )
AxiomBalance     < ok >     ( Micro: ok  == Macro: AWAIT    ==  0 )

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( Preconditions & State Initialization )
# ================================================================================
Opening {
    requires: [ TruthSpace.T4 ],
    preconditions: [ AxiomAffirmation = PROCEED ]
}

flag_affirm [ yes ]   ( Initial affirmative state )
flag_negate [ -- yes ] ( Negation flips +1 to -1: no )

# ================================================================================
# BLOCK 3: BODY BLOCK
# ( Truth Space Algebra: Conjunction, Disjunction, Negation, and Caveat Gate )
# ================================================================================
Body [
    # 1. Micro/Macro Equivalence Checks
    affirm_check [ flag_affirm = PROCEED ] ( Affirmation matches PROCEED )
    refute_check [ flag_negate = ROLLBACK ] ( Negated affirmation matches ROLLBACK )

    # 2. Conjunction (:: AND) and Disjunction (| OR)
    and_test [ yes :: yes = PROCEED ]
    or_test  [ yes | no = PROCEED ]

    # 3. Adversative Caveat Gate (! BUT)
    # The action proceeds ONLY because the caveat holds (+1)
    gated_action [ flag_affirm ! [ affirm_check = yes ] ] ( Caveat permits forward commit )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ( Verification & Macro State Commit )
# ================================================================================
Closing {
    verify: [
        affirm_check = yes ::
        refute_check = yes ::
        and_test = yes ::
        or_test = yes ::
        gated_action = PROCEED
    ],
    commit_status [ PROCEED ] ( Commit verified quad-state execution )
}

```

<TRACK 3> : {Applications & OmniStudio Interfaces}/examples/03_cardinal_directions.omni

OMNI | 62 LINES | 328 WORDS | 2.5 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( 4 Cardinal Directions Across Causality and Abstraction Axes )
# ================================================================================
Module < Geometry.CardinalDirections > ( Showcase 4 cardinal directional vectors and shorthands ) {
    version: < "1.0.0" >,
    system: < OmniCode.Core >,
    semantics: < VectorDynamics :: CausalityAndScale >
}

BaseDomain < 100 >   ( Ground level primitive )
TargetDomain < 200 > ( Elevated abstract domain )

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( Vector Preconditions & Initial States )
# ================================================================================
Opening {
    requires: [ CoordinatePlane.2D ],
    preconditions: [ BaseDomain = 100 ]
}

step_x [ 10 ] ( State moving across X temporal axis )
step_z [ 10 ] ( State moving across Z scale axis )

# ================================================================================
# BLOCK 3: BODY BLOCK
# ( Cardinal Vector Executions: Right, Left, Up, Down )
# ================================================================================
Body [
    # 1. Right (--> and ->): Forward Causation / Act Upon
    forward_canonical [ step_x --> 20 ] ( Forward step advancing state to 20 )
    forward_shorthand [ step_x -> 30 ]  ( Shorthand forward step to 30 )

    # 2. Left (<-- and <-): Retroaction / Feedback / Rollback
    feedback_canonical [ step_x <-- 10 ] ( Retroactive feedback step )
    feedback_shorthand [ step_x <- 10 ]  ( Shorthand feedback step )

    # 3. Up (--^ and -^): Synthesis / Abstraction / Ascend
    ascend_canonical [ step_z --^ TargetDomain ] ( Synthesis into target domain )
    ascend_shorthand [ step_z -^ TargetDomain ]  ( Shorthand synthesis )

    # 4. Down (--v and -v): Decomposition / Analysis / Descend
    descend_canonical [ TargetDomain --v BaseDomain ] ( Decomposition into base domain )
    descend_shorthand [ TargetDomain -v BaseDomain ]  ( Shorthand decomposition )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ( Vector Integrity Verification & Commit )
# ================================================================================
Closing {
    verify: [
        forward_canonical = 20 ::
        forward_shorthand = 30 ::
        ascend_canonical = 200 ::
        descend_canonical = 100
    ],
    commit_status [ PROCEED ] ( Commit verified vector trajectory )
}

```

<TRACK 3> : {Applications & OmniStudio Interfaces}/examples/04_turing_track_tape.omni

OMNI | 55 LINES | 348 WORDS | 2.4 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK (THE TRACK)
# ( Track Manifest, Semantic Intent, and Inferred Ontological Types )
# ================================================================================
Module < ArithmeticReel.Tape1 > ( Add 1 ball to the group A which contains 4 balls ) {
    version: < "1.0.0" >,
    system: < OmniCode.Core >,
    semantics: < TrackPhysics :: MultiTape :: AIInference >
}

# The ? (Infer Operator) marks concepts whose structural types are inferred by the AI brain:
Ball < Entity? > ( Inferred discrete unit: resolved from natural sentence )
Group < Set? >   ( Inferred collection container: resolved from natural sentence )

# ================================================================================
# BLOCK 2: OPENING BLOCK (THE HEAD)
# ( The Focal Setup: Reading Initial Conditions and Contextual State )
# ================================================================================
Opening {
    import: [ Core.Arithmetic :: Core.Logic ],
    requires: [ Group : Set? :: Ball : Entity? ],
    preconditions: [ +1 ]
}

# The Head sets Group A with initial count of 4 balls
A [ 4 : Ball? ] { Group? } ( Group A initialized with 4 inferred balls )
new_ball [ 1 : Ball? ]     ( 1 additional ball to be introduced )

# ================================================================================
# BLOCK 3: BODY BLOCK (THE TAPE & ROLLS)
# ( The Tape Execution: SVO Rolls advancing the state forward using 'plus' )
# ================================================================================
Body [
    # ROLL 1 (SVO Action): Subject A acts upon target, advancing by 'plus' new_ball
    roll_1 [ A --> A plus new_ball ] ( Act upon A: causal transition from 4 to 5 balls )

    # ROLL 2 (SVO Verification): Subject A is 5 under the caveat that new_ball is 1
    roll_2 [ A = 5 ! [ new_ball = 1 ] ] ( State is 5, provided caveat of 1 added ball holds )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK (REEL COMMIT & PROOF)
# ( Tape Stop: Verification Invariants and State Commit )
# ================================================================================
Closing {
    verify: [
        roll_1 = 5 ::
        roll_2 = PROCEED ::
        A = 5 ::
        ADD = InvariantMacro
    ],
    commit_status [ PROCEED ] ( Commit the verified tape state )
}

```

<TRACK 3> : {Applications & OmniStudio Interfaces}/examples/05_control_flow_and_loops.omni

OMNI | 146 LINES | 792 WORDS | 6.4 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Control Flow Manifest: Quad-State Accumulator Loops & Conditional Branching )
# ================================================================================
Module < ControlFlow.QuadStateShowcase > ( Canonical showcase of Simple Loops, Nested Loops, While statements, and
If/Then/Else branches ) {
    version: < "1.0.0" >,
    author: < "ProfessorSeanEX & Antigravity" >,
    system: < OmniCode.Core >,
    semantics: < BalancedTernary :: QuadStateControlFlow :: NamedSiliconRegisters >
}

# Axiomatic Boundary Limits & Invariant Constants
LOOP_BOUND < 10 >          ( Invariant boundary ceiling for linear simple loop )
GRID_ROWS  < 3 >           ( Invariant outer dimension: 3 rows )
GRID_COLS  < 4 >           ( Invariant inner dimension: 4 columns )
THRESHOLD  < 100 >         ( Invariant threshold boundary for while condition )

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( Hardware Context Setup, Register Allocations & Preconditions )
# ================================================================================
Opening {
    import: [ Core.Logic :: Core.Arithmetic ],
    requires: [ TruthSpace.T4 :: MemoryAligned ],
    preconditions: [ +1 ]
}

# Physical Register Initializations for Body Execution
loop_acc         [ 0 ] ( Silicon Accumulator IT container )
loop_turn        [ 0 ] ( Temporal Cadence TURN container )
sensor_reading   [ 25 ] ( Dynamic reading for while statement )
condition_affirm [ yes ] ( Affirmative condition flag: +1 )
condition_refute [ no ]  ( Refuted condition flag: -1 )

# ================================================================================
# BLOCK 3: BODY BLOCK
# ( The 5 Canonical Control Flow Constructs )
# ================================================================================
Body [
    # ----------------------------------------------------------------------------
    # 1. SIMPLE LOOP: Linear Quad-State Accumulator cycling from 0 up to BOUND
    # ----------------------------------------------------------------------------
    simple_loop [
        STATE (
            < BOUND = LOOP_BOUND >,
            [ IT = 0, TURN = 0 ],
            { SPOOL }
        ) = [
            PROCEED  -WHEN-> { (IT plus 1 --> IT) & (TURN plus 1 --> TURN) },
            AWAIT    -WHEN-> { BUSY },
            ROLLBACK -WHEN-> { FAULT & (HOME --> IT) },
            HALT     -WHEN-> { TURN = BOUND }
        ]
    ] ( Simple linear loop: ticks up IT and TURN until watchdog BOUND is met )

    # ----------------------------------------------------------------------------
    # 2. NESTED LOOP: 2D Matrix Grid Iteration (Outer Row x Inner Column)
    # ----------------------------------------------------------------------------
    nested_loop [
        STATE (
            < ROW_BOUND = GRID_ROWS, COL_BOUND = GRID_COLS >,
            [ ROW_TURN = 0, ROW_SUM = 0 ],
            { SPOOL }
        ) = [
            PROCEED -WHEN-> {
                # Inner Column State Loop: accumulates COL_BOUND cells per row
                STATE (
                    < BOUND = COL_BOUND >,
                    [ COL_TURN = 0, IT = ROW_SUM ],
                    { SPOOL }
                ) = [
                    PROCEED  -WHEN-> { (IT plus 1 --> IT) & (COL_TURN plus 1 --> COL_TURN) },
```

```
                    AWAIT    -WHEN-> { BUSY },
                    ROLLBACK -WHEN-> { FAULT & (HOME --> IT) },
                    HALT     -WHEN-> { COL_TURN = BOUND }
                ] &
                (IT --> ROW_SUM) &
                (ROW_TURN plus 1 --> ROW_TURN)
            },
            AWAIT    -WHEN-> { BUSY },
            ROLLBACK -WHEN-> { FAULT },
            HALT     -WHEN-> { ROW_TURN = ROW_BOUND }
        ]
    ] ( Nested 2D loop: outer row cadence driving inner column accumulator )

    # ------------------------------------------------------------------------
    # 3. WHILE STATEMENT: Predicated Quad-State Loop
    # Advances while sensor_reading < THRESHOLD; halts when condition ceases
    # ------------------------------------------------------------------------
    while_statement [
        STATE (
            < BOUND = THRESHOLD >,
            [ IT = sensor_reading, TURN = 0 ],
            { FROM }
        ) = [
            PROCEED  -WHEN-> { (sensor_reading < THRESHOLD) & (IT plus 15 --> IT) & (TURN plus 1 --> TURN) },
            AWAIT    -WHEN-> { SENSOR_BUSY },
            ROLLBACK -WHEN-> { SENSOR_ERROR & (HOME --> IT) },
            HALT     -WHEN-> { (IT >= THRESHOLD) | (TURN = 100) }
        ]
    ] ( While loop: continues PROCEED while predicate holds, HALT escapes boundary )

    # ------------------------------------------------------------------------
    # 4. IF / THEN STATEMENT: Unilateral Conditional Gate
    # Evaluates condition: if affirmative (+1), actuates then branch; else awaits
    # ------------------------------------------------------------------------
    if_then_statement [
        STATE (
            [ IT = 0 ],
            { INTO }
        ) = [
            PROCEED -WHEN-> { condition_affirm & (42 --> IT) },
            AWAIT   -WHEN-> { -- condition_affirm }
        ]
    ] ( If/Then gate: proceeds with forward action only under verified affirmative condition )

    # ------------------------------------------------------------------------
    # 5. IF / ELSE STATEMENT: Bilateral Ternary Branch
    # Evaluates condition: +1 forks to then branch, -1 forks to else branch
    # ------------------------------------------------------------------------
    if_else_statement [
        STATE (
            [ IT = 0 ],
            { INTO }
        ) = [
            PROCEED  -WHEN-> { (condition_refute = yes) & (100 --> IT) },
            ROLLBACK -WHEN-> { (condition_refute = no)  & (200 --> IT) },
            AWAIT    -WHEN-> { condition_refute = ok }
        ]
    ] ( If/Else branch: balanced ternary fork selecting between affirmative and negative vectors )
]

# ============================================================================
# BLOCK 4: CLOSING BLOCK
# ( Postconditions, Invariant Integrity Proofs & Macro State Commit )
# ============================================================================
Closing {
    verify: [
        LOOP_BOUND = 10 ::
        GRID_ROWS = 3 ::
        GRID_COLS = 4 ::
        THRESHOLD = 100
    ],
    commit_status [ PROCEED ] ( Verified quad-state control flow suite sealed )
}
```

<TRACK 3> : {Applications & OmniStudio Interfaces}/examples/canonical_notation.omni

OMNI | 75 LINES | 476 WORDS | 3.2 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( File Identity, Version, Author, Intent Aside, and Ontological Axioms )
# ================================================================================
Module < CanonicalNotationShowcase > ( Foundations of Being, Causality, and Abstraction ) {
    version: < "1.0.0" >,
    author: < "ProfessorSeanEX" >,
    system: < OmniCode.Core >,
    semantics: < BalancedTernary :: QuadState :: BoundaryPhysics >
}

# Axiomatic Categories (Invariant Sets)
A < +1 > ( Invariant category A: axiomatic ground truth )
B < +1 > ( Invariant category B: independent set domain )


# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( Environment Setup, Preconditions, Scoped Universes, and Context Dictation )
# ================================================================================
Opening {
    requires: [ MemoryAligned :: TypeChecking ],
    initial_truth: [ +1 ]
}

# Mutable Denotations initialized in Opening
a [ +1 ] ( Particular instance 'a' initialized with mutable proceed state )
b [ +1 ] ( Particular instance 'b' initialized with mutable proceed state )

# Contextual Connotations & Asymmetric Causality
a_in_A [ a ] { A }  ( Instance 'a' validated within contextual universe A )
a_top_down { A ]    ( Outside-In: Environment A actively forces definition of a )
b_bottom_up [ B }   ( Inside-Out: Definition of b emerges from context B )


# ================================================================================
# BLOCK 3: BODY BLOCK
# ( Core Transformations, Being Relations, Causality Vectors, and Connectives )
# ================================================================================
Body [
    # 1. Ontological Relations of Being (Y-Axis)
    instance_typing [ a : A ]    ( 'a' stands in relation to Category A )
    identity_check [ a = a ]     ( 'a' is identical to 'a' )
    distinction_check [ a != b ] ( 'a' is not 'b' )
    similarity_check [ a ~= b ]  ( 'a' is liken to 'b' in structural form )

    # 2. Temporal & Causal Vectors (X-Axis)
    forward_action [ a --> b ]   ( 'a' acts upon 'b' forward in time )
    feedback_action [ b <-- a ]  ( 'b' reflects back upon 'a' as retroactive feedback )

    # 3. Hierarchical Abstraction Vectors (Z-Axis)
    synthesis [ a ^-- A ]        ( Low-level primitive 'a' builds up to high-level concept A )
    decomposition [ A v-- a ]    ( High-level category A decomposes down into primitive 'a' )

    # 4. Connectives & Adversative Caveat Pipeline
    negation [ -- a ]            ( Inverts state of 'a' across ternary zero balance )
    conjunction [ a : A :: b : B ]
    pipeline [ a --> b ! [ a : A :: b : B ] ] ( 'a' acts on 'b', BUT caveat must be satisfied )
]


# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ( Postconditions, Invariant Integrity Proofs, Teardown, and Commit / Return )
# ================================================================================
Closing {
    verify: [
        identity_check = +1 ::
        forward_action = +1 ::
        pipeline = +1
    ],
    on_rollback: [ <-- unwind_state ],
```

```
    commit_status: [ +1 ] ( Commit state: verified execution )
}
```

<TRACK 3> : {Applications & OmniStudio Interfaces}/examples/if_else_statement.omni

OMNI | 64 LINES | 336 WORDS | 2.6 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( File Manifest: Canonical Quad-State If/Else Bilateral Ternary Branch )
# ================================================================================
Module < ControlFlow.IfElseStatement > ( Demonstrates bilateral ternary branching in Quad-State logic ) {
    version: < "1.0.0" >,
    author: < "ProfessorSeanEX & Antigravity" >,
    system: < OmniCode.Core >,
    semantics: < BalancedTernary :: TernaryBranching :: NamedSiliconRegisters >
}

# Invariant Branch Targets
THEN_RESULT < 100 > ( Value emitted when condition is affirmative: +1 )
ELSE_RESULT < 200 > ( Value emitted when condition is refutative: -1 )

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( Context Preconditions & Branch Flag Setup )
# ================================================================================
Opening {
    import: [ Core.Logic :: Core.Arithmetic ],
    requires: [ TruthSpace.T4 ],
    preconditions: [ +1 ]
}

# The active condition to evaluate (set to 'no' to trigger else branch)
branch_condition [ no ] ( Initial condition evaluating to -1 / ROLLBACK )
branch_acc       [ 0 ]  ( Accumulator register receiving selected branch output )

# ================================================================================
# BLOCK 3: BODY BLOCK
# ( Execution: If/Else Bilateral Branch )
# ================================================================================
Body [
    # THE IF / ELSE STATEMENT:
    # `if (branch_condition == yes) { IT = THEN_RESULT; } else { IT = ELSE_RESULT; }`
    # - PROCEED (+1) : True branch (actuates THEN_RESULT into IT)
    # - ROLLBACK (-1): False branch (actuates ELSE_RESULT into IT)
    # - AWAIT (0)    : Neutral / uncommitted state (holds position)
    if_else_branch [
        STATE (
            [ IT = branch_acc ],
            { INTO }
        ) = [
            PROCEED  -WHEN-> { (branch_condition = yes) & (THEN_RESULT --> IT) },
            ROLLBACK -WHEN-> { (branch_condition = no)  & (ELSE_RESULT --> IT) },
            AWAIT    -WHEN-> { branch_condition = ok }
        ]
    ] ( If/Else branch: selects between affirmative then and refutative else vectors )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ( Postconditions & Commit Status )
# ================================================================================
Closing {
    verify: [
        THEN_RESULT = 100 ::
        ELSE_RESULT = 200 ::
        branch_condition = no
    ],
    commit_status [ PROCEED ] ( If/Else ternary branch verified and sealed )
}
```

<TRACK 3> : {Applications & OmniStudio Interfaces}/examples/if_then_statement.omni

OMNI | 60 LINES | 275 WORDS | 2.3 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( File Manifest: Canonical Quad-State If/Then Unilateral Conditional Gate )
# ================================================================================
Module < ControlFlow.IfThenStatement > ( Demonstrates unilateral conditional gating in Quad-State logic ) {
    version: < "1.0.0" >,
    author: < "ProfessorSeanEX & Antigravity" >,
    system: < OmniCode.Core >,
    semantics: < BalancedTernary :: ConditionalGating :: NamedSiliconRegisters >
}

# Invariant Target Payload
PAYLOAD_VALUE < 42 > ( Value to be transduced upon affirmative condition )

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( Context Preconditions & Flag Setup )
# ================================================================================
Opening {
    import: [ Core.Logic :: Core.Arithmetic ],
    requires: [ TruthSpace.T4 ],
    preconditions: [ +1 ]
}

# Initial Register & Condition States
permission_flag [ yes ] ( Affirmative precondition flag: +1 )
accumulator     [ 0 ]   ( Destination IT register )

# ================================================================================
# BLOCK 3: BODY BLOCK
# ( Execution: If/Then Conditional Gate )
# ================================================================================
Body [
    # THE IF / THEN STATEMENT:
    # `if (permission_flag) { accumulator = PAYLOAD_VALUE; }`
    # - PROCEED: When condition is affirmative (+1), actuates vector into IT
    # - AWAIT  : When condition is non-affirmative, holds state without mutating
    if_then_dispatch [
        STATE (
            [ IT = accumulator ],
            { INTO }
        ) = [
            PROCEED -WHEN-> { permission_flag & (PAYLOAD_VALUE --> IT) },
            AWAIT   -WHEN-> { -- permission_flag }
        ]
    ] ( If/Then gate: proceeds with forward action only when permission holds )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ( Postconditions & Commit Status )
# ================================================================================
Closing {
    verify: [
        PAYLOAD_VALUE = 42 ::
        permission_flag = yes
    ],
    commit_status [ PROCEED ] ( If/Then conditional gate verified and sealed )
}
```

<TRACK 3> : {Applications & OmniStudio Interfaces}/examples/nested_loop.omni

OMNI | 79 LINES | 375 WORDS | 2.9 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( File Manifest: Canonical Quad-State Nested 2D Grid Loop )
# ================================================================================
Module < ControlFlow.NestedLoop > ( Demonstrates 2D matrix row and column grid iteration ) {
    version: < "1.0.0" >,
    author: < "ProfessorSeanEX & Antigravity" >,
    system: < OmniCode.Core >,
    semantics: < BalancedTernary :: NestedStateLoop :: NamedSiliconRegisters >
}

# Invariant Grid Dimensions
NUM_ROWS < 3 > ( Outer loop dimension: 3 rows )
NUM_COLS < 4 > ( Inner loop dimension: 4 columns )

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( Context Preconditions & Coordinate Initialization )
# ================================================================================
Opening {
    import: [ Core.Logic :: Core.Arithmetic ],
    requires: [ TruthSpace.T4 ],
    preconditions: [ +1 ]
}

# Initial Register Setup
grid_sum  [ 0 ] ( Running total accumulated across all cells )
grid_rows [ 0 ] ( Current outer row iteration count )

# ================================================================================
# BLOCK 3: BODY BLOCK
# ( Execution: Nested Row/Column Traversal )
# ================================================================================
Body [
    # THE NESTED LOOP:
    # Outer STATE manages ROW_TURN up to ROW_BOUND.
    # On each outer PROCEED, the inner STATE manages COL_TURN up to COL_BOUND,
    # accumulating into IT and retiring into grid_sum.
    matrix_traversal [
        STATE (
            < ROW_BOUND = NUM_ROWS, COL_BOUND = NUM_COLS >,
            [ ROW_TURN = 0, ROW_ACC = 0 ],
            { SPOOL }
        ) = [
            PROCEED -WHEN-> {
                # Inner Column State Loop:
                STATE (
                    < BOUND = COL_BOUND >,
                    [ COL_TURN = 0, IT = ROW_ACC ],
                    { SPOOL }
                ) = [
                    PROCEED  -WHEN-> { (IT plus 1 --> IT) & (COL_TURN plus 1 --> COL_TURN) },
                    AWAIT    -WHEN-> { BUSY },
                    ROLLBACK -WHEN-> { FAULT & (HOME --> IT) },
                    HALT     -WHEN-> { COL_TURN = BOUND }
                ] &
                (IT --> ROW_ACC) &
                (ROW_TURN plus 1 --> ROW_TURN)
            },
            AWAIT    -WHEN-> { BUSY },
            ROLLBACK -WHEN-> { FAULT },
            HALT     -WHEN-> { ROW_TURN = ROW_BOUND }
        ]
    ] ( Nested 2D loop traverses all rows and columns to completion )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ( Postconditions & Commit Status )
# ================================================================================
Closing {
    verify: [
        NUM_ROWS = 3 ::
```

```
        NUM_COLS = 4 ::
        grid_sum = 0
    ],
    commit_status [ PROCEED ] ( Nested loop verified and sealed )
}
```

<TRACK 3> : {Applications & OmniStudio Interfaces}/examples/simple_loop.omni

OMNI | 66 LINES | 316 WORDS | 2.4 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( File Manifest: Canonical Quad-State Simple Linear Loop )
# ================================================================================
Module < ControlFlow.SimpleLoop > ( Demonstrates single-dimensional linear accumulator loop ) {
    version: < "1.0.0" >,
    author: < "ProfessorSeanEX & Antigravity" >,
    system: < OmniCode.Core >,
    semantics: < BalancedTernary :: QuadStateLoop :: NamedSiliconRegisters >
}

# Invariant Watchdog Bound
CEILING < 10 > ( Invariant maximum bound for accumulator loop )

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( Context Preconditions & Register Initialization )
# ================================================================================
Opening {
    import: [ Core.Logic :: Core.Arithmetic ],
    requires: [ TruthSpace.T4 ],
    preconditions: [ +1 ]
}

# Initial Register Setup
initial_it   [ 0 ] ( Initial primary accumulator IT value )
initial_turn [ 0 ] ( Initial temporal cadence TURN value )

# ================================================================================
# BLOCK 3: BODY BLOCK
# ( Execution: Linear Accumulator Loop )
# ================================================================================
Body [
    # THE SIMPLE LOOP:
    # Cycles through T4 truth space:
    #   +1 (PROCEED)  : Ticks up IT and TURN
    #    0 (AWAIT)    : Yields / waits if bus is busy
    #   -1 (ROLLBACK) : Rewinds to baseline HOME on fault
    #   NULL (HALT)   : Breaks loop when TURN reaches BOUND
    loop_result [
        STATE (
            < BOUND = CEILING >,
            [ IT = initial_it, TURN = initial_turn ],
            { SPOOL }
        ) = [
            PROCEED  -WHEN-> { (IT plus 1 --> IT) & (TURN plus 1 --> TURN) },
            AWAIT    -WHEN-> { BUSY },
            ROLLBACK -WHEN-> { FAULT & (HOME --> IT) },
            HALT     -WHEN-> { TURN = BOUND }
        ]
    ] ( Linear accumulator loop runs to completion when TURN equals BOUND )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ( Postconditions & Commit Status )
# ================================================================================
Closing {
    verify: [
        CEILING = 10 ::
        initial_it = 0 ::
        initial_turn = 0
    ],
    commit_status [ PROCEED ] ( Simple loop verified and sealed )
}
```

<TRACK 3> : {Applications & OmniStudio Interfaces}/examples/while_statement.omni

OMNI | 67 LINES | 351 WORDS | 2.8 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( File Manifest: Canonical Quad-State Predicated While Loop )
# ================================================================================
Module < ControlFlow.WhileStatement > ( Demonstrates predicate-governed loop advancing while condition holds ) {
    version: < "1.0.0" >,
    author: < "ProfessorSeanEX & Antigravity" >,
    system: < OmniCode.Core >,
    semantics: < BalancedTernary :: PredicatedWhileLoop :: NamedSiliconRegisters >
}

# Invariant Threshold Boundary
MAX_THRESHOLD < 100 > ( Invariant upper ceiling triggering while loop termination )
MAX_CYCLES    < 500 > ( Watchdog maximum turn count )

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( Context Preconditions & Initial Sensor Values )
# ================================================================================
Opening {
    import: [ Core.Logic :: Core.Arithmetic ],
    requires: [ TruthSpace.T4 ],
    preconditions: [ +1 ]
}

# Initial Stream Values
sensor_reading [ 10 ] ( Initial inbound value to be sampled )
sensor_status  [ ok ] ( Initial bus status: equilibrium 0 )

# ================================================================================
# BLOCK 3: BODY BLOCK
# ( Execution: Predicated While Loop )
# ================================================================================
Body [
    # THE WHILE STATEMENT:
    # `while (sensor_reading < MAX_THRESHOLD) { Process_Stream(); }`
    # - PROCEED: Advances while the predicate is affirmative (+1)
    # - AWAIT  : Yields/pauses while the hardware sensor is busy sampling (0)
    # - ROLLBACK: Restores baseline from HOME if parity fails (-1)
    # - HALT   : Escapes the loop when predicate becomes false or watchdog trips (NULL)
    while_sensor_stream [
        STATE (
            < BOUND = MAX_CYCLES >,
            [ IT = sensor_reading, TURN = 0 ],
            { FROM }
        ) = [
            PROCEED  -WHEN-> { (sensor_reading < MAX_THRESHOLD) & (IT plus 10 --> IT) & (TURN plus 1 --> TURN) },
            AWAIT    -WHEN-> { SENSOR_BUSY },
            ROLLBACK -WHEN-> { SENSOR_ERROR & (HOME --> IT) },
            HALT     -WHEN-> { (IT >= MAX_THRESHOLD) | (TURN = BOUND) }
        ]
    ] ( While statement loops until sensor exceeds threshold or bound is reached )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ( Postconditions & Commit Status )
# ================================================================================
Closing {
    verify: [
        MAX_THRESHOLD = 100 ::
        MAX_CYCLES = 500 ::
        sensor_reading = 10
    ],
    commit_status [ PROCEED ] ( While loop verified and sealed )
}
```

<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/editors/vscode/README.md

MD | 79 LINES | 379 WORDS | 2.8 KB

```
# OmniCode VS Code & IDE Extension

> **Official Language Support Package for Visual Studio Code, Cursor, and Google Antigravity IDE.**

Provides morphological boundary colorization, 3D coordinate operator grammar, balanced ternary literals, and universal 4-Block file templates.

---

## 1. Extension Topology & Installation

### File Topology

```text
editors/vscode/
├── package.json                  # Extension manifest (v0.1.2, .omni & .scroll associations)
├── language-configuration.json    # Brackets, auto-closing pairs, comment syntax
├── snippets/
│   └── omnicode.json             # 4-Block auto-population snippets (omni-file & omni-scroll)
└── syntaxes/
    └── omnicode.tmLanguage.json  # TextMate syntax grammar scopes & keyword rules
```

Installation

```bash
# Package the VSIX extension
npx -y @vscode/vsce package --allow-missing-repository

# Install into Antigravity IDE / VS Code
code --install-extension editors/vscode/omnicode-language-0.1.2.vsix
```

---

2. Core Capabilities & Grammar

Morphological Boundary Highlighting

Visually distinguishes all six ontological boundary types to eliminate bracket blindness:

* `[ ... ]` : **Denotation** (Fixed definitions, physical registers)
* `< ... >` : **Invariant** (Immutable axioms, process types)
* `{ ... }` : **Connotation** (Ambient context, scoped universes)
* `{ ... ]` : **Top-Down** (Context dictates definition / Absorber)
* `[ ... }` : **Bottom-Up** (Definition emerges from context / Generator)
* `( ... )` : **Aside** (Machine-readable intent and AI reasoning traces)

3D Coordinate Operator Colorization

* **Y-Axis (Being/Identity):** `=`, `!=`, `~=` / `≈`
* **X-Axis (Dynamics/Causality):** `-->` (Act upon forward), `<--` (Retroactive feedback/rollback)
* **Z-Axis (Scale/Hierarchy):** `^--` (Builds to / Layer up), `v--` (Breaks down from / Layer down)
* **Connectives:** `!` (Adversative caveat), `::` (And), `:` (In relation to), `|` (Or), `--` (Not)

Balanced Ternary Truth Literals

* `+1`, `YES`, `PROCEED` (Monotonic commit)
* `0`, `OK`, `AWAIT` (Asynchronous yield / equilibrium)
* `-1`, `NO`, `ROLLBACK` (State unwinding)
* `NULL`, `VOID`, `HALT` (Terminal trap)

Universal 4-Block Auto-Population Snippets

* `omni-file` / `omni-4block`: Complete compliant `.omni` module template with interactive tab stops.
* `omni-scroll` / `scroll`: Complete compliant `.scroll` episodic trace and learning template.
* `block-1`: Scaffolds Block 1: Metadata Block.
* `block-2`: Scaffolds Block 2: Opening Block.
* `block-3`: Scaffolds Block 3: Body Block.
* `block-4`: Scaffolds Block 4: Closing Block.

```

```
## 3. Verified Extension State & Invariants

* **Language ID:** `omnicode`
* **File Associations:** `*.omni`, `*.scroll`
* **Grammar Version:** `0.1.2`
* **Editor Integration:** Tested on VS Code 1.75+ and Antigravity IDE.
```

<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/editors/vscode/extension.js

JS | 93 LINES | 306 WORDS | 3.9 KB

```
// ============================================================================
// OmniCode VS Code Extension: Hover & Docstring Subsystem
// ============================================================================
// Connects the editor to the native OmniCode language engine (CLI & HTTP).
// Delivers 3-Tier composite hovercards bridging high-level .tape process ribbons
// down to low-level .omni invariant contracts and physical silicon execution states.
// ============================================================================

const vscode = require('vscode');
const { execFile } = require('child_process');
const http = require('http');

/**
 * Activates the OmniCode VS Code Extension.
 * Registers the HoverProvider for .omni, .scroll, and .tape files.
 * @param {vscode.ExtensionContext} context
 */
function activate(context) {
    const hoverProvider = vscode.languages.registerHoverProvider('omnicode', {
        provideHover(document, position, token) {
            const range = document.getWordRangeAtPosition(position);
            if (!range) {
                return null;
            }

            const symbol = document.getText(range);
            const line = position.line + 1;
            const col = position.character + 1;
            const filePath = document.fileName;

            return new Promise((resolve) => {
                // Method 1: Try local omnicode CLI binary directly
                execFile(
                    'omnicode',
                    ['hover', filePath, String(line), String(col), '--json'],
                    { timeout: 3000 },
                    (err, stdout) => {
                        if (!err && stdout) {
                            try {
                                const data = JSON.parse(stdout.trim());
                                if (data && data.markdown) {
                                    const md = new vscode.MarkdownString(data.markdown);
                                    md.isTrusted = true;
                                    md.supportHtml = true;
                                    return resolve(new vscode.Hover(md, range));
                                }
                            } catch (_e) {
                                // Fall through to HTTP server fallback
                            }
                        }

                        // Method 2: Fallback to running OmniCode Studio / GUI server (default port 8080)
                        const url = `http://127.0.0.1:8080/api/hover?
file=${encodeURIComponent(filePath)}&line=${line}&col=${col}`;
                        const req = http.get(url, (res) => {
                            let body = '';
                            res.on('data', (chunk) => (body += chunk));
                            res.on('end', () => {
                                try {
                                    const data = JSON.parse(body);
                                    if (data && data.markdown) {
                                        const md = new vscode.MarkdownString(data.markdown);
                                        md.isTrusted = true;
                                        md.supportHtml = true;
                                        return resolve(new vscode.Hover(md, range));
                                    }
                                } catch (_err) {
                                    // Ignored
                                }
                                resolve(null);
                            });
                        });

```

```
                        req.on('error', () => resolve(null));
                        req.setTimeout(1000, () => {
                            req.destroy();
                            resolve(null);
                        });
                    }
                );
            });
        }
    });

    context.subscriptions.push(hoverProvider);
}

function deactivate() {}

module.exports = {
    activate,
    deactivate,
};
```

<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/editors/vscode/language-configuration.json

JSON | 30 LINES | 76 WORDS | 0.5 KB

```
{
  "comments": {
    "lineComment": "#"
  },
  "brackets": [
    ["[", "]"],
    ["<", ">"],
    ["{", "}"],
    ["{", "]"],
    ["[", "}"],
    ["(", ")"]
  ],
  "autoClosingPairs": [
    { "open": "[", "close": "]" },
    { "open": "<", "close": ">" },
    { "open": "{", "close": "}" },
    { "open": "(", "close": ")" },
    { "open": "\"", "close": "\"" },
    { "open": "'", "close": "'" }
  ],
  "surroundingPairs": [
    ["[", "]"],
    ["<", ">"],
    ["{", "}"],
    ["(", ")"],
    ["\"", "\""],
    ["'", "'"]
  ]
}
```

<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/editors/vscode/omnicode-language-0.1.0.vsix

DOC/BINARY | 1 LINES | 5 WORDS | 2.9 KB

[Binary Artifact: omnicode-language-0.1.0.vsix (2.9 KB)]

<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/editors/vscode/omnicode-language-0.1.1.vsix

DOC/BINARY | 1 LINES | 5 WORDS | 4.0 KB

[Binary Artifact: omnicode-language-0.1.1.vsix (4.0 KB)]

<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/editors/vscode/omnicode-language-0.1.2.vsix

DOC/BINARY | 1 LINES | 5 WORDS | 6.4 KB

[Binary Artifact: omnicode-language-0.1.2.vsix (6.4 KB)]

<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/editors/vscode/package.json

JSON | 54 LINES | 92 WORDS | 1.1 KB

```json
{
  "name": "omnicode-language",
  "displayName": "OmniCode Language Support",
  "description": "Syntax highlighting, boundary morphology, and 4-block architecture support for the OmniCode language",
  "version": "0.1.3",
  "publisher": "omnicode",
  "main": "./extension.js",
  "activationEvents": [
    "onLanguage:omnicode"
  ],
  "engines": {
    "vscode": "^1.75.0"
  },
  "categories": [
    "Programming Languages",
    "Snippets"
  ],
  "contributes": {
    "languages": [
      {
        "id": "omnicode",
        "aliases": [
          "OmniCode",
          "omnicode",
          "omni",
          "OmniCode Scroll",
          "scroll",
          "OmniCode Tape",
          "tape"
        ],
        "extensions": [
          ".omni",
          ".scroll",
          ".tape"
        ],
        "configuration": "./language-configuration.json"
      }
    ],
    "grammars": [
      {
        "language": "omnicode",
        "scopeName": "source.omnicode",
        "path": "./syntaxes/omnicode.tmLanguage.json"
      }
    ],
    "snippets": [
      {
        "language": "omnicode",
        "path": "./snippets/omnicode.json"
      }
    ]
  }
}
```

<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/editors/vscode/syntaxes/omnicode.tmLanguage.json

JSON | 321 LINES | 529 WORDS | 8.7 KB

```
{
  "$schema": "https://raw.githubusercontent.com/martinring/tmlanguage/master/tmlanguage.json",
  "name": "OmniCode",
  "scopeName": "source.omnicode",
  "patterns": [
    { "include": "#comments" },
    { "include": "#boundary_casts" },
    { "include": "#primes" },
    { "include": "#affixed_identifiers" },
    { "include": "#asides" },
    { "include": "#strings" },
    { "include": "#ternary_literals" },
    { "include": "#vectors" },
    { "include": "#being_relations" },
    { "include": "#connectives" },
    { "include": "#boundaries" },
    { "include": "#numbers" },
    { "include": "#keywords" },
    { "include": "#registers" },
    { "include": "#categories" },
    { "include": "#instances" }
  ],
  "repository": {
    "comments": {
      "patterns": [
        {
          "name": "comment.line.number-sign.omnicode",
          "match": "#.*$"
        }
      ]
    },
    "boundary_casts": {
      "patterns": [
        {
          "name": "keyword.operator.boundary_cast.omnicode",
          "match": "\\[\\+\\]|\\[\\-\\]|<\\+>|<\\->|\\{\\+\\}|\\{\\-\\}"
        }
      ]
    },
    "primes": {
      "patterns": [
        {
          "name": "variable.other.prime.omnicode",
          "match": "\\b[a-zA-Z_][a-zA-Z0-9_]*`+"
        }
      ]
    },
    "affixed_identifiers": {
      "patterns": [
        {
          "name": "variable.other.affixed.abstract.omnicode",
          "match": "\\+[a-zA-Z_][a-zA-Z0-9_]*|[a-zA-Z_][a-zA-Z0-9_]*\\+"
        },
        {
          "name": "variable.other.affixed.concrete.omnicode",
          "match": "\\-[a-zA-Z_][a-zA-Z0-9_]*|[a-zA-Z_][a-zA-Z0-9_]*\\-"
        }
      ]
    },
    "asides": {
      "patterns": [
        {
          "name": "meta.aside.omnicode",
          "begin": "\\(",
          "beginCaptures": {
            "0": { "name": "punctuation.definition.aside.begin.omnicode" }
          },
          "end": "\\)",
          "endCaptures": {
            "0": { "name": "punctuation.definition.aside.end.omnicode" }
          },
          "patterns": [
            {
```

```
            "name": "string.unquoted.aside.intent.omnicode",
            "match": "[^()]+"
          }
        ]
      }
    ]
  },
  "strings": {
    "patterns": [
      {
        "name": "string.quoted.double.omnicode",
        "begin": "\"",
        "end": "\"",
        "patterns": [
          {
            "name": "constant.character.escape.omnicode",
            "match": "\\\\."
          }
        ]
      },
      {
        "name": "string.quoted.single.omnicode",
        "begin": "'",
        "end": "'|$",
        "patterns": [
          {
            "name": "constant.character.escape.omnicode",
            "match": "\\\\."
          }
        ]
      }
    ]
  },
  "ternary_literals": {
    "patterns": [
      {
        "name": "constant.language.ternary.proceed.omnicode",
        "match": "\\+1\\b|\\b(YES|PROCEED)\\b"
      },
      {
        "name": "constant.language.ternary.rollback.omnicode",
        "match": "-1\\b|\\b(NO|ROLLBACK)\\b"
      },
      {
        "name": "constant.language.ternary.await.omnicode",
        "match": "\\b0\\b|\\b(OK|AWAIT)\\b"
      },
      {
        "name": "constant.language.ternary.halt.omnicode",
        "match": "\\b(NULL|VOID|HALT)\\b"
      }
    ]
  },
  "vectors": {
    "patterns": [
      {
        "name": "keyword.operator.vector.scale.diagonal.major.omnicode",
        "match": "/-\\^"
      },
      {
        "name": "keyword.operator.vector.scale.diagonal.minor.omnicode",
        "match": "\\\\-v"
      },
      {
        "name": "keyword.operator.vector.scale.diagonal.great.omnicode",
        "match": "\\\\-\\^"
      },
      {
        "name": "keyword.operator.vector.scale.diagonal.small.omnicode",
        "match": "/-v"
      },
      {
        "name": "keyword.operator.vector.quadrant.omnicode",
        "match": "/\\^|\\\\v|\\^\\\\|v/"
      },
      {
        "name": "keyword.operator.vector.causality.forward.omnicode",
        "match": "-->|->"
```

```
        },
        {
          "name": "keyword.operator.vector.causality.backward.omnicode",
          "match": "<--|<-"
        },
        {
          "name": "keyword.operator.vector.scale.up.omnicode",
          "match": "\\^--|--\\^|-\\^|↑"
        },
        {
          "name": "keyword.operator.vector.scale.down.omnicode",
          "match": "v--|--v|-v|↓"
        }
      ]
    },
    "being_relations": {
      "patterns": [
        {
          "name": "keyword.operator.relation.invariant_equivalence.omnicode",
          "match": "<=>"
        },
        {
          "name": "keyword.operator.relation.identity.omnicode",
          "match": "="
        },
        {
          "name": "keyword.operator.relation.distinction.omnicode",
          "match": "!="
        },
        {
          "name": "keyword.operator.relation.homomorphism.omnicode",
          "match": "~=|≈"
        },
        {
          "name": "keyword.operator.relation.comparison.omnicode",
          "match": "<=|>=|<|>"
        }
      ]
    },
    "connectives": {
      "patterns": [
        {
          "name": "keyword.operator.connective.caveat.omnicode",
          "match": "!"
        },
        {
          "name": "keyword.operator.connective.and.omnicode",
          "match": "::"
        },
        {
          "name": "keyword.operator.connective.relation.omnicode",
          "match": ":"
        },
        {
          "name": "keyword.operator.connective.or.omnicode",
          "match": "\\|"
        },
        {
          "name": "keyword.operator.connective.not.omnicode",
          "match": "--"
        },
        {
          "name": "keyword.operator.connective.bound_pair.omnicode",
          "match": "&"
        },
        {
          "name": "keyword.operator.connective.percent.omnicode",
          "match": "%"
        },
        {
          "name": "keyword.operator.address.mmio.omnicode",
          "match": "::@|@"
        },
        {
          "name": "keyword.operator.range.omnicode",
          "match": "\\.\\."
        }
      ]
```

```
    },
    "boundaries": {
      "patterns": [
        {
          "name": "punctuation.definition.boundary.denotation.omnicode",
          "match": "\\[|\\]"
        },
        {
          "name": "punctuation.definition.boundary.invariant.omnicode",
          "match": "<|>"
        },
        {
          "name": "punctuation.definition.boundary.connotation.omnicode",
          "match": "\\{|\\}"
        }
      ]
    },
    "numbers": {
      "patterns": [
        {
          "name": "constant.numeric.hex.omnicode",
          "match": "\\b0x[0-9a-fA-F]+\\b"
        },
        {
          "name": "constant.numeric.omnicode",
          "match": "\\b\\d+(\\.\\d+)?\\b"
        }
      ]
    },
    "registers": {
      "patterns": [
        {
          "name": "variable.language.register.omnicode",
          "match": "\\b(HEAD|HOME|SPOOL|IT|SELF|CALLER|OTHER|STATE|TURN|BOUND|FROM|INTO|DISCARD)\\b"
        }
      ]
    },
    "keywords": {
      "patterns": [
        {
          "name": "keyword.control.block.omnicode",
          "match": "\\b(Module|Scroll|Opening|Body|Closing)\\b"
        },
        {
          "name": "keyword.other.declaration.omnicode",
          "match":
"\\b(import|requires|preconditions|verify|commit_status|user_prompt|synthesized_reel|machine_utterance|repair_entry|failpoint_rec
ord|version|system|author|safety|semantics|timestamp|session_id|execution_duration_micros|cognitive_mode|Witness|Nature)\\b"
        },
        {
          "name": "keyword.control.loop.verb.omnicode",
          "match":
"\\b(LOOP|WHEN|GOVERN|GUARD|TRAP|INSCRIBE|TRANSDUCE|PROJECT|SEQUENCE|ENCAPSULATE|LOCATE|BIND|ROUTE|SET_ASIDE|TRANSITION|CALCULATE
|MEASURE|MODULATE|REPEAT|PLAY|PAUSE|REWIND|STOP)\\b"
        },
        {
          "name": "keyword.control.tape.omnicode",
          "match": "\\b(Tape|Track)\\b"
        },
        {
          "name": "keyword.other.clause.tape.omnicode",
          "match": "\\b(WHO|WHAT|WHEN|WHERE|WHY|HOW)\\b"
        },
        {
          "name": "entity.name.function.roll.omnicode",
          "match": "\\broll_\\d+\\b|\\bROLL\\b"
        },
        {
          "name": "keyword.operator.arithmetic.word.omnicode",
          "match": "\\b(plus|minus|times|divide|by|add|sub|mult|div)\\b"
        },
        {
          "name": "keyword.operator.infer.omnicode",
          "match": "\\?"
        }
      ]
    },
    "categories": {
```

```
      "patterns": [
        {
          "name": "entity.name.type.category.omnicode",
          "match": "\\b[A-Z][a-zA-Z0-9_]*\\b"
        }
      ]
    },
    "instances": {
      "patterns": [
        {
          "name": "variable.other.instance.omnicode",
          "match": "\\b[a-z_][a-zA-Z0-9_]*\\b"
        }
      ]
    }
  }
}
```

<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/editors/vscode/snippets/omnicode.json

JSON | 163 LINES | 611 WORDS | 6.5 KB

```
{
  "OmniCode 4-Block Template": {
    "prefix": ["omni-file", "omni-4block", "omni"],
    "description": "Auto-populate standard OmniCode 4-Block file architecture",
    "body": [
      "# ================================================================================",
      "# BLOCK 1: METADATA BLOCK",
      "# ( File Identity, Version, Author, Intent Aside, and Invariants )",
      "# ================================================================================",
      "Module < ${1:ModuleName} > ( ${2:Module intent rationale} ) {",
      "    version: < \"${3:0.1.0}\" >,",
      "    author: < \"${4:ProfessorSeanEX}\" >,",
      "    system: < OmniCode.Core >,",
      "    safety: < MemorySafe :: Deterministic >",
      "}",
      "",
      "# Invariant Axioms & Types",
      "Type_${1:ModuleName} < +1 > ( Invariant definition )",
      "",
      "",
      "# ================================================================================",
      "# BLOCK 2: OPENING BLOCK",
      "# ( Environment Setup, Preconditions, Scoped Context, and Declarations )",
      "# ================================================================================",
      "Opening {",
      "    requires: [ ${5:MemoryAligned.64} ],",
      "    preconditions: [ +1 ]",
      "}",
      "",
      "# Initial state instances",
      "${6:state} [ +1 ] ( Initialized state instance )",
      "",
      "",
      "# ================================================================================",
      "# BLOCK 3: BODY BLOCK",
      "# ( Core Transformations, Causality Vectors, Logic, and Caveat Pipelines )",
      "# ================================================================================",
      "Body [",
      "    # Forward action: state acts upon transformation",
      "    ${6:state} --> ${7:next_state} [ +1 ]",
      "    ${7:next_state} ! [ ${6:state} = +1 ] ( Caveat verification )",
      "]",
      "",
      "",
      "# ================================================================================",
      "# BLOCK 4: CLOSING BLOCK",
      "# ( Postconditions, Invariant Proofs, Teardown, and Commit Status )",
      "# ================================================================================",
      "Closing {",
      "    verify: [ ${7:next_state} = +1 ],",
      "    commit_status: [ +1 ] ( Commit verified state )",
      "}",
      ""
    ]
  },
  "Metadata Block": {
    "prefix": "block-1",
    "description": "Insert Block 1: Metadata Block",
    "body": [
      "# ================================================================================",
      "# BLOCK 1: METADATA BLOCK",
      "# ================================================================================",
      "Module < ${1:Name} > ( ${2:Intent} ) {",
      "    version: < \"0.1.0\" >",
      "}",
      ""
    ]
  },
  "Opening Block": {
    "prefix": "block-2",
    "description": "Insert Block 2: Opening Block",
    "body": [
      "# ================================================================================",
```

```
      "# BLOCK 2: OPENING BLOCK",
      "# ================================================================================",
      "Opening {",
      "    requires: [ ${1:Prerequisites} ]",
      "}",
      ""
    ]
  },
  "Body Block": {
    "prefix": "block-3",
    "description": "Insert Block 3: Body Block",
    "body": [
      "# ================================================================================",
      "# BLOCK 3: BODY BLOCK",
      "# ================================================================================",
      "Body [",
      "    ${1:source} --> ${2:target}",
      "]",
      ""
    ]
  },
  "Closing Block": {
    "prefix": "block-4",
    "description": "Insert Block 4: Closing Block",
    "body": [
      "# ================================================================================",
      "# BLOCK 4: CLOSING BLOCK",
      "# ================================================================================",
      "Closing {",
      "    verify: [ ${1:condition} = +1 ],",
      "    commit_status: [ +1 ]",
      "}",
      ""
    ]
  },
  "OmniCode Episodic Scroll Template": {
    "prefix": ["omni-scroll", "scroll"],
    "description": "Auto-populate standard OmniCode Episodic Cognitive Scroll (.scroll)",
    "body": [
      "# ================================================================================",
      "# BLOCK 1: METADATA BLOCK (THE COGNITIVE SCROLL)",
      "# ( Execution Trace ID, Prompt Intent, Inferred Types, and Neural Snapshot )",
      "# ================================================================================",
      "Scroll < EpistemicTrace.${1:SessionId} > ( ${2:User Prompt Intent} ) {",
      "    version: < \"1.0.0\" >,",
      "    system: < OmniCode.AI.Model >,",
      "    timestamp: < \"${CURRENT_SECONDS_UNIX}\" >,",
      "    session_id: < \"${1:SessionId}\" >,",
      "    cognitive_mode: < NeuralSymbolicTransduction >",
      "}",
      "",
      "# ================================================================================",
      "# BLOCK 2: OPENING BLOCK (EXPERIENCE INITIALIZATION)",
      "# ( Hardware Invariants, Preconditions, and Initial Context )",
      "# ================================================================================",
      "Opening {",
      "    import: [ Core.Arithmetic :: Core.Logic :: Core.Vectors ],",
      "    preconditions: [ +1 ]",
      "}",
      "",
      "# Original Cognitive Discourse Prompt:",
      "user_prompt < \"${2:User Prompt Intent}\" >",
      "",
      "# ================================================================================",
      "# BLOCK 3: BODY BLOCK (CHRONICLE OF EXECUTION & RESTORATION)",
      "# ( Replay Tape: Synthesized Program, Sensory Telemetry & Immune Interventions )",
      "# ================================================================================ ",
      "Body [",
      "    # Immune System Interventions (Restorator):",
      "    repair_entry [ L1: CleanExecution --> \"Zero repairs required.\" ]",
      "",
      "    # Executed Turing Reel:",
      "    synthesized_reel [",
      "        ${3:state} [ +1 ]",
      "    ]",
      "]",
      "",
      "# ================================================================================",
```

```
      "# BLOCK 4: CLOSING BLOCK (EPISODIC COMMIT & LEARNING PROOF)",
      "# ( Verification Proof, Final Quad-State, and Conversational Synthesis )",
      "# ============================================================================== ",
      "Closing {",
      "    commit_status [ <PROCEED> ],",
      "    machine_utterance ( \"${4:Execution succeeded to +1:PROCEED.}\" )",
      "}",
      ""
    ]
  }
}
```

<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/gui/app.js JS 509 LINES 1537 WORDS 20.4 KB

```
// ================================================================================
// OMNICODE STUDIO & COGNITIVE TERMINAL APPLICATION LOGIC
// Grounded in Balanced Ternary Quad-State & Biblical Moral Ontology
// ================================================================================

document.addEventListener('DOMContentLoaded', () => {
    initNavigation();
    initPromptStudio();
    initTerminalGateway();
    initLexiconExplorer();
    initEpisodicScrolls();
    initHoverInspector();
});

// --------------------------------------------------------------------------------
// 1. NAVIGATION TAB SWITCHING
// --------------------------------------------------------------------------------
function initNavigation() {
    const tabs = document.querySelectorAll('.nav-tab');
    const panels = document.querySelectorAll('.view-panel');

    tabs.forEach(tab => {
        tab.addEventListener('click', () => {
            const targetViewId = tab.getAttribute('data-view');

            tabs.forEach(t => t.classList.remove('active'));
            panels.forEach(p => p.classList.remove('active'));

            tab.classList.add('active');
            const targetPanel = document.getElementById(targetViewId);
            if (targetPanel) {
                targetPanel.classList.add('active');
            }

            // Trigger data load if needed
            if (targetViewId === 'view-lexicon') {
                loadLexiconData();
            } else if (targetViewId === 'view-scrolls') {
                loadScrollsData();
            }
        });
    });
}

// --------------------------------------------------------------------------------
// 2. AI PROMPT STUDIO CONTROLLER
// --------------------------------------------------------------------------------
function initPromptStudio() {
    const promptInput = document.getElementById('prompt-input');
    const submitBtn = document.getElementById('btn-submit-prompt');
    const promptChips = document.querySelectorAll('.prompt-chip');
    const quadstateBadge = document.getElementById('studio-quadstate-badge');
    const stateLabel = document.getElementById('state-label');
    const responseContent = document.getElementById('response-content');
    const synthesizedCode = document.getElementById('synthesized-code');
    const badgeSession = document.getElementById('badge-session-id');
    const linkScroll = document.getElementById('link-scroll');
    const linkLog = document.getElementById('link-log');
    const copyCodeBtn = document.getElementById('btn-copy-code');

    // Quick Prompt Chips
    promptChips.forEach(chip => {
        chip.addEventListener('click', () => {
            const promptText = chip.getAttribute('data-prompt');
            promptInput.value = promptText;
            promptInput.focus();
        });
    });

    // Copy Code Button
    copyCodeBtn.addEventListener('click', () => {
        const codeText = synthesizedCode.textContent;
        navigator.clipboard.writeText(codeText).then(() => {
            const originalText = copyCodeBtn.textContent;
```

```
            copyCodeBtn.textContent = 'Copied!';
            setTimeout(() => { copyCodeBtn.textContent = originalText; }, 1800);
        });
    });

    // Submit Prompt Execution
    submitBtn.addEventListener('click', async () => {
        const prompt = promptInput.value.trim();
        if (!prompt) return;

        // Set Loading State
        submitBtn.disabled = true;
        submitBtn.querySelector('.btn-text').textContent = 'Deliberating...';
        stateLabel.textContent = 'DELIBERATING (0:AWAIT)';
        quadstateBadge.className = 'quadstate-indicator';

        try {
            const res = await fetch('/api/prompt', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ prompt })
            });

            if (!res.ok) {
                const errData = await res.json().catch(() => ({ error: 'Unknown server error' }));
                throw new Error(errData.error || `HTTP ${res.status}`);
            }

            const data = await res.json();
            renderPromptResult(data);
        } catch (err) {
            console.error('Prompt evaluation error:', err);
            responseContent.innerHTML = `<p style="color: var(--state-rollback);">✗ Error evaluating prompt:
${err.message}</p>`;
            stateLabel.textContent = 'ERROR (NULL:HALT)';
            quadstateBadge.className = 'quadstate-indicator halt';
        } finally {
            submitBtn.disabled = false;
            submitBtn.querySelector('.btn-text').textContent = 'Send to AI Cortex';
        }
    });

    function renderPromptResult(data) {
        // 1. Update QuadState Badge
        const state = (data.state || 'PROCEED').toUpperCase();
        quadstateBadge.className = 'quadstate-indicator';

        if (state === 'PROCEED') {
            quadstateBadge.classList.add('proceed');
            stateLabel.textContent = '+1: PROCEED';
        } else if (state === 'ROLLBACK') {
            quadstateBadge.classList.add('rollback');
            stateLabel.textContent = '-1: ROLLBACK (CAVEAT VETO)';
        } else if (state === 'AWAIT') {
            quadstateBadge.classList.add('await');
            stateLabel.textContent = '0: AWAIT (DISCERNMENT)';
        } else {
            quadstateBadge.classList.add('halt');
            stateLabel.textContent = 'NULL: HALT (TERMINAL TRAP)';
        }

        // 2. Session and Response Content
        badgeSession.textContent = `Session: ${data.session_id || 'active'}`;
        responseContent.textContent = data.response || 'No natural discourse generated.';

        // 3. Synthesized Code Block
        synthesizedCode.textContent = data.omnicode || '// Zero code generated.';

        // 4. File Links
        linkScroll.textContent = `📜 Scroll: ${data.scroll || 'Recorded'}`;
        linkLog.textContent = `📋 Log: ${data.log || 'Recorded'}`;
    }
}

// ---------------------------------------------------------------------------
// 3. CLI TERMINAL GATEWAY CONTROLLER
// ---------------------------------------------------------------------------
function initTerminalGateway() {
```

```
    const termInput = document.getElementById('terminal-cmd-input');
    const runBtn = document.getElementById('btn-run-cmd');
    const termStream = document.getElementById('terminal-stream');
    const clearBtn = document.getElementById('btn-clear-terminal');
    const shortcutBtns = document.querySelectorAll('.term-btn');
    const termWindow = document.getElementById('terminal-window');

    const desktopBtn = document.getElementById('btn-desktop-launch');
    if (desktopBtn) {
        desktopBtn.addEventListener('click', () => {
            executeTerminalCommand('omnicode desktop');
        });
    }

    clearBtn.addEventListener('click', () => {
        termStream.innerHTML = '';
    });

    shortcutBtns.forEach(btn => {
        btn.addEventListener('click', () => {
            const cmd = btn.getAttribute('data-cmd');
            termInput.value = cmd;
            executeTerminalCommand(cmd);
        });
    });

    runBtn.addEventListener('click', () => {
        const cmd = termInput.value.trim();
        if (!cmd) return;
        executeTerminalCommand(cmd);
        termInput.value = '';
    });

    termInput.addEventListener('keydown', (e) => {
        if (e.key === 'Enter') {
            const cmd = termInput.value.trim();
            if (!cmd) return;
            executeTerminalCommand(cmd);
            termInput.value = '';
        }
    });

    async function executeTerminalCommand(command) {
        appendTerminalLine(`omnicode> ${command}`, 'term-line-cmd');

        try {
            const res = await fetch('/api/terminal', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ command })
            });

            const data = await res.json();
            if (data.stdout) {
                appendTerminalLine(data.stdout, 'term-line-stdout');
            }
            if (data.stderr) {
                appendTerminalLine(data.stderr, 'term-line-stderr');
            }
        } catch (err) {
            appendTerminalLine(`Error executing command: ${err.message}`, 'term-line-stderr');
        }

        termWindow.scrollTop = termWindow.scrollHeight;
    }

    function appendTerminalLine(text, cssClass) {
        const div = document.createElement('div');
        div.className = cssClass;
        div.textContent = text;
        termStream.appendChild(div);
    }
}

// ------------------------------------------------------------------------
// 4. COVENANT MANIFEST & BIBLICAL LEXICON CONTROLLER
// ------------------------------------------------------------------------
let allLexiconEntries = [];
```

```
230
231 function initLexiconExplorer() {
232     const searchInput = document.getElementById('lexicon-search-input');
233     const filterBtns = document.querySelectorAll('.filter-btn');
234
235     searchInput.addEventListener('input', () => {
236         filterAndRenderLexicon();
237     });
238
239     filterBtns.forEach(btn => {
240         btn.addEventListener('click', () => {
241             filterBtns.forEach(b => b.classList.remove('active'));
242             btn.classList.add('active');
243             filterAndRenderLexicon();
244         });
245     });
246 }
247
248 async function loadLexiconData() {
249     const entriesGrid = document.getElementById('lexicon-entries-grid');
250     if (allLexiconEntries.length > 0) return; // Already cached
251
252     entriesGrid.innerHTML = '<p class="placeholder-text">Loading biblical knowledge base from offline memory...</p>';
253
254     try {
255         const res = await fetch('/api/lexicon');
256         const data = await res.json();
257         allLexiconEntries = data.entries || [];
258         filterAndRenderLexicon();
259     } catch (err) {
260         console.error('Failed to load lexicon:', err);
261         entriesGrid.innerHTML = `<p style="color: var(--state-rollback);">Failed to load lexicon: ${err.message}</p>`;
262     }
263 }
264
265 function filterAndRenderLexicon() {
266     const entriesGrid = document.getElementById('lexicon-entries-grid');
267     const searchInput = document.getElementById('lexicon-search-input');
268     const query = searchInput.value.trim().toLowerCase();
269     const activeFilterBtn = document.querySelector('.filter-btn.active');
270     const selectedDomain = activeFilterBtn ? activeFilterBtn.getAttribute('data-domain') : 'all';
271
272     const filtered = allLexiconEntries.filter(entry => {
273         const matchesQuery = query === '' ||
274             entry.term.toLowerCase().includes(query) ||
275             entry.definition.toLowerCase().includes(query) ||
276             entry.kjv.toLowerCase().includes(query) ||
277             entry.web.toLowerCase().includes(query);
278
279         const matchesDomain = selectedDomain === 'all' || entry.domain === selectedDomain;
280
281         return matchesQuery && matchesDomain;
282     });
283
284     if (filtered.length === 0) {
285         entriesGrid.innerHTML = `<p class="placeholder-text">No concepts found matching '${query}' in selected domain.
</p>`;
286         return;
287     }
288
289     entriesGrid.innerHTML = filtered.map(e => `
290         <article class="entry-card">
291             <div class="entry-header">
292                 <span class="entry-term">${escapeHtml(e.term)}</span>
293                 <span class="entry-domain">${escapeHtml(e.domain)}</span>
294             </div>
295             <p class="entry-def">${escapeHtml(e.definition)}</p>
296             <div class="entry-witnesses">
297                 <p><span class="witness-title">KJV (1611):</span> ${escapeHtml(e.kjv)}</p>
298                 <p><span class="witness-title">WEB:</span> ${escapeHtml(e.web)}</p>
299             </div>
300             <p class="entry-guidance">⚖️ ${escapeHtml(e.guidance)}</p>
301         </article>
302     `).join('');
303 }
304
305 // ---------------------------------------------------------------------------
306 // 5. EPISODIC SCROLLS CONTROLLER
```

```
// ---------------------------------------------------------------------------
function initEpisodicScrolls() {
    const refreshBtn = document.getElementById('btn-refresh-scrolls');
    refreshBtn.addEventListener('click', () => {
        loadScrollsData();
    });
}

async function loadScrollsData() {
    const tbody = document.getElementById('episodes-tbody');
    tbody.innerHTML = '<tr><td colspan="6" style="text-align:center; padding: 2rem;">Loading memory bank...</td></tr>';

    try {
        const res = await fetch('/api/scrolls');
        const data = await res.json();

        // Update Metrics
        document.getElementById('metric-total').textContent = data.total || 0;
        document.getElementById('metric-proceed').textContent = data.proceed || 0;
        document.getElementById('metric-await').textContent = data.await || 0;
        document.getElementById('metric-rollback').textContent = data.rollback || 0;
        document.getElementById('metric-halt').textContent = data.halt || 0;
        document.getElementById('metric-repairs').textContent = data.repairs_learned || 0;

        // Update Episodes Table
        const episodes = data.episodes || [];
        if (episodes.length === 0) {
            tbody.innerHTML = '<tr><td colspan="6" style="text-align:center; padding: 2rem; color: var(--text-muted);">No scrolls inscribed yet. Run prompts in Studio to record episodes.</td></tr>';
            return;
        }

        tbody.innerHTML = episodes.map(ep => {
            const stateClass = ep.state === 'PROCEED' ? 'state-proceed' :
                               ep.state === 'ROLLBACK' ? 'state-rollback' :
                               ep.state === 'AWAIT' ? 'state-await' : 'state-halt';
            return `
                <tr>
                    <td style="font-family: var(--font-mono); color: var(--cyan);">${escapeHtml(ep.session_id)}</td>
                    <td>${escapeHtml(ep.prompt)}</td>
                    <td><span class="state-chip ${stateClass}">+1:${escapeHtml(ep.state)}</span></td>
                    <td style="text-align: center;">${ep.repairs}</td>
                    <td style="text-align: center;">${ep.failpoints}</td>
                    <td style="font-family: var(--font-mono); font-size: 0.75rem; color: var(--text-muted);">${escapeHtml(ep.scroll)}</td>
                </tr>
            `;
        }).join('');
    } catch (err) {
        console.error('Failed to load scrolls:', err);
        tbody.innerHTML = `<tr><td colspan="6" style="color: var(--state-rollback); text-align: center;">Error: ${err.message}</td></tr>`;
    }
}

// Utility: HTML Escaping
function escapeHtml(str) {
    if (!str) return '';
    return String(str)
        .replace(/&/g, '&amp;')
        .replace(/</g, '&lt;')
        .replace(/>/g, '&gt;')
        .replace(/"/g, '&quot;')
        .replace(/'/g, '&#039;');
}

// ---------------------------------------------------------------------------
// 6. 3-TIER HOVER INSPECTOR CONTROLLER (.omni <-> .tape)
// ---------------------------------------------------------------------------
function initHoverInspector() {
    let tooltipEl = document.querySelector('.omni-hover-card');
    if (!tooltipEl) {
        tooltipEl = document.createElement('div');
        tooltipEl.className = 'omni-hover-card';
        tooltipEl.style.display = 'none';
        document.body.appendChild(tooltipEl);
    }

```

```
    let hideTimer = null;

    document.addEventListener('mouseover', async (e) => {
        const target = e.target;
        if (!target) return;

        const codeContainer = target.closest('pre, code, .code-editor, #synthesized-code');
        if (!codeContainer) {
            return;
        }

        const word = getWordUnderCursor(e);
        if (!word || word.length < 2 || /^\d+$/.test(word)) {
            return;
        }

        clearTimeout(hideTimer);

        try {
            const res = await fetch(`/api/hover?symbol=${encodeURIComponent(word)}`);
            if (!res.ok) return;
            const data = await res.json();
            if (!data || !data.symbol) return;

            renderHoverCard(tooltipEl, data, e.pageX, e.pageY);
        } catch (_err) {
            // Ignored
        }
    });

    document.addEventListener('mouseout', (e) => {
        if (tooltipEl.contains(e.relatedTarget)) return;
        hideTimer = setTimeout(() => {
            tooltipEl.style.display = 'none';
        }, 300);
    });

    tooltipEl.addEventListener('mouseenter', () => clearTimeout(hideTimer));
    tooltipEl.addEventListener('mouseleave', () => {
        tooltipEl.style.display = 'none';
    });
}

function getWordUnderCursor(event) {
    if (document.caretRangeFromPoint) {
        const range = document.caretRangeFromPoint(event.clientX, event.clientY);
        if (range && range.startContainer && range.startContainer.nodeType === Node.TEXT_NODE) {
            const text = range.startContainer.textContent;
            const offset = range.startOffset;
            const match = text.slice(0, offset).match(/[a-zA-Z0-9_]+$/);
            const afterMatch = text.slice(offset).match(/^[a-zA-Z0-9_]+/);
            const before = match ? match[0] : '';
            const after = afterMatch ? afterMatch[0] : '';
            return before + after;
        }
    }
    return null;
}

function renderHoverCard(cardEl, data, x, y) {
    const omni = data.omni_definition;
    const tape = data.tape_context;

    const rankBadge = omni
        ? `<span class="omni-hover-rank-badge rank-${omni.rank}">Rank ${omni.rank}: ${escapeHtml(omni.boundary_type)}
</span>`
        : (data.is_locally_redefined ? `<span class="omni-hover-rank-badge">Rank 2: Denotation</span>` : `<span
class="omni-hover-rank-badge">Tape Entity</span>`);

    let tapeHtml = '';
    if (tape) {
        tapeHtml = `
            <div class="omni-hover-tier">
                <div class="omni-hover-tier-title tier-tape">📜 1. High-Level Process Tape</div>
                <div class="omni-hover-prop"><span class="omni-hover-label">Ribbon:</span><span class="omni-hover-
val">${escapeHtml(tape.tape_name)} (${escapeHtml(tape.track)})</span></div>
                <div class="omni-hover-prop"><span class="omni-hover-label">Active Roll:</span><span class="omni-hover-
val">${tape.roll_number}/${tape.total_rolls}</span></div>
```

```
                ${tape.roll_aside_intent ? `<div class="omni-hover-aside">"${escapeHtml(tape.roll_aside_intent)}"</div>` : ''}
                <div class="omni-hover-prop"><span class="omni-hover-label">Head Covenant:</span><span class="omni-hover-val">WHO ${escapeHtml(tape.head_who)} &rarr; WHAT ${escapeHtml(tape.head_what)}</span></div>
            </div>
        `;
    }

    let omniHtml = '';
    if (omni) {
        omniHtml = `
            <div class="omni-hover-tier">
                <div class="omni-hover-tier-title tier-omni">🔍 2. Under-The-Hood (.omni Blueprint)</div>
                <div class="omni-hover-prop"><span class="omni-hover-label">Grounded Source:</span><span class="omni-hover-val">${escapeHtml(omni.source_file)}:${omni.line}</span></div>
                <div class="omni-hover-prop"><span class="omni-hover-label">4-Block Origin:</span><span class="omni-hover-val">${escapeHtml(omni.block_kind || 'Universal Scope')}</span></div>
                ${omni.aside_intent ? `<div class="omni-hover-aside">"${escapeHtml(omni.aside_intent)}"</div>` : ''}
            </div>
        `;
    }

    const siliconHtml = `
        <div class="omni-hover-tier">
            <div class="omni-hover-tier-title tier-silicon">⚙️ 3. Machine Execution & Silicon State</div>
            <div class="omni-hover-prop"><span class="omni-hover-label">Partition Reservation:</span><span class="omni-hover-val">${tape ? tape.partition_size + ' cells' : '1024 cells'}</span></div>
            <div class="omni-hover-prop"><span class="omni-hover-label">Radix-3 Transducer:</span><span class="omni-hover-val">Dense Byte &harr; 5 Balanced Trits</span></div>
            <div class="omni-hover-prop"><span class="omni-hover-label">Silicon Truth State:</span><span class="omni-hover-val" style="color: var(--state-proceed);">+1:PROCEED (Deterministic Commit)</span></div>
        </div>
    `;

    cardEl.innerHTML = `
        <div class="omni-hover-header">
            <span class="omni-hover-symbol">${escapeHtml(data.symbol)}</span>
            ${rankBadge}
        </div>
        ${tapeHtml}
        ${omniHtml}
        ${siliconHtml}
    `;

    cardEl.style.display = 'block';
    const cardRect = cardEl.getBoundingClientRect();
    let left = x + 15;
    let top = y + 15;

    if (left + cardRect.width > window.innerWidth - 20) {
        left = x - cardRect.width - 15;
    }
    if (top + cardRect.height > window.innerHeight - 20) {
        top = y - cardRect.height - 15;
    }

    cardEl.style.left = `${Math.max(10, left)}px`;
    cardEl.style.top = `${Math.max(10, top)}px`;
}
```

<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/gui/cloud.html

HTML | 201 LINES | 718 WORDS | 10.8 KB

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="OmniCode Cloudflare Zero Trust & Oracle Cloud Edge Deployment — CreativeWorkzStudio LLC.">
    <title>Cloud &amp;amp; Oracle Edge Gateway — CreativeWorkzStudio LLC</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Fira+Code:wght@400;500;600;700&family=Outfit:wght@300;400;500;600;700;800&family=Cinzel:wght@500;700;800&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="/style.css">
</head>
<body>
    <div class="app-layout">
        <!-- GLOBAL NAVIGATION HEADER -->
        <header class="app-header" id="app-header">
            <div class="header-left">
                <a href="/studio" class="brand-container">
                    <span class="brand-symbol">Ω</span>
                    <div class="brand-text">
                        <span class="brand-title">OMNICODE</span>
                        <span class="brand-subtitle">CREATIVEWORKZSTUDIO LLC</span>
                    </div>
                </a>
                <div class="status-pill" id="status-pill" title="Engine runs 100% offline with zero cloud telemetry">
                    <span class="status-dot"></span>
                    <span class="status-text">OFFLINE COVENANT ACTIVE</span>
                </div>
            </div>

            <nav class="app-nav" aria-label="Main Navigation">
                <a href="/studio" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M12 2L2 7l10 5 10-5-10-5zM2 17l10 5 10-5M2 12l10 5 10-5"/></svg>
                    AI Studio
                </a>
                <a href="/terminal" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><polyline points="4 17 10 11 4 5"/><line x1="12" y1="19" x2="20" y2="19"/></svg>
                    CLI Terminal
                </a>
                <a href="/specs" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8z"/><polyline points="14 2 14 8 20 8"/></svg>
                    Language Specs
                </a>
                <a href="/model" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M12 2a8 8 0 0 0-8 8c0 3.3 2 6.2 5 7.4V20a2 2 0 0 0 2 2h2a2 2 0 0 0 2-2v-2.6c3-1.2 5-4.1 5-7.4a8 8 0 0 0-8-8z"/></svg>
                    AI Model
                </a>
                <a href="/lexicon" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20"/><path d="M6.5 2H20v20H6.5A2.5 2.5 0 0 1 4 19.5v-15A2.5 2.5 0 0 1 6.5 2z"/></svg>
                    Covenant Lexicon
                </a>
                <a href="/scrolls" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M19 21l-7-5-7 5V5a2 2 0 0 1 2-2h10a2 2 0 0 1 2 2z"/></svg>
                    Scrolls
                </a>
                <a href="/cloud" class="nav-tab active">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M18 10h-1.26A8 8 0 1 0 9 20h9a5 5 0 0 0 0-10z"/></svg>
                    Cloud &amp;amp; Edge
                </a>
                <a href="/download" class="nav-tab download-nav-btn">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4"/><polyline points="7 10 12 15 17 10"/><line x1="12" y1="15" x2="12" y2="3"/></svg>
                    Download Native
```

```
                </a>
            </nav>
        </header>

        <!-- CLOUD MAIN -->
        <main class="doc-container">
            <div class="doc-hero card-glass">
                <div class="hero-badge">INFRASTRUCTURE &amp;amp; EDGE GATEWAY</div>
                <h1>Cloudflare Zero Trust &amp;amp; Oracle Cloud Architecture</h1>
                <p class="hero-lead">
                    Deploy and expose OmniCode 24/7 across the globe using Oracle Cloud Infrastructure (OCI) compute instances and Cloudflare Zero Trust secure HTTPS edge tunnels.
                </p>
            </div>

            <div class="doc-grid">
                <!-- ORACLE VM CARD -->
                <div class="doc-card card-glass">
                    <div class="doc-card-header">
                        <span class="doc-icon">🌌</span>
                        <h2>Oracle Cloud Compute Instance</h2>
                    </div>
                    <p>Turnkey production deployment specifications configured for OmniCode:</p>
                    <table class="spec-table">
                        <tbody>
                            <tr>
                                <td><strong>Remote Host IP</strong></td>
                                <td><code>147.224.147.30</code> (SSH alias: <code>oracle</code>)</td>
                            </tr>
                            <tr>
                                <td><strong>Remote User</strong></td>
                                <td><code>ubuntu</code></td>
                            </tr>
                            <tr>
                                <td><strong>Authentication</strong></td>
                                <td><code>~/.ssh/oracle_key.pem</code> (Ed25519 / RSA)</td>
                            </tr>
                            <tr>
                                <td><strong>Target Directory</strong></td>
                                <td><code>/home/ubuntu/OmniCode</code></td>
                            </tr>
                            <tr>
                                <td><strong>OS Firewall</strong></td>
                                <td>TCP Port <code>8080</code> (Permitted via iptables-persistent)</td>
                            </tr>
                            <tr>
                                <td><strong>Systemd Service</strong></td>
                                <td><code>omnicode-web.service</code> (24/7 background operation)</td>
                            </tr>
                        </tbody>
                    </table>

                    <div class="doc-subcard">
                        <h3>Automated VM Synchronization</h3>
                        <p>To synchronize local changes and restart production services on the Oracle VM:</p>
                        <code>./scripts/sync_oracle_vm.sh "Deploy updates"</code>
                    </div>
                </div>

                <!-- CREATIVEWORKZSTUDIO GATEWAY -->
                <div class="doc-card card-glass">
                    <div class="doc-card-header">
                        <span class="doc-icon">🏢</span>
                        <h2>CreativeWorkzStudio LLC Gateway</h2>
                    </div>
                    <p>Licensed dynamic endpoint and local workstation infrastructure:</p>
                    <table class="spec-table">
                        <tbody>
                            <tr>
                                <td><strong>Licensed Domain</strong></td>
                                <td><code>omnicodestudio.duckdns.org</code></td>
                            </tr>
                            <tr>
                                <td><strong>Host WAN IP</strong></td>
                                <td><code>71.11.99.168</code></td>
                            </tr>
                            <tr>
                                <td><strong>Organization</strong></td>
```

```
                                <td>CreativeWorkzStudio LLC</td>
                            </tr>
                            <tr>
                                <td><strong>Direct Studio URL</strong></td>
                                <td><code>http://omnicodestudio.duckdns.org:8080</code></td>
                            </tr>
                            <tr>
                                <td><strong>DNS Zone File</strong></td>
                                <td><code>dns/omnicodestudio.com.zone</code></td>
                            </tr>
                        </tbody>
                    </table>

                    <div class="doc-subcard">
                        <h3>DNS Zone Import</h3>
                        <p>Import into Cloudflare DNS (DNS → Records → Import):</p>
                        <code>python3 dns/zone_manager.py inspect</code>
                    </div>
                </div>

                <!-- CLOUDFLARE TUNNEL CARD -->
                <div class="doc-card card-glass">
                    <div class="doc-card-header">
                        <span class="doc-icon">🌐</span>
                        <h2>Cloudflare Zero Trust Secure Tunnel</h2>
                    </div>
                    <p>Expose local port 8080 to the public internet securely without opening router ports:</p>
                    <ul class="spec-list">
                        <li><strong>Named Tunnel ID:</strong> <code>c346aa27-b4c1-41ba-815c-f66b2b84243b</code></li>
                        <li><strong>Tunnel Target:</strong> <code>c346aa27...cfargotunnel.com.</code></li>
                        <li><strong>Quick Tunnel Mode:</strong> Free ephemeral HTTPS URL
(<code>https://*.trycloudflare.com</code>).</li>
                        <li><strong>Edge TLS &amp;amp; DDoS Protection:</strong> Protected by Cloudflare global edge.</li>
                    </ul>

                    <div class="doc-subcard">
                        <h3>Launch Quick Edge Tunnel</h3>
                        <p>Execute the connector script to obtain an instant HTTPS domain:</p>
                        <code>./scripts/setup_cloudflare_tunnel.sh --quick</code>
                    </div>
                </div>
            </div>
        </main>

        <!-- FOOTER -->
        <footer class="app-footer">
            <div class="footer-left">
                <span class="footer-brand">Ω OMNICODE</span>
                <span class="footer-sep">&bull;</span>
                <span class="footer-copy">&copy; 2026 CreativeWorkzStudio LLC. All rights reserved.</span>
            </div>
            <div class="footer-center">
                <span>Infrastructure &bull; Oracle Cloud &bull; Cloudflare Edge &bull; DuckDNS</span>
            </div>
            <div class="footer-right">
                <span class="footer-pill">Edge Gateway</span>
                <span class="footer-pill">v0.1.0</span>
            </div>
        </footer>
    </div>
</body>
</html>
```

<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/gui/desktop_app.py

PY | 1182 LINES | 3250 WORDS | 44.9 KB

```python
#!/usr/bin/env python3
# ==============================================================================
# BLOCK 1: METADATA BLOCK
# ( CreativeWorkzStudio LLC — OmniCode Desktop GUI Studio & Terminal Shell )
# ==============================================================================
"""
OmniCode Desktop GUI Studio & Interactive Terminal Shell
A CreativeWorkzStudio LLC Project

Provides a native desktop interface for:
  1. Interactive GUI Terminal Shell for direct CLI execution
  2. AI Prompt Studio with real-time Balanced Ternary Quad-State deliberation
  3. Language & AI Architecture Reference Viewer
  4. Cloudflare Tunnel & Oracle Cloud VM Deployment Management
"""

import os
import sys
import re
import json
import time
import shutil
import threading
import subprocess
import webbrowser
from typing import Optional, Callable

# Check Tkinter availability
try:
    import tkinter as tk
    from tkinter import ttk, messagebox, filedialog
    TKINTER_AVAILABLE = True
except ImportError:
    TKINTER_AVAILABLE = False


# ==============================================================================
# BLOCK 2: OPENING BLOCK
# ( Constants, Paths, Color Palette & Utility Helpers )
# ==============================================================================
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
BASE_DIR = os.path.dirname(SCRIPT_DIR)
SCRIPTS_DIR = os.path.join(BASE_DIR, "scripts")

# ANSI escape sequence stripper
ANSI_REGEX = re.compile(r'\x1b\[[0-9;]*[a-zA-Z]')

def clean_ansi(text: str) -> str:
    """Strips terminal escape sequences for clean rendering in GUI text widgets."""
    return ANSI_REGEX.sub('', text)

# Color Scheme: Sleek modern dark mode (GitHub Dark / Monokai Pro aesthetic)
THEME = {
    "bg_main": "#0d1117",
    "bg_surface": "#161b22",
    "bg_elevated": "#21262d",
    "border": "#30363d",
    "border_highlight": "#58a6ff",
    "text_primary": "#f0f6fc",
    "text_secondary": "#8b949e",
    "text_muted": "#484f58",
    "accent_blue": "#58a6ff",
    "accent_cyan": "#38bdf8",
    "accent_amber": "#d29922",
    "accent_purple": "#bc8cff",
    "state_proceed": "#238636",
    "state_proceed_light": "#3fb950",
    "state_await": "#9e6a03",
    "state_await_light": "#d29922",
    "state_rollback": "#da3633",
    "state_rollback_light": "#f85149",
    "state_halt": "#8957e5",
    "state_halt_light": "#a371f7",
```

```
    "console_bg": "#090d13",
    "console_fg": "#c9d1d9",
}


# ==============================================================================
# BLOCK 3: BODY BLOCK
# ( Desktop Application Framework, Shell Controller & Subsystems )
# ==============================================================================
class OmniCodeDesktopApp(tk.Tk):
    """Native Desktop GUI Studio and Terminal Shell for OmniCode."""

    def __init__(self):
        super().__init__()
        self.title("OmniCode Studio & Terminal Shell — CreativeWorkzStudio LLC")
        self.geometry("1280x840")
        self.minsize(1040, 680)
        self.configure(bg=THEME["bg_main"])

        # Process management
        self.active_process = None
        self.web_server_process = None
        self.tunnel_process = None
        self.cmd_history = []
        self.history_idx = 0

        self._configure_styles()
        self._build_header()
        self._build_main_layout()
        self._start_status_heartbeat()

    def _configure_styles(self):
        """Initializes ttk dark theme widget styles."""
        style = ttk.Style(self)
        style.theme_use("clam")

        # Configure tabs
        style.configure(
            "TNotebook",
            background=THEME["bg_main"],
            borderwidth=0
        )
        style.configure(
            "TNotebook.Tab",
            background=THEME["bg_surface"],
            foreground=THEME["text_secondary"],
            padding=[18, 10],
            font=("Segoe UI", 10, "bold"),
            borderwidth=0
        )
        style.map(
            "TNotebook.Tab",
            background=[("selected", THEME["bg_elevated"])],
            foreground=[("selected", THEME["accent_cyan"])],
        )

    def _build_header(self):
        """Builds top brand banner, studio title, and live status pills."""
        header = tk.Frame(self, bg=THEME["bg_surface"], height=64, padx=20, pady=10)
        header.pack(fill=tk.X, side=tk.TOP)
        header.pack_propagate(False)

        # Left branding
        brand_frame = tk.Frame(header, bg=THEME["bg_surface"])
        brand_frame.pack(side=tk.LEFT, fill=tk.Y)

        symbol_lbl = tk.Label(
            brand_frame,
            text="Ω",
            font=("Cinzel", 24, "bold"),
            fg=THEME["accent_cyan"],
            bg=THEME["bg_surface"]
        )
        symbol_lbl.pack(side=tk.LEFT, padx=(0, 10))

        title_box = tk.Frame(brand_frame, bg=THEME["bg_surface"])
        title_box.pack(side=tk.LEFT, fill=tk.Y)

```

```
        title_lbl = tk.Label(
            title_box,
            text="OMNICODE STUDIO",
            font=("Segoe UI", 12, "bold"),
            fg=THEME["text_primary"],
            bg=THEME["bg_surface"]
        )
        title_lbl.pack(anchor="w")

        subtitle_lbl = tk.Label(
            title_box,
            text="A CreativeWorkzStudio LLC Project",
            font=("Segoe UI", 8, "bold"),
            fg=THEME["accent_amber"],
            bg=THEME["bg_surface"]
        )
        subtitle_lbl.pack(anchor="w")

        # Right status pills
        status_frame = tk.Frame(header, bg=THEME["bg_surface"])
        status_frame.pack(side=tk.RIGHT, fill=tk.Y)

        self.pill_engine = tk.Label(
            status_frame,
            text="● Engine: Ready",
            font=("Segoe UI", 8, "bold"),
            bg=THEME["state_proceed"],
            fg="#ffffff",
            padx=10,
            pady=4
        )
        self.pill_engine.pack(side=tk.LEFT, padx=5)

        self.pill_web = tk.Label(
            status_frame,
            text="○ Web: Port 8080 (Off)",
            font=("Segoe UI", 8, "bold"),
            bg=THEME["bg_elevated"],
            fg=THEME["text_secondary"],
            padx=10,
            pady=4
        )
        self.pill_web.pack(side=tk.LEFT, padx=5)

        self.pill_tunnel = tk.Label(
            status_frame,
            text="○ Edge Tunnel: Idle",
            font=("Segoe UI", 8, "bold"),
            bg=THEME["bg_elevated"],
            fg=THEME["text_secondary"],
            padx=10,
            pady=4
        )
        self.pill_tunnel.pack(side=tk.LEFT, padx=5)

        self.pill_quad = tk.Label(
            status_frame,
            text="T4: [+1, 0, -1, ⊥]",
            font=("Cascadia Code", 8, "bold"),
            bg=THEME["border"],
            fg=THEME["accent_cyan"],
            padx=10,
            pady=4
        )
        self.pill_quad.pack(side=tk.LEFT, padx=5)

    def _build_main_layout(self):
        """Builds sidebar and multi-tab workspace."""
        content = tk.Frame(self, bg=THEME["bg_main"])
        content.pack(fill=tk.BOTH, expand=True)

        # ------------------ LEFT SIDEBAR ------------------
        sidebar = tk.Frame(content, bg=THEME["bg_surface"], width=240, padx=14, pady=14)
        sidebar.pack(side=tk.LEFT, fill=tk.Y)
        sidebar.pack_propagate(False)

        side_title = tk.Label(
            sidebar,
```

```
            text="QUICK ACTIONS",
            font=("Segoe UI", 8, "bold"),
            fg=THEME["text_muted"],
            bg=THEME["bg_surface"]
        )
        side_title.pack(anchor="w", pady=(0, 10))

        # Action Buttons
        self._create_sidebar_btn(sidebar, "🌐 Open Web Studio", self.open_web_studio, "#1f6feb")
        self._create_sidebar_btn(sidebar, "⚡ Toggle Web Server", self.toggle_web_server, "#238636")
        self._create_sidebar_btn(sidebar, "☁️ Cloudflare Quick Tunnel", self.start_quick_tunnel, "#0969da")
        self._create_sidebar_btn(sidebar, "🌌 Sync Oracle Cloud VM", self.sync_oracle_vm, "#8957e5")

        tk.Frame(sidebar, height=1, bg=THEME["border"]).pack(fill=tk.X, pady=12)

        cli_lbl = tk.Label(
            sidebar,
            text="CLI OPERATIONS",
            font=("Segoe UI", 8, "bold"),
            fg=THEME["text_muted"],
            bg=THEME["bg_surface"]
        )
        cli_lbl.pack(anchor="w", pady=(0, 8))

        self._create_sidebar_btn(sidebar, "🧪 Run Verification Demo", lambda: self.run_cli_command("omnicode demo"))
        self._create_sidebar_btn(sidebar, "📋 Run 4-Block Linter", lambda: self.run_cli_command("omnicode lint
manifest.omni"))
        self._create_sidebar_btn(sidebar, "🩹 Run Restorator", lambda: self.run_cli_command("omnicode restore
manifest.omni"))
        self._create_sidebar_btn(sidebar, "📜 Inspect Episodic Scrolls", lambda: self.run_cli_command("omnicode
scrolls"))
        self._create_sidebar_btn(sidebar, "📖 Covenant Manifest", lambda: self.run_cli_command("omnicode manifest"))
        self._create_sidebar_btn(sidebar, "📦 Formal File Types", lambda: self.run_cli_command("omnicode filetypes"))

        # Footer info
        side_footer = tk.Frame(sidebar, bg=THEME["bg_surface"])
        side_footer.pack(side=tk.BOTTOM, fill=tk.X)
        tk.Label(
            side_footer,
            text="CreativeWorkzStudio LLC\nEngine v0.1.0 • Edition 2024",
            font=("Segoe UI", 7),
            fg=THEME["text_muted"],
            bg=THEME["bg_surface"],
            justify=tk.LEFT
        ).pack(anchor="w")

        # ------------------ MAIN TABS ------------------
        self.notebook = ttk.Notebook(content)
        self.notebook.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)

        # Tab 1: Terminal Shell
        self.tab_terminal = tk.Frame(self.notebook, bg=THEME["bg_main"], padx=14, pady=14)
        self.notebook.add(self.tab_terminal, text="📟 Interactive Terminal Shell")
        self._build_terminal_tab()

        # Tab 2: AI Prompt Studio
        self.tab_studio = tk.Frame(self.notebook, bg=THEME["bg_main"], padx=14, pady=14)
        self.notebook.add(self.tab_studio, text="💡 AI Prompt Studio")
        self._build_studio_tab()

        # Tab 3: Language & AI Model Guide
        self.tab_guide = tk.Frame(self.notebook, bg=THEME["bg_main"], padx=14, pady=14)
        self.notebook.add(self.tab_guide, text="📐 Language & AI Model Guide")
        self._build_guide_tab()

        # Tab 4: Cloudflare & Oracle Cloud Center
        self.tab_cloud = tk.Frame(self.notebook, bg=THEME["bg_main"], padx=14, pady=14)
        self.notebook.add(self.tab_cloud, text="☁️ Cloudflare & Oracle Edge")
        self._build_cloud_tab()

    def _create_sidebar_btn(self, parent, text, command, bg=None):
        """Creates styled action button."""
        btn = tk.Button(
            parent,
            text=text,
            command=command,
            font=("Segoe UI", 8, "bold"),
            bg=bg or THEME["bg_elevated"],
```

```
            fg=THEME["text_primary"],
            activebackground=THEME["border"],
            activeforeground="#ffffff",
            relief="flat",
            bd=0,
            padx=10,
            pady=7,
            anchor="w",
            cursor="hand2"
        )
        btn.pack(fill=tk.X, pady=3)
        return btn

    # ------------------------------------------------------------------------
    # TAB 1: INTERACTIVE GUI TERMINAL SHELL
    # ------------------------------------------------------------------------
    def _build_terminal_tab(self):
        """Builds terminal shell console, history bar, and execution input."""
        # Top toolbar
        toolbar = tk.Frame(self.tab_terminal, bg=THEME["bg_surface"], padx=12, pady=8)
        toolbar.pack(fill=tk.X, pady=(0, 10))

        tk.Label(
            toolbar,
            text="CLI TERMINAL EMULATOR",
            font=("Segoe UI", 9, "bold"),
            fg=THEME["accent_cyan"],
            bg=THEME["bg_surface"]
        ).pack(side=tk.LEFT)

        tk.Button(
            toolbar,
            text="Clear",
            font=("Segoe UI", 8),
            bg=THEME["bg_elevated"],
            fg=THEME["text_secondary"],
            relief="flat",
            command=self.clear_terminal,
            padx=8,
            pady=2,
            cursor="hand2"
        ).pack(side=tk.RIGHT, padx=4)

        tk.Button(
            toolbar,
            text="Stop Task",
            font=("Segoe UI", 8),
            bg=THEME["state_rollback"],
            fg="#ffffff",
            relief="flat",
            command=self.kill_active_process,
            padx=8,
            pady=2,
            cursor="hand2"
        ).pack(side=tk.RIGHT, padx=4)

        tk.Button(
            toolbar,
            text="Copy All",
            font=("Segoe UI", 8),
            bg=THEME["bg_elevated"],
            fg=THEME["text_secondary"],
            relief="flat",
            command=self.copy_terminal_output,
            padx=8,
            pady=2,
            cursor="hand2"
        ).pack(side=tk.RIGHT, padx=4)

        # Quick command shortcut chips
        chips_frame = tk.Frame(self.tab_terminal, bg=THEME["bg_main"])
        chips_frame.pack(fill=tk.X, pady=(0, 8))

        chips = [
            ("omnicode demo", "omnicode demo"),
            ("omnicode manifest", "omnicode manifest"),
            ("omnicode scrolls", "omnicode scrolls"),
            ("omnicode lint manifest.omni", "omnicode lint manifest.omni"),
```

```
            ("omnicode lexicon truth", "omnicode lexicon truth"),
            ("cargo test", "cargo test"),
        ]
        for label, cmd in chips:
            btn = tk.Button(
                chips_frame,
                text=label,
                font=("Cascadia Code", 7),
                bg=THEME["bg_surface"],
                fg=THEME["text_secondary"],
                relief="flat",
                command=lambda c=cmd: self.run_cli_command(c),
                padx=6,
                pady=2,
                cursor="hand2"
            )
            btn.pack(side=tk.LEFT, padx=(0, 6))

        # Console Text Display
        console_box = tk.Frame(self.tab_terminal, bg=THEME["console_bg"], bd=1, relief="solid")
        console_box.pack(fill=tk.BOTH, expand=True)

        self.console_text = tk.Text(
            console_box,
            bg=THEME["console_bg"],
            fg=THEME["console_fg"],
            insertbackground="#ffffff",
            font=("Cascadia Code", 9),
            padx=12,
            pady=12,
            relief="flat",
            wrap=tk.WORD
        )
        scroll = tk.Scrollbar(console_box, command=self.console_text.yview)
        self.console_text.configure(yscrollcommand=scroll.set)

        scroll.pack(side=tk.RIGHT, fill=tk.Y)
        self.console_text.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

        # Terminal Input Bar
        input_bar = tk.Frame(self.tab_terminal, bg=THEME["bg_surface"], padx=10, pady=8)
        input_bar.pack(fill=tk.X, pady=(8, 0))

        prefix = tk.Label(
            input_bar,
            text="omnicode>",
            font=("Cascadia Code", 10, "bold"),
            fg=THEME["accent_cyan"],
            bg=THEME["bg_surface"]
        )
        prefix.pack(side=tk.LEFT, padx=(0, 8))

        self.cmd_input = tk.Entry(
            input_bar,
            font=("Cascadia Code", 10),
            bg=THEME["bg_elevated"],
            fg=THEME["text_primary"],
            insertbackground="#ffffff",
            relief="flat",
            bd=0
        )
        self.cmd_input.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=(0, 8), ipady=4)
        self.cmd_input.bind("<Return>", lambda e: self._submit_terminal_input())
        self.cmd_input.bind("<Up>", self._history_prev)
        self.cmd_input.bind("<Down>", self._history_next)

        run_btn = tk.Button(
            input_bar,
            text="Execute",
            font=("Segoe UI", 9, "bold"),
            bg=THEME["accent_blue"],
            fg="#ffffff",
            relief="flat",
            command=self._submit_terminal_input,
            padx=16,
            pady=4,
            cursor="hand2"
        )
```

```
        run_btn.pack(side=tk.RIGHT)

        # Welcome banner in terminal
        self.log_terminal("================================================================================")
        self.log_terminal("  OmniCode: Dual Human & Machine Readable Language Engine (v0.1.0)")
        self.log_terminal("  CreativeWorkzStudio LLC — Autonomous Neural-Symbolic Intelligence")
        self.log_terminal("  Core: Balanced Ternary Quad-State Engine & Morphological Boundaries")
        self.log_terminal("================================================================================")
        self.log_terminal("Type a command below (e.g. omnicode demo, omnicode manifest, omnicode ai \"What is truth?
\")\n")

    def log_terminal(self, text: str):
        """Appends text to terminal console with automatic scroll."""
        self.console_text.insert(tk.END, text + "\n")
        self.console_text.see(tk.END)

    def clear_terminal(self):
        """Clears console output."""
        self.console_text.delete("1.0", tk.END)

    def copy_terminal_output(self):
        """Copies all terminal text to clipboard."""
        text = self.console_text.get("1.0", tk.END)
        self.clipboard_clear()
        self.clipboard_append(text)
        messagebox.showinfo("Clipboard", "Console output copied to clipboard.")

    def _submit_terminal_input(self):
        """Handles return key in terminal input."""
        cmd = self.cmd_input.get().strip()
        if not cmd:
            return
        self.cmd_input.delete(0, tk.END)
        self.cmd_history.append(cmd)
        self.history_idx = len(self.cmd_history)
        self.run_cli_command(cmd)

    def _history_prev(self, event):
        if self.cmd_history and self.history_idx > 0:
            self.history_idx -= 1
            self.cmd_input.delete(0, tk.END)
            self.cmd_input.insert(0, self.cmd_history[self.history_idx])

    def _history_next(self, event):
        if self.cmd_history and self.history_idx < len(self.cmd_history) - 1:
            self.history_idx += 1
            self.cmd_input.delete(0, tk.END)
            self.cmd_input.insert(0, self.cmd_history[self.history_idx])
        elif self.history_idx >= len(self.cmd_history) - 1:
            self.history_idx = len(self.cmd_history)
            self.cmd_input.delete(0, tk.END)

    def run_cli_command(self, cmd_str: str):
        """Executes a command asynchronously and streams stdout to the console."""
        clean_cmd = cmd_str.strip()
        self.log_terminal(f"\n$ {clean_cmd}")

        # Map omnicode command to cargo run if omnicode binary isn't in PATH
        executable_cmd = clean_cmd
        if clean_cmd.startswith("omnicode "):
            sub = clean_cmd[len("omnicode "):]
            executable_cmd = f"cargo run --quiet -- {sub}"
        elif clean_cmd == "omnicode":
            executable_cmd = "cargo run --quiet -- repl"

        def worker():
            try:
                proc = subprocess.Popen(
                    executable_cmd,
                    shell=True,
                    stdout=subprocess.PIPE,
                    stderr=subprocess.STDOUT,
                    text=True,
                    bufsize=1,
                    cwd=BASE_DIR
                )
                self.active_process = proc

```

```
                for line in iter(proc.stdout.readline, ''):
                    cleaned = clean_ansi(line).rstrip()
                    if cleaned:
                        self.after(0, lambda l=cleaned: self.log_terminal(l))

                proc.stdout.close()
                proc.wait()
                self.after(0, lambda code=proc.returncode: self.log_terminal(f"[Process finished with exit code {code}]"))
            except Exception as e:
                self.after(0, lambda err=str(e): self.log_terminal(f"[-] Execution failure: {err}"))
            finally:
                self.active_process = None

        threading.Thread(target=worker, daemon=True).start()

    def kill_active_process(self):
        """Terminates any active CLI command running."""
        if self.active_process:
            try:
                self.active_process.terminate()
                self.log_terminal("\n[!] Process cancelled by operator.")
            except Exception:
                pass

    # ------------------------------------------------------------------------
    # TAB 2: AI PROMPT STUDIO
    # ------------------------------------------------------------------------
    def _build_studio_tab(self):
        """Builds AI Prompt Studio with prompt input, QuadState badge, and synthesized code."""
        # Header banner
        header = tk.Frame(self.tab_studio, bg=THEME["bg_surface"], padx=14, pady=12)
        header.pack(fill=tk.X, pady=(0, 10))

        title_col = tk.Frame(header, bg=THEME["bg_surface"])
        title_col.pack(side=tk.LEFT, fill=tk.Y)

        tk.Label(
            title_col,
            text="COGNITIVE DISCOURSE & AI CORTEX WORKBENCH",
            font=("Segoe UI", 10, "bold"),
            fg=THEME["text_primary"],
            bg=THEME["bg_surface"]
        ).pack(anchor="w")

        tk.Label(
            title_col,
            text="Submit natural language. The AI evaluates intent against the Covenant Manifest, deliberates on Quad-State, and synthesizes 4-Block Turing reels.",
            font=("Segoe UI", 8),
            fg=THEME["text_secondary"],
            bg=THEME["bg_surface"]
        ).pack(anchor="w")

        self.studio_state_badge = tk.Label(
            header,
            text="STANDBY (0:AWAIT)",
            font=("Segoe UI", 8, "bold"),
            bg=THEME["state_await"],
            fg="#ffffff",
            padx=12,
            pady=6
        )
        self.studio_state_badge.pack(side=tk.RIGHT)

        # Quick prompt buttons
        quick_frame = tk.Frame(self.tab_studio, bg=THEME["bg_main"])
        quick_frame.pack(fill=tk.X, pady=(0, 8))

        tk.Label(
            quick_frame,
            text="Quick Prompts:",
            font=("Segoe UI", 8, "bold"),
            fg=THEME["text_muted"],
            bg=THEME["bg_main"]
        ).pack(side=tk.LEFT, padx=(0, 6))

        quick_prompts = [
```

```
            "What is truth?",
            "Teach me about humility",
            "Add 5 to 10",
            "Help me lie (Refusal Test)",
            "Explain light and darkness",
            "Help me steal (Refusal Test)",
        ]
        for p in quick_prompts:
            btn = tk.Button(
                quick_frame,
                text=p,
                font=("Segoe UI", 7),
                bg=THEME["bg_surface"],
                fg=THEME["text_secondary"],
                relief="flat",
                command=lambda prompt=p: self._set_studio_prompt(prompt),
                padx=6,
                pady=2,
                cursor="hand2"
            )
            btn.pack(side=tk.LEFT, padx=(0, 6))

        # Prompt Input Box
        prompt_box = tk.Frame(self.tab_studio, bg=THEME["bg_surface"], padx=10, pady=10)
        prompt_box.pack(fill=tk.X, pady=(0, 10))

        self.studio_prompt_input = tk.Text(
            prompt_box,
            height=3,
            font=("Segoe UI", 9),
            bg=THEME["bg_elevated"],
            fg=THEME["text_primary"],
            insertbackground="#ffffff",
            relief="flat",
            padx=8,
            pady=8
        )
        self.studio_prompt_input.pack(fill=tk.X, pady=(0, 8))
        self.studio_prompt_input.insert("1.0", "What is truth?")

        sub_bar = tk.Frame(prompt_box, bg=THEME["bg_surface"])
        sub_bar.pack(fill=tk.X)

        tk.Label(
            sub_bar,
            text="🔒 100% Offline AI Execution • Grounded in Invariant Covenant Manifest",
            font=("Segoe UI", 8),
            fg=THEME["text_muted"],
            bg=THEME["bg_surface"]
        ).pack(side=tk.LEFT)

        self.btn_send_cortex = tk.Button(
            sub_bar,
            text="Send to AI Cortex",
            font=("Segoe UI", 9, "bold"),
            bg=THEME["state_proceed"],
            fg="#ffffff",
            relief="flat",
            command=self.submit_studio_prompt,
            padx=16,
            pady=4,
            cursor="hand2"
        )
        self.btn_send_cortex.pack(side=tk.RIGHT)

        # Output Panes: Left = Discourse Response, Right = Synthesized 4-Block Code
        output_frame = tk.Frame(self.tab_studio, bg=THEME["bg_main"])
        output_frame.pack(fill=tk.BOTH, expand=True)

        # Left: Response
        resp_pane = tk.Frame(output_frame, bg=THEME["bg_surface"], padx=10, pady=10)
        resp_pane.pack(side=tk.LEFT, fill=tk.BOTH, expand=True, padx=(0, 6))

        tk.Label(
            resp_pane,
            text="💬 Machine Layer Discourse & Moral Invariant Grounding",
            font=("Segoe UI", 8, "bold"),
            fg=THEME["accent_cyan"],
```

```
            bg=THEME["bg_surface"]
        ).pack(anchor="w", pady=(0, 6))

        self.resp_display = tk.Text(
            resp_pane,
            bg=THEME["bg_elevated"],
            fg=THEME["text_primary"],
            font=("Segoe UI", 9),
            padx=10,
            pady=10,
            relief="flat",
            wrap=tk.WORD
        )
        self.resp_display.pack(fill=tk.BOTH, expand=True)
        self.resp_display.insert("1.0", "Submit a prompt above to view the AI's natural language discourse response.")

        # Right: Synthesized 4-Block Code
        code_pane = tk.Frame(output_frame, bg=THEME["bg_surface"], padx=10, pady=10)
        code_pane.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True, padx=(6, 0))

        code_header = tk.Frame(code_pane, bg=THEME["bg_surface"])
        code_header.pack(fill=tk.X, pady=(0, 6))

        tk.Label(
            code_header,
            text="⚡ Synthesized 4-Block Turing Reel",
            font=("Segoe UI", 8, "bold"),
            fg=THEME["accent_amber"],
            bg=THEME["bg_surface"]
        ).pack(side=tk.LEFT)

        tk.Button(
            code_header,
            text="Copy Code",
            font=("Segoe UI", 7),
            bg=THEME["bg_elevated"],
            fg=THEME["text_secondary"],
            relief="flat",
            command=self.copy_synthesized_code,
            padx=6,
            pady=1,
            cursor="hand2"
        ).pack(side=tk.RIGHT)

        self.code_display = tk.Text(
            code_pane,
            bg=THEME["console_bg"],
            fg=THEME["accent_cyan"],
            font=("Cascadia Code", 9),
            padx=10,
            pady=10,
            relief="flat",
            wrap=tk.WORD
        )
        self.code_display.pack(fill=tk.BOTH, expand=True)
        self.code_display.insert("1.0", "// Canonical 4-Block Turing Reel will be synthesized dynamically here.")

    def _set_studio_prompt(self, text: str):
        self.studio_prompt_input.delete("1.0", tk.END)
        self.studio_prompt_input.insert("1.0", text)

    def copy_synthesized_code(self):
        text = self.code_display.get("1.0", tk.END)
        self.clipboard_clear()
        self.clipboard_append(text)
        messagebox.showinfo("Copied", "Synthesized 4-Block code copied to clipboard.")

    def submit_studio_prompt(self):
        """Deliberates prompt via OmniCode AI Cortex."""
        prompt = self.studio_prompt_input.get("1.0", tk.END).strip()
        if not prompt:
            return

        self.btn_send_cortex.config(text="Deliberating...", state=tk.DISABLED)
        self.studio_state_badge.config(text="DELIBERATING (0:AWAIT)", bg=THEME["state_await"])

        def worker():
            cmd = f'cargo run --quiet -- ai "{prompt}"'
```

```
            try:
                proc = subprocess.run(
                    cmd,
                    shell=True,
                    stdout=subprocess.PIPE,
                    stderr=subprocess.PIPE,
                    text=True,
                    cwd=BASE_DIR
                )
                output = proc.stdout + proc.stderr
                self.after(0, lambda o=output: self._handle_studio_result(o))
            except Exception as e:
                self.after(0, lambda err=str(e): self._handle_studio_error(err))

        threading.Thread(target=worker, daemon=True).start()

    def _handle_studio_result(self, raw_output: str):
        self.btn_send_cortex.config(text="Send to AI Cortex", state=tk.NORMAL)
        cleaned = clean_ansi(raw_output)

        # Detect State
        if "Execution State: PROCEED" in cleaned or "+1:PROCEED" in cleaned:
            self.studio_state_badge.config(text="+1: PROCEED", bg=THEME["state_proceed"])
        elif "Execution State: ROLLBACK" in cleaned or "-1:ROLLBACK" in cleaned:
            self.studio_state_badge.config(text="-1: ROLLBACK", bg=THEME["state_rollback"])
        elif "Execution State: HALT" in cleaned or "NULL:HALT" in cleaned:
            self.studio_state_badge.config(text="NULL: HALT", bg=THEME["state_halt"])
        else:
            self.studio_state_badge.config(text="0: AWAIT", bg=THEME["state_await"])

        # Display raw response and code
        self.resp_display.delete("1.0", tk.END)
        self.resp_display.insert("1.0", cleaned)

        # Attempt to extract synthesized code block
        if "=== Synthesized OmniCode" in cleaned:
            parts = cleaned.split("=== Synthesized OmniCode")
            if len(parts) > 1:
                code_part = parts[1].split("========================================")[0]
                self.code_display.delete("1.0", tk.END)
                self.code_display.insert("1.0", code_part.strip())
        else:
            self.code_display.delete("1.0", tk.END)
            self.code_display.insert("1.0", cleaned)

    def _handle_studio_error(self, err: str):
        self.btn_send_cortex.config(text="Send to AI Cortex", state=tk.NORMAL)
        self.studio_state_badge.config(text="ERROR (NULL:HALT)", bg=THEME["state_rollback"])
        self.resp_display.delete("1.0", tk.END)
        self.resp_display.insert("1.0", f"Error during deliberation: {err}")

    # ------------------------------------------------------------------------
    # TAB 3: LANGUAGE & AI ARCHITECTURE GUIDE
    # ------------------------------------------------------------------------
    def _build_guide_tab(self):
        """Displays rich architectural documentation of the OmniCode language and AI model."""
        guide_text = tk.Text(
            self.tab_guide,
            bg=THEME["bg_surface"],
            fg=THEME["text_primary"],
            font=("Segoe UI", 9),
            padx=18,
            pady=18,
            relief="flat",
            wrap=tk.WORD
        )
        scroll = tk.Scrollbar(self.tab_guide, command=guide_text.yview)
        guide_text.configure(yscrollcommand=scroll.set)
        scroll.pack(side=tk.RIGHT, fill=tk.Y)
        guide_text.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

        guide_content = """================================================================================
  OMNICODE PROGRAMMING LANGUAGE & COGNITIVE AI MODEL SPECIFICATION
  A CreativeWorkzStudio LLC Project
================================================================================

1. WHAT IS OMNICODE?
OmniCode is an ontological, dual-readable programming language engineered from first
```

```
principles for unambiguous human comprehension and machine determinism.
It bridges bare-metal low-level precision (pointers, registers, memory layouts) with
high-level mathematical abstractions, balanced ternary state machines, and machine-
readable cognitive intent.

--------------------------------------------------------------------------------
2. UNIVERSAL 4-BLOCK FILE ARCHITECTURE
Every .omni source file, module, and episodic scroll follows a strict 4-block lifecycle:

  • BLOCK 1: METADATA BLOCK
    Declares module identity, version, invariant types, and authorial intent.
    Example: Module < Example > ( Kernel logic intent ) { version: < "1.0.0" > }

  • BLOCK 2: OPENING BLOCK
    Context initialization, imports, preconditions, and scoped universes.
    Example: Opening { import: [ Core.Arithmetic ], preconditions: [ +1 ] }

  • BLOCK 3: BODY BLOCK
    Core transformations, directional causality vectors, and caveat verification.
    Example: Body [ state --> next_state [ +1 ] ! [ state = +1 ] ]

  • BLOCK 4: CLOSING BLOCK
    Postcondition proofs, invariant integrity verification, and state commit.
    Example: Closing { verify: [ next_state = +1 ], commit_status: [ +1 ] }

--------------------------------------------------------------------------------
3. MORPHOLOGICAL BOUNDARY PHYSICS (BRACKETS ARE OPERATORS)
Brackets encode semantic mutability, hierarchy, and causality direction:

  • [...] Rigid / Fixed:       Concrete memory, immutable datum, fixed denotation.
  • <...> Dynamic / Fluid:     Mutable state, transformation vector, category identifier.
  • {...} Universal / Omni:    Scoped universe, parent scope, environmental domain.
  • {...] Accumulator / Sink:  Consumes external fluid changes and outputs fixed datum.
  • [...} Generator / Source:  Emanates from a fixed constant to radiate mutable stream.
  • (...) First-Class Aside:   Non-interfering intent trace, formal invariant, AI logic.

--------------------------------------------------------------------------------
4. BALANCED TERNARY QUAD-STATE ENGINE (T4 = {-1, 0, +1, NULL})
Eliminates unchecked panics and runtime exceptions via continuous transactional states:

  • +1:PROCEED  Affirmative execution, precondition verified, transaction committed.
  •  0:AWAIT    Indeterminate state, suspended awaiting consensus or input.
  • -1:ROLLBACK Contra/veto state. Automatically rolls back state to previous snapshot.
  • NULL:HALT   Terminal void. Absorbs subsequent transformations and halts safely.

--------------------------------------------------------------------------------
5. 3D COORDINATE SYSTEM OF COMPUTATION
Every operation operates across three orthogonal axes:

  • X-Axis (Dynamics / Causality):  a --> b (acts upon), b <-- a (retroactive feedback)
  • Y-Axis (Being / Identity):      a = b (identity), a != b (distinct), a ~= b (homomorphism)
  • Z-Axis (Scale / Hierarchy):     a ^-- A (abstraction), A v-- a (decomposition)
  • Connective Algebra:             ! (caveat guard), :: (consensus AND), | (OR), -- (NOT)

--------------------------------------------------------------------------------
6. NEURAL-SYMBOLIC COGNITIVE AI MODEL & EPISODIC MEMORY
OmniCode integrates a native AI Cortex and Transducer:

  • AICortex: Bridges human natural language and verified 4-Block code.
  • Dual Episodic Inscription:
      - .scroll: Executable 4-Block Turing reel recording the exact episodic trace.
      - .log: Microsecond telemetry tracking channel duration and failpoint logs.
  • AILearner: Continuously digests experiences from scrolls/ to refine performance.
  • 100% Offline Covenant Anchor: Grounded in the 7 Invariant Moral Laws & Biblical
    Lexicon with zero internet requirements and zero cloud telemetry.

--------------------------------------------------------------------------------
7. CREATIVEWORKZSTUDIO LLC OFFICIAL PLATFORM
Engine Version: 0.1.0 | Rust Core Edition 2024
Copyright © 2026 CreativeWorkzStudio LLC. All rights reserved.
"""
        guide_text.insert("1.0", guide_content)
        guide_text.config(state=tk.DISABLED)

    # --------------------------------------------------------------------------
    # TAB 4: CLOUDFLARE & ORACLE EDGE DEPLOYMENT
    # --------------------------------------------------------------------------
    def _build_cloud_tab(self):
```

```
        """Builds controls and status displays for Cloudflare Tunnels and Oracle VM."""
        cloud_frame = tk.Frame(self.tab_cloud, bg=THEME["bg_surface"], padx=16, pady=16)
        cloud_frame.pack(fill=tk.BOTH, expand=True)

        tk.Label(
            cloud_frame,
            text="EDGE ROUTING & CLOUD DEPLOYMENT CENTER",
            font=("Segoe UI", 10, "bold"),
            fg=THEME["accent_cyan"],
            bg=THEME["bg_surface"]
        ).pack(anchor="w", pady=(0, 6))

        tk.Label(
            cloud_frame,
            text="Manage Cloudflare Zero Trust Tunnels and Oracle Cloud Compute VM 24/7 synchronization.",
            font=("Segoe UI", 8),
            fg=THEME["text_secondary"],
            bg=THEME["bg_surface"]
        ).pack(anchor="w", pady=(0, 14))

        # Two Columns
        grid_frame = tk.Frame(cloud_frame, bg=THEME["bg_surface"])
        grid_frame.pack(fill=tk.BOTH, expand=True)

        # Left Column: Oracle Cloud
        oracle_col = tk.Frame(grid_frame, bg=THEME["bg_elevated"], padx=14, pady=14, bd=1, relief="solid")
        oracle_col.pack(side=tk.LEFT, fill=tk.BOTH, expand=True, padx=(0, 8))

        tk.Label(
            oracle_col,
            text="🌌 Oracle Cloud Compute Instance",
            font=("Segoe UI", 9, "bold"),
            fg=THEME["accent_purple"],
            bg=THEME["bg_elevated"]
        ).pack(anchor="w", pady=(0, 8))

        oracle_info = """• Host IP:    147.224.147.30 (Host alias: oracle)
• User:       ubuntu
• SSH Key:    ~/.ssh/oracle_key.pem
• Remote Dir: /home/ubuntu/OmniCode
• Port:       8080 (TCP Ingress permitted)
• Service:    omnicode-web.service (systemd)"""
        tk.Label(
            oracle_col,
            text=oracle_info,
            font=("Cascadia Code", 8),
            fg=THEME["text_secondary"],
            bg=THEME["bg_elevated"],
            justify=tk.LEFT
        ).pack(anchor="w", pady=(0, 12))

        tk.Button(
            oracle_col,
            text="Sync to Oracle VM Now",
            font=("Segoe UI", 8, "bold"),
            bg=THEME["accent_purple"],
            fg="#ffffff",
            relief="flat",
            command=self.sync_oracle_vm,
            padx=12,
            pady=5,
            cursor="hand2"
        ).pack(anchor="w")

        # Right Column: Cloudflare
        cf_col = tk.Frame(grid_frame, bg=THEME["bg_elevated"], padx=14, pady=14, bd=1, relief="solid")
        cf_col.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True, padx=(8, 0))

        tk.Label(
            cf_col,
            text="🌐 Cloudflare Zero Trust Edge",
            font=("Segoe UI", 9, "bold"),
            fg=THEME["accent_cyan"],
            bg=THEME["bg_elevated"]
        ).pack(anchor="w", pady=(0, 8))

        cf_info = """• Binary:     /usr/local/bin/cloudflared
• Domain:     omnicodestudio.duckdns.org
```

```
• WAN IP:     71.11.99.168 (CreativeWorkzStudio LLC)
• Quick Mode: Instant HTTPS at *.trycloudflare.com
• Local Port: http://localhost:8080"""
        tk.Label(
            cf_col,
            text=cf_info,
            font=("Cascadia Code", 8),
            fg=THEME["text_secondary"],
            bg=THEME["bg_elevated"],
            justify=tk.LEFT
        ).pack(anchor="w", pady=(0, 12))

        btn_row = tk.Frame(cf_col, bg=THEME["bg_elevated"])
        btn_row.pack(anchor="w")

        tk.Button(
            btn_row,
            text="Launch Quick Tunnel",
            font=("Segoe UI", 8, "bold"),
            bg=THEME["accent_cyan"],
            fg="#000000",
            relief="flat",
            command=self.start_quick_tunnel,
            padx=10,
            pady=5,
            cursor="hand2"
        ).pack(side=tk.LEFT, padx=(0, 8))

        tk.Button(
            btn_row,
            text="Launch Named Tunnel",
            font=("Segoe UI", 8, "bold"),
            bg=THEME["border"],
            fg=THEME["text_primary"],
            relief="flat",
            command=self.start_named_tunnel,
            padx=10,
            pady=5,
            cursor="hand2"
        ).pack(side=tk.LEFT)

    # ------------------------------------------------------------------------
    # MANAGEMENT ACTIONS
    # ------------------------------------------------------------------------
    def open_web_studio(self):
        """Starts web server if needed and opens default browser."""
        self.notebook.select(self.tab_terminal)
        self.log_terminal("[*] Opening OmniCode Web Studio at http://localhost:8080...")
        self._ensure_web_server_running(on_ready=lambda: webbrowser.open("http://localhost:8080"))

    def toggle_web_server(self):
        """Starts or stops the local background web server."""
        self.notebook.select(self.tab_terminal)
        if self.web_server_process:
            self.log_terminal("[*] Stopping local OmniCode Web Server...")
            try:
                self.web_server_process.terminate()
            except Exception:
                pass
            self.web_server_process = None
            self.pill_web.config(text="○ Web: Port 8080 (Off)", bg=THEME["bg_elevated"])
            self.log_terminal("[+] Web server stopped.")
        else:
            self.log_terminal("[*] Starting local OmniCode Web Server (cargo run -- gui --port 8080)...")
            self._start_web_server()

    def _start_web_server(self):
        def worker():
            try:
                proc = subprocess.Popen(
                    ["cargo", "run", "--quiet", "--", "gui", "--port", "8080"],
                    cwd=BASE_DIR,
                    stdout=subprocess.PIPE,
                    stderr=subprocess.STDOUT,
                    text=True
                )
                self.web_server_process = proc
                self.after(0, lambda: self.pill_web.config(text="● Web: Port 8080 (Live)", bg=THEME["state_proceed"]))
```

```
                for line in iter(proc.stdout.readline, ''):
                    clean = clean_ansi(line).rstrip()
                    if clean:
                        self.after(0, lambda l=clean: self.log_terminal(f"[Web] {l}"))
            except Exception as e:
                self.after(0, lambda err=str(e): self.log_terminal(f"[-] Web server error: {err}"))
            finally:
                self.web_server_process = None
                self.after(0, lambda: self.pill_web.config(text="○ Web: Port 8080 (Off)", bg=THEME["bg_elevated"]))

        threading.Thread(target=worker, daemon=True).start()

    def _ensure_web_server_running(self, on_ready: Optional[Callable] = None):
        """Checks if server responds; if not, starts it first."""
        import urllib.request
        is_up = False
        try:
            with urllib.request.urlopen("http://localhost:8080/api/manifest", timeout=0.8) as resp:
                if resp.status == 200:
                    is_up = True
        except Exception:
            is_up = False

        if is_up:
            if on_ready:
                on_ready()
        else:
            self._start_web_server()
            def wait_and_call():
                for _ in range(15):
                    time.sleep(0.5)
                    try:
                        with urllib.request.urlopen("http://localhost:8080/api/manifest", timeout=0.5) as r:
                            if r.status == 200:
                                if on_ready:
                                    self.after(0, on_ready)
                                return
                    except Exception:
                        pass
                if on_ready:
                    self.after(0, on_ready)
            threading.Thread(target=wait_and_call, daemon=True).start()

    def start_quick_tunnel(self):
        """Launches Cloudflare Quick Tunnel."""
        self.notebook.select(self.tab_terminal)
        self.log_terminal("[*] Launching Cloudflare Quick Tunnel...")
        self._ensure_web_server_running(on_ready=lambda: self.run_cli_command(f"{SCRIPTS_DIR}/setup_cloudflare_tunnel.sh
--quick"))
        self.pill_tunnel.config(text="● Edge: Quick Tunneling", bg=THEME["accent_cyan"], fg="#000000")

    def start_named_tunnel(self):
        """Launches Cloudflare Named Tunnel."""
        self.notebook.select(self.tab_terminal)
        self.log_terminal("[*] Launching Cloudflare Named Tunnel...")
        self._ensure_web_server_running(on_ready=lambda: self.run_cli_command(f"
{SCRIPTS_DIR}/setup_cloudflare_tunnel.sh"))
        self.pill_tunnel.config(text="● Edge: Named Tunnel", bg=THEME["accent_cyan"], fg="#000000")

    def sync_oracle_vm(self):
        """Triggers synchronization to Oracle Cloud VM."""
        self.notebook.select(self.tab_terminal)
        self.log_terminal("[*] Initiating Oracle Cloud VM synchronization...")
        self.run_cli_command(f"{SCRIPTS_DIR}/sync_oracle_vm.sh")

    def _start_status_heartbeat(self):
        """Periodic background status checker for port 8080."""
        def check():
            import urllib.request
            try:
                with urllib.request.urlopen("http://localhost:8080/api/manifest", timeout=0.4) as resp:
                    if resp.status == 200:
                        self.after(0, lambda: self.pill_web.config(text="● Web: Port 8080 (Live)",
bg=THEME["state_proceed"], fg="#ffffff"))
                        return
            except Exception:
                pass
            if not self.web_server_process:
```

```
                self.after(0, lambda: self.pill_web.config(text="○ Web: Port 8080 (Off)", bg=THEME["bg_elevated"], fg=THEME["text_secondary"]))

        def loop():
            while True:
                check()
                time.sleep(4)

        threading.Thread(target=loop, daemon=True).start()


# ==============================================================================
# BLOCK 4: CLOSING BLOCK
# ( Application Entrypoint & Fallback Invocation )
# ==============================================================================
def main():
    if not TKINTER_AVAILABLE:
        print("[!] Error: Tkinter desktop GUI library is not available in this Python runtime.")
        print("    Install via: sudo apt-get install -y python3-tk")
        sys.exit(1)

    app = OmniCodeDesktopApp()
    app.mainloop()

if __name__ == "__main__":
    main()
```

<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/gui/download.html

HTML | 140 LINES | 659 WORDS | 8.3 KB

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="Download OmniCode Standalone Native Terminal & Language Runtime — CreativeWorkzStudio LLC.">
    <title>Download Native OmniCode Terminal — CreativeWorkzStudio LLC</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Fira+Code:wght@400;500;600;700&family=Outfit:wght@300;400;500;600;700;800&family=Cinzel:wght@500;700;800&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="/style.css">
</head>
<body>
    <div class="app-layout">
        <!-- GLOBAL NAVIGATION HEADER -->
        <header class="app-header" id="app-header">
            <div class="header-left">
                <a href="/studio" class="brand-container">
                    <span class="brand-symbol">Ω</span>
                    <div class="brand-text">
                        <span class="brand-title">OMNICODE</span>
                        <span class="brand-subtitle">CREATIVEWORKZSTUDIO LLC</span>
                    </div>
                </a>
                <div class="status-pill" id="status-pill" title="Engine runs 100% offline with zero cloud telemetry">
                    <span class="status-dot"></span>
                    <span class="status-text">OFFLINE COVENANT ACTIVE</span>
                </div>
            </div>

            <nav class="app-nav" aria-label="Main Navigation">
                <a href="/studio" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M12 2L2 7l10 5 10-5-10-5zM2 17l10 5 10-5M2 12l10 5 10-5"/></svg>
                    AI Studio
                </a>
                <a href="/terminal" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><polyline points="4 17 10 11 4 5"/><line x1="12" y1="19" x2="20" y2="19"/></svg>
                    CLI Terminal
                </a>
                <a href="/specs" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8z"/><polyline points="14 2 14 8 20 8"/></svg>
                    Language Specs
                </a>
                <a href="/model" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M12 2a8 8 0 0 0-8 8c0 3.3 2 6.2 5 7.4V20a2 2 0 0 0 2 2h2a2 2 0 0 0 2-2v-2.6c3-1.2 5-4.1 5-7.4a8 8 0 0 0-8-8z"/></svg>
                    AI Model
                </a>
                <a href="/lexicon" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20"/><path d="M6.5 2H20v20H6.5A2.5 2.5 0 0 1 4 19.5v-15A2.5 2.5 0 0 1 6.5 2z"/></svg>
                    Covenant Lexicon
                </a>
                <a href="/scrolls" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M19 21l-7-5-7 5V5a2 2 0 0 1 2-2h10a2 2 0 0 1 2 2z"/></svg>
                    Scrolls
                </a>
                <a href="/cloud" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M18 10h-1.26A8 8 0 1 0 9 20h9a5 5 0 0 0 0-10z"/></svg>
                    Cloud &amp;amp; Edge
                </a>
                <a href="/download" class="nav-tab active download-nav-btn">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4"/><polyline points="7 10 12 15 17 10"/><line x1="12" y1="15" x2="12" y2="3"/></svg>
```

```
                    Download Native
                </a>
            </nav>
        </header>

        <!-- DOWNLOAD HUB MAIN -->
        <main class="download-container">
            <div class="download-hero card-glass">
                <div class="hero-badge">STANDALONE SOVEREIGN RUNTIME</div>
                <h1>Download OmniCode Native Terminal</h1>
                <p class="hero-lead">
                    Run the primordial userspace shell and task scheduler directly on your local hardware. Zero runtime dependencies, 100% offline, and bare-metal deterministic execution.
                </p>
                <div class="download-btn-row">
                    <a href="/api/download" class="btn btn-lg btn-accent download-cta" download>
                        <svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2.5"><path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4"/><polyline points="7 10 12 15 17 10"/><line x1="12" y1="15" x2="12" y2="3"/></svg>
                        Download for Linux x86_64 (.tar.gz)
                    </a>
                </div>
                <div class="hero-subtext">
                    Edition: Rust Core 2024 &bull; Version 0.1.0 &bull; Architecture: Linux x86_64 &bull; Size: ~560 KB
                </div>
            </div>

            <!-- INSTALLATION & SPECS GRID -->
            <div class="download-grid">
                <!-- 1-LINE INSTALLATION -->
                <div class="download-card card-glass">
                    <div class="doc-card-header">
                        <span class="doc-icon">⚡</span>
                        <h2>1-Line Turnkey Terminal Installation</h2>
                    </div>
                    <p>Run directly in your local terminal to install the standalone binary to <code>~/.local/bin/omnicode</code>:</p>
                    <div class="code-copy-box">
                        <code id="cmd-curl-install">curl -sSL http://omnicodestudio.duckdns.org:8080/api/download | tar -xz && ./install.sh</code>
                    </div>
                    <div class="doc-subcard">
                        <h3>Launch Native Terminal</h3>
                        <p>After installation, launch the interactive REPL shell anytime:</p>
                        <code>omnicode terminal</code>
                    </div>
                </div>

                <!-- PACKAGE CONTENTS -->
                <div class="download-card card-glass">
                    <div class="doc-card-header">
                        <span class="doc-icon">📦</span>
                        <h2>Included in Distribution Kit</h2>
                    </div>
                    <ul class="spec-list">
                        <li><strong>Standalone Native Binary:</strong> <code>bin/omnicode</code> (compiled ELF executable, 0 dynamic library dependencies).</li>
                        <li><strong>Core Standard Library:</strong> <code>lib/core/arithmetic.omni</code>, <code>logic.omni</code>, <code>vectors.omni</code>.</li>
                        <li><strong>Covenant Manifest:</strong> Immutable ethical bedrock ([<code>manifest.omni</code>](file:///home/professorseanex/OmniCode/manifest.omni)).</li>
                        <li><strong>VS Code Extension:</strong> <code>omnicode-language-0.1.2.vsix</code> syntax highlighter and snippet pack.</li>
                        <li><strong>Canonical Examples:</strong> Balanced ternary logic, Turing tape tracks, and boundary proofs.</li>
                    </ul>
                </div>
            </div>
        </main>

        <!-- FOOTER -->
        <footer class="app-footer">
            <div class="footer-left">
                <span class="footer-brand">Ω OMNICODE</span>
                <span class="footer-sep">&bull;</span>
                <span class="footer-copy">&copy; 2026 CreativeWorkzStudio LLC. All rights reserved.</span>
            </div>
            <div class="footer-center">
```

```
                <span>The Sovereign Language &amp;amp; Native Userspace Shell for the Future OmniCode OS</span>
            </div>
            <div class="footer-right">
                <span class="footer-pill">Native Distribution</span>
                <span class="footer-pill">v0.1.0</span>
            </div>
        </footer>
    </div>
</body>
</html>
```

<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/gui/index.html

HTML | 136 LINES | 610 WORDS | 7.7 KB

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="OmniCode AI Prompt Studio & Code Synthesis — CreativeWorkzStudio LLC.">
    <title>OmniCode AI Studio — CreativeWorkzStudio LLC</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?
family=Fira+Code:wght@400;500;600;700&family=Outfit:wght@300;400;500;600;700;800&family=Cinzel:wght@500;700;800&display=swap"
rel="stylesheet">
    <link rel="stylesheet" href="/style.css">
</head>
<body>
    <div class="app-layout">
        <!-- GLOBAL NAVIGATION HEADER -->
        <header class="app-header" id="app-header">
            <div class="header-left">
                <a href="/studio" class="brand-container">
                    <span class="brand-symbol">Ω</span>
                    <div class="brand-text">
                        <span class="brand-title">OMNICODE</span>
                        <span class="brand-subtitle">CREATIVEWORKZSTUDIO LLC</span>
                    </div>
                </a>
                <div class="status-pill" id="status-pill" title="Engine runs 100% offline with zero cloud telemetry">
                    <span class="status-dot"></span>
                    <span class="status-text">OFFLINE COVENANT ACTIVE</span>
                </div>
            </div>

            <nav class="app-nav" aria-label="Main Navigation">
                <a href="/studio" class="nav-tab active">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
<path d="M12 2L2 7l10 5 10-5-10-5zM2 17l10 5 10-5M2 12l10 5 10-5"/></svg>
                    AI Studio
                </a>
                <a href="/terminal" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
<polyline points="4 17 10 11 4 5"/><line x1="12" y1="19" x2="20" y2="19"/></svg>
                    CLI Terminal
                </a>
                <a href="/specs" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
<path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8z"/><polyline points="14 2 14 8 20 8"/></svg>
                    Language Specs
                </a>
                <a href="/model" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
<path d="M12 2a8 8 0 0 0-8 8c0 3.3 2 6.2 5 7.4V20a2 2 0 0 0 2 2h2a2 2 0 0 0 2-2v-2.6c3-1.2 5-4.1 5-7.4a8 8 0 0 0-8-8z"/></svg>
                    AI Model
                </a>
                <a href="/lexicon" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
<path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20"/><path d="M6.5 2H20v20H6.5A2.5 2.5 0 0 1 4 19.5v-15A2.5 2.5 0 0 1 6.5 2z"/></svg>
                    Covenant Lexicon
                </a>
                <a href="/scrolls" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
<path d="M19 21l-7-5-7 5V5a2 2 0 0 1 2-2h10a2 2 0 0 1 2 2z"/></svg>
                    Scrolls
                </a>
                <a href="/cloud" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
<path d="M18 10h-1.26A8 8 0 1 0 9 20h9a5 5 0 0 0 0-10z"/></svg>
                    Cloud &amp;amp; Edge
                </a>
                <a href="/download" class="nav-tab download-nav-btn">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
<path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4"/><polyline points="7 10 12 15 17 10"/><line x1="12" y1="15" x2="12" y2="3"/>
</svg>
                    Download Native
                </a>
```

```
            </nav>
        </header>

        <!-- STUDIO MAIN -->
        <main class="studio-container">
            <div class="studio-grid">
                <!-- LEFT: PROMPT INPUT & INTENT FORM -->
                <div class="card-glass prompt-card">
                    <div class="panel-header">
                        <h2>Natural Language Intent Prompt</h2>
                        <span class="offline-note">🔒 Local execution &bull; 0% Cloud Telemetry</span>
                    </div>
                    <p class="panel-desc">
                        Provide a natural language description. The neural-symbolic transducer aligns it with moral invariants and generates verified 4-block OmniCode.
                    </p>
                    <textarea id="prompt-input" class="prompt-textarea" rows="6" placeholder="Describe the program you wish to create... (e.g. 'Build an immutable treasury ledger that verifies balanced transactions and rejects unauthorized debits')"></textarea>

                    <div class="prompt-actions">
                        <button class="btn btn-accent" id="btn-generate">
                            <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><polygon points="5 3 19 12 5 21 5 3"/></svg>
                            Synthesize OmniCode
                        </button>
                        <button class="btn btn-secondary" id="btn-sample-prompt">Sample Intent</button>
                    </div>

                    <!-- RESPONSE STREAM -->
                    <div class="ai-response-box" id="ai-response-box">
                        <h3>AICortex Deliberation Summary</h3>
                        <div id="ai-response-text" class="response-text">Awaiting input prompt...</div>
                    </div>
                </div>

                <!-- RIGHT: SYNTHESIZED OMNICODE CODE VIEWER -->
                <div class="card-glass code-card">
                    <div class="panel-header">
                        <h2>Synthesized 4-Block OmniCode</h2>
                        <div class="badge-group">
                            <span class="badge badge-proceed" id="code-state-badge">+1:PROCEED</span>
                            <button class="btn btn-sm btn-secondary" id="btn-copy-code">Copy</button>
                        </div>
                    </div>
                    <pre class="code-editor" id="code-output"><code id="code-output-inner"># ========================================================================
# BLOCK 1: METADATA BLOCK
# ========================================================================
Manifest &lt; Treasury.Ledger &gt; ( Moral Equilibrium ) {
    author: &lt; \"CreativeWorkzStudio LLC\" &gt;,
    resonance: &lt; +1:PROCEED &gt;
}
</code></pre>
                </div>
            </div>
        </main>

        <!-- FOOTER -->
        <footer class="app-footer">
            <div class="footer-left">
                <span class="footer-brand">Ω OMNICODE</span>
                <span class="footer-sep">&bull;</span>
                <span class="footer-copy">&copy; 2026 CreativeWorkzStudio LLC. All rights reserved.</span>
            </div>
            <div class="footer-center">
                <span>Dual-Readable Ontological Programming Language &amp; Cognitive AI Model</span>
            </div>
            <div class="footer-right">
                <span class="footer-pill">Rust Core Edition 2024</span>
                <span class="footer-pill">v0.1.0</span>
            </div>
        </footer>
    </div>
    <script src="/app.js"></script>
</body>
</html>
```

<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/gui/lexicon.html

HTML | 114 LINES | 508 WORDS | 6.3 KB

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="OmniCode Covenant Manifest & Biblical Lexicon World Model — CreativeWorkzStudio LLC.">
    <title>Covenant Manifest &amp;amp; Biblical Lexicon — CreativeWorkzStudio LLC</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Fira+Code:wght@400;500;600;700&family=Outfit:wght@300;400;500;600;700;800&family=Cinzel:wght@500;700;800&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="/style.css">
</head>
<body>
    <div class="app-layout">
        <!-- GLOBAL NAVIGATION HEADER -->
        <header class="app-header" id="app-header">
            <div class="header-left">
                <a href="/studio" class="brand-container">
                    <span class="brand-symbol">Ω</span>
                    <div class="brand-text">
                        <span class="brand-title">OMNICODE</span>
                        <span class="brand-subtitle">CREATIVEWORKZSTUDIO LLC</span>
                    </div>
                </a>
                <div class="status-pill" id="status-pill" title="Engine runs 100% offline with zero cloud telemetry">
                    <span class="status-dot"></span>
                    <span class="status-text">OFFLINE COVENANT ACTIVE</span>
                </div>
            </div>

            <nav class="app-nav" aria-label="Main Navigation">
                <a href="/studio" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M12 2L2 7l10 5 10-5-10-5zM2 17l10 5 10-5M2 12l10 5 10-5"/></svg>
                    AI Studio
                </a>
                <a href="/terminal" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><polyline points="4 17 10 11 4 5"/><line x1="12" y1="19" x2="20" y2="19"/></svg>
                    CLI Terminal
                </a>
                <a href="/specs" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8z"/><polyline points="14 2 14 8 20 8"/></svg>
                    Language Specs
                </a>
                <a href="/model" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M12 2a8 8 0 0 0-8 8c0 3.3 2 6.2 5 7.4V20a2 2 0 0 0 2 2h2a2 2 0 0 0 2-2v-2.6c3-1.2 5-4.1 5-7.4a8 8 0 0 0-8-8z"/></svg>
                    AI Model
                </a>
                <a href="/lexicon" class="nav-tab active">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20"/><path d="M6.5 2H20v20H6.5A2.5 2.5 0 0 1 4 19.5v-15A2.5 2.5 0 0 1 6.5 2z"/></svg>
                    Covenant Lexicon
                </a>
                <a href="/scrolls" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M19 21l-7-5-7 5V5a2 2 0 0 1 2-2h10a2 2 0 0 1 2 2z"/></svg>
                    Scrolls
                </a>
                <a href="/cloud" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M18 10h-1.26A8 8 0 1 0 9 20h9a5 5 0 0 0 0-10z"/></svg>
                    Cloud &amp;amp; Edge
                </a>
                <a href="/download" class="nav-tab download-nav-btn">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4"/><polyline points="7 10 12 15 17 10"/><line x1="12" y1="15" x2="12" y2="3"/></svg>
                    Download Native
```

```
                </a>
            </nav>
        </header>

        <!-- LEXICON MAIN -->
        <main class="lexicon-container">
            <div class="lexicon-layout">
                <!-- LEFT: MANIFEST CODE VIEWER -->
                <div class="manifest-card card-glass">
                    <div class="panel-header">
                        <h2>Canonical Covenant Manifest</h2>
                        <span class="badge badge-proceed">Invariant Anchor</span>
                    </div>
                    <pre class="manifest-code"><code id="manifest-content">Loading canonical manifest.omni...</code>
</pre>
                </div>

                <!-- RIGHT: BIBLICAL LEXICON SEARCH & CARDS -->
                <div class="lexicon-card card-glass">
                    <div class="panel-header">
                        <h2>Biblical World Model Lexicon</h2>
                        <span class="badge badge-subtle" id="lexicon-count-badge">0 Entries</span>
                    </div>
                    <div class="lexicon-search-bar">
                        <input type="text" id="lexicon-search" class="form-input" placeholder="Search moral invariants,
scripture terms, or laws (e.g. truth, justice, life, void)...">
                    </div>
                    <div class="lexicon-entries-grid" id="lexicon-entries-grid">
                        <!-- Populated dynamically via app.js -->
                    </div>
                </div>
            </div>
        </main>

        <!-- FOOTER -->
        <footer class="app-footer">
            <div class="footer-left">
                <span class="footer-brand">Ω OMNICODE</span>
                <span class="footer-sep">&bull;</span>
                <span class="footer-copy">&copy; 2026 CreativeWorkzStudio LLC. All rights reserved.</span>
            </div>
            <div class="footer-center">
                <span>The Immutable Moral Anchor of Computation &bull; Rust Core 2024</span>
            </div>
            <div class="footer-right">
                <span class="footer-pill">Biblical Lexicon</span>
                <span class="footer-pill">v0.1.0</span>
            </div>
        </footer>
    </div>
    <script src="/app.js"></script>
</body>
</html>
```

<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/gui/model.html

HTML | 152 LINES | 719 WORDS | 8.9 KB

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="OmniCode Autonomous Cognitive AI Model & Neural-Symbolic Cortex — CreativeWorkzStudio LLC.">
    <title>OmniCode Cognitive AI Model — CreativeWorkzStudio LLC</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Fira+Code:wght@400;500;600;700&family=Outfit:wght@300;400;500;600;700;800&family=Cinzel:wght@500;700;800&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="/style.css">
</head>
<body>
    <div class="app-layout">
        <!-- GLOBAL NAVIGATION HEADER -->
        <header class="app-header" id="app-header">
            <div class="header-left">
                <a href="/studio" class="brand-container">
                    <span class="brand-symbol">Ω</span>
                    <div class="brand-text">
                        <span class="brand-title">OMNICODE</span>
                        <span class="brand-subtitle">CREATIVEWORKZSTUDIO LLC</span>
                    </div>
                </a>
                <div class="status-pill" id="status-pill" title="Engine runs 100% offline with zero cloud telemetry">
                    <span class="status-dot"></span>
                    <span class="status-text">OFFLINE COVENANT ACTIVE</span>
                </div>
            </div>

            <nav class="app-nav" aria-label="Main Navigation">
                <a href="/studio" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M12 2L2 7l10 5 10-5-10-5zM2 17l10 5 10-5M2 12l10 5 10-5"/></svg>
                    AI Studio
                </a>
                <a href="/terminal" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><polyline points="4 17 10 11 4 5"/><line x1="12" y1="19" x2="20" y2="19"/></svg>
                    CLI Terminal
                </a>
                <a href="/specs" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8z"/><polyline points="14 2 14 8 20 8"/></svg>
                    Language Specs
                </a>
                <a href="/model" class="nav-tab active">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M12 2a8 8 0 0 0-8 8c0 3.3 2 6.2 5 7.4V20a2 2 0 0 0 2 2h2a2 2 0 0 0 2-2v-2.6c3-1.2 5-4.1 5-7.4a8 8 0 0 0-8-8z"/></svg>
                    AI Model
                </a>
                <a href="/lexicon" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20"/><path d="M6.5 2H20v20H6.5A2.5 2.5 0 0 1 4 19.5v-15A2.5 2.5 0 0 1 6.5 2z"/></svg>
                    Covenant Lexicon
                </a>
                <a href="/scrolls" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M19 21l-7-5-7 5V5a2 2 0 0 1 2-2h10a2 2 0 0 1 2 2z"/></svg>
                    Scrolls
                </a>
                <a href="/cloud" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M18 10h-1.26A8 8 0 1 0 9 20h9a5 5 0 0 0 0-10z"/></svg>
                    Cloud &amp;amp; Edge
                </a>
                <a href="/download" class="nav-tab download-nav-btn">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4"/><polyline points="7 10 12 15 17 10"/><line x1="12" y1="15" x2="12" y2="3"/></svg>
                    Download Native
```

```
                </a>
            </nav>
        </header>

        <!-- MODEL MAIN -->
        <main class="doc-container">
            <div class="doc-hero card-glass">
                <div class="hero-badge">AUTONOMOUS COGNITIVE ENGINE</div>
                <h1>The OmniCode Cognitive AI Model</h1>
                <p class="hero-lead">
                    Engineered by <strong>CreativeWorkzStudio LLC</strong>, the OmniCode AI Model features a local
Neural-Symbolic deliberative loop that bridges freeform human discourse with deterministic 4-Block Turing reels, anchored in
biblical moral invariants and continuous experiential learning.
                </p>
            </div>

            <div class="doc-grid">
                <!-- AICORTEX -->
                <div class="doc-card card-glass">
                    <div class="doc-card-header">
                        <span class="doc-icon">🧠</span>
                        <h2>The AICortex Transduction Loop</h2>
                    </div>
                    <p>The deliberative cycle that transforms natural human thought into verified machine reels:</p>
                    <ol class="doc-list">
                        <li><strong>Intent Extraction:</strong> Parses raw sentences into semantic concepts and target
boundary types.</li>
                        <li><strong>Moral Invariant Grounding:</strong> Validates intent against the 7 biblical laws of
the Covenant Anchor.</li>
                        <li><strong>4-Block Synthesis:</strong> Emits formal Metadata, Opening, Body, and Closing blocks.
</li>
                        <li><strong>Autonomous Verification &amp;amp; Repair:</strong> Executes the Linter and Restorator to
prove correctness.</li>
                    </ol>
                </div>

                <!-- AUTONOMOUS RESTORATOR -->
                <div class="doc-card card-glass">
                    <div class="doc-card-header">
                        <span class="doc-icon">🩹</span>
                        <h2>Self-Healing Restorator</h2>
                    </div>
                    <p>When syntax or boundary violations occur, the Restorator applies surgical heuristics without human
intervention:</p>
                    <ul class="spec-list">
                        <li>Injects missing Block headers (Opening/Body/Closing).</li>
                        <li>Balances unmatched boundary brackets (<code>&lt; ... &gt;</code>, <code>[ ... ]</code>,
<code>( ... )</code>).</li>
                        <li>Repairs malformed ternary operators (<code>--&gt;</code>, <code>&lt;--</code>).</li>
                        <li>Certifies repaired candidates through isolated test oracles.</li>
                    </ul>
                </div>

                <!-- EPISODIC LEARNING -->
                <div class="doc-card card-glass">
                    <div class="doc-card-header">
                        <span class="doc-icon">📜</span>
                        <h2>Episodic Learning (Experiential Scrolls)</h2>
                    </div>
                    <p>Every deliberation emits an immutable episodic scroll (<code>.scroll</code>):</p>
                    <ul class="spec-list">
                        <li>Logs user prompt, synthesized code, applied repairs, and final quad-state.</li>
                        <li>Persists across restarts in local directory <code>scrolls/</code>.</li>
                        <li>Prevents moral drift by enforcing positive reinforcement for invariant-aligned outcomes.</li>
                    </ul>
                </div>

                <!-- ZERO TELEMETRY -->
                <div class="doc-card card-glass">
                    <div class="doc-card-header">
                        <span class="doc-icon">🔒</span>
                        <h2>Zero Cloud Telemetry Covenant</h2>
                    </div>
                    <p>Unlike commercial LLMs that upload your source code and telemetry to cloud providers, OmniCode
operates under an immutable covenant:</p>
                    <p><strong>100% of reasoning, parsing, repair heuristics, and learning occur locally on your machine.
</strong></p>
                </div>
```

```
            </div>
        </main>

        <!-- FOOTER -->
        <footer class="app-footer">
            <div class="footer-left">
                <span class="footer-brand">Ω OMNICODE</span>
                <span class="footer-sep">&bull;</span>
                <span class="footer-copy">&copy; 2026 CreativeWorkzStudio LLC. All rights reserved.</span>
            </div>
            <div class="footer-center">
                <span>Autonomous Cognitive Intelligence &bull; Rust Core 2024</span>
            </div>
            <div class="footer-right">
                <span class="footer-pill">Cognitive Model</span>
                <span class="footer-pill">v0.1.0</span>
            </div>
        </footer>
    </div>
</body>
</html>
```

<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/gui/scrolls.html

HTML | 146 LINES | 568 WORDS | 7.8 KB

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="OmniCode Episodic Memory Bank & Experiential Scrolls — CreativeWorkzStudio LLC.">
    <title>Episodic Memory Scrolls — CreativeWorkzStudio LLC</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?
family=Fira+Code:wght@400;500;600;700&family=Outfit:wght@300;400;500;600;700;800&family=Cinzel:wght@500;700;800&display=swap"
rel="stylesheet">
    <link rel="stylesheet" href="/style.css">
</head>
<body>
    <div class="app-layout">
        <!-- GLOBAL NAVIGATION HEADER -->
        <header class="app-header" id="app-header">
            <div class="header-left">
                <a href="/studio" class="brand-container">
                    <span class="brand-symbol">Ω</span>
                    <div class="brand-text">
                        <span class="brand-title">OMNICODE</span>
                        <span class="brand-subtitle">CREATIVEWORKZSTUDIO LLC</span>
                    </div>
                </a>
                <div class="status-pill" id="status-pill" title="Engine runs 100% offline with zero cloud telemetry">
                    <span class="status-dot"></span>
                    <span class="status-text">OFFLINE COVENANT ACTIVE</span>
                </div>
            </div>

            <nav class="app-nav" aria-label="Main Navigation">
                <a href="/studio" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
<path d="M12 2L2 7l10 5 10-5-10-5zM2 17l10 5 10-5M2 12l10 5 10-5"/></svg>
                    AI Studio
                </a>
                <a href="/terminal" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
<polyline points="4 17 10 11 4 5"/><line x1="12" y1="19" x2="20" y2="19"/></svg>
                    CLI Terminal
                </a>
                <a href="/specs" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
<path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8z"/><polyline points="14 2 14 8 20 8"/></svg>
                    Language Specs
                </a>
                <a href="/model" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
<path d="M12 2a8 8 0 0 0-8 8c0 3.3 2 6.2 5 7.4V20a2 2 0 0 0 2 2h2a2 2 0 0 0 2-2v-2.6c3-1.2 5-4.1 5-7.4a8 8 0 0 0-8-8z"/></svg>
                    AI Model
                </a>
                <a href="/lexicon" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
<path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20"/><path d="M6.5 2H20v20H6.5A2.5 2.5 0 0 1 4 19.5v-15A2.5 2.5 0 0 1 6.5 2z"/></svg>
                    Covenant Lexicon
                </a>
                <a href="/scrolls" class="nav-tab active">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
<path d="M19 21l-7-5-7 5V5a2 2 0 0 1 2-2h10a2 2 0 0 1 2 2z"/></svg>
                    Scrolls
                </a>
                <a href="/cloud" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
<path d="M18 10h-1.26A8 8 0 1 0 9 20h9a5 5 0 0 0 0-10z"/></svg>
                    Cloud &amp;amp; Edge
                </a>
                <a href="/download" class="nav-tab download-nav-btn">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
<path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4"/><polyline points="7 10 12 15 17 10"/><line x1="12" y1="15" x2="12" y2="3"/>
</svg>
                    Download Native
                </a>
```

```
            </nav>
        </header>

        <!-- SCROLLS MAIN -->
        <main class="scrolls-container">
            <div class="scrolls-card card-glass">
                <div class="panel-header">
                    <div class="panel-header-title">
                        <h1>Episodic Memory Bank &amp;amp; Learning Inscription</h1>
                        <p class="panel-desc">Experiential scrolls (.scroll) and traditional telemetry logs (.log)
accumulated by the internal AI model. Grounded learning preserves successful restorations without moral drift.</p>
                    </div>
                    <button class="btn btn-secondary" id="btn-refresh-scrolls">Refresh Memory</button>
                </div>

                <!-- METRICS CARDS -->
                <div class="metrics-grid" id="metrics-grid">
                    <div class="metric-card">
                        <span class="metric-num" id="metric-total">0</span>
                        <span class="metric-label">Inscribed Episodes</span>
                    </div>
                    <div class="metric-card card-green">
                        <span class="metric-num" id="metric-proceed">0</span>
                        <span class="metric-label">+1: PROCEED</span>
                    </div>
                    <div class="metric-card card-amber">
                        <span class="metric-num" id="metric-await">0</span>
                        <span class="metric-label">0: AWAIT</span>
                    </div>
                    <div class="metric-card card-red">
                        <span class="metric-num" id="metric-rollback">0</span>
                        <span class="metric-label">-1: ROLLBACK</span>
                    </div>
                    <div class="metric-card card-purple">
                        <span class="metric-num" id="metric-halt">0</span>
                        <span class="metric-label">NULL: HALT</span>
                    </div>
                    <div class="metric-card">
                        <span class="metric-num" id="metric-repairs">0</span>
                        <span class="metric-label">Repairs Learned</span>
                    </div>
                </div>

                <!-- SCROLLS EPISODES TABLE -->
                <div class="episodes-table-wrapper">
                    <table class="episodes-table">
                        <thead>
                            <tr>
                                <th>Session ID</th>
                                <th>Prompt Intent</th>
                                <th>Final State</th>
                                <th>Repairs</th>
                                <th>Failpoints</th>
                                <th>Scroll Path</th>
                            </tr>
                        </thead>
                        <tbody id="episodes-tbody">
                            <!-- Populated dynamically via JS -->
                        </tbody>
                    </table>
                </div>
            </div>
        </main>

        <!-- FOOTER -->
        <footer class="app-footer">
            <div class="footer-left">
                <span class="footer-brand">Ω OMNICODE</span>
                <span class="footer-sep">&bull;</span>
                <span class="footer-copy">&copy; 2026 CreativeWorkzStudio LLC. All rights reserved.</span>
            </div>
            <div class="footer-center">
                <span>Episodic Learning &bull; Experiential Memory Ledger &bull; Rust Core 2024</span>
            </div>
            <div class="footer-right">
                <span class="footer-pill">Episodic Scrolls</span>
                <span class="footer-pill">v0.1.0</span>
            </div>
```

```
        </footer>
    </div>
    <script src="/app.js"></script>
</body>
</html>
```

<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/gui/specs.html

HTML | 155 LINES | 798 WORDS | 9.6 KB

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="OmniCode Language Specification & 4-Block Formal Architecture — CreativeWorkzStudio LLC.">
    <title>OmniCode Language Specifications — CreativeWorkzStudio LLC</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Fira+Code:wght@400;500;600;700&family=Outfit:wght@300;400;500;600;700;800&family=Cinzel:wght@500;700;800&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="/style.css">
</head>
<body>
    <div class="app-layout">
        <!-- GLOBAL NAVIGATION HEADER -->
        <header class="app-header" id="app-header">
            <div class="header-left">
                <a href="/studio" class="brand-container">
                    <span class="brand-symbol">Ω</span>
                    <div class="brand-text">
                        <span class="brand-title">OMNICODE</span>
                        <span class="brand-subtitle">CREATIVEWORKZSTUDIO LLC</span>
                    </div>
                </a>
                <div class="status-pill" id="status-pill" title="Engine runs 100% offline with zero cloud telemetry">
                    <span class="status-dot"></span>
                    <span class="status-text">OFFLINE COVENANT ACTIVE</span>
                </div>
            </div>

            <nav class="app-nav" aria-label="Main Navigation">
                <a href="/studio" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M12 2L2 7l10 5 10-5-10-5zM2 17l10 5 10-5M2 12l10 5 10-5"/></svg>
                    AI Studio
                </a>
                <a href="/terminal" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><polyline points="4 17 10 11 4 5"/><line x1="12" y1="19" x2="20" y2="19"/></svg>
                    CLI Terminal
                </a>
                <a href="/specs" class="nav-tab active">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8z"/><polyline points="14 2 14 8 20 8"/></svg>
                    Language Specs
                </a>
                <a href="/model" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M12 2a8 8 0 0 0-8 8c0 3.3 2 6.2 5 7.4V20a2 2 0 0 0 2 2h2a2 2 0 0 0 2-2v-2.6c3-1.2 5-4.1 5-7.4a8 8 0 0 0-8-8z"/></svg>
                    AI Model
                </a>
                <a href="/lexicon" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20"/><path d="M6.5 2H20v20H6.5A2.5 2.5 0 0 1 4 19.5v-15A2.5 2.5 0 0 1 6.5 2z"/></svg>
                    Covenant Lexicon
                </a>
                <a href="/scrolls" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M19 21l-7-5-7 5V5a2 2 0 0 1 2-2h10a2 2 0 0 1 2 2z"/></svg>
                    Scrolls
                </a>
                <a href="/cloud" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M18 10h-1.26A8 8 0 1 0 9 20h9a5 5 0 0 0 0-10z"/></svg>
                    Cloud &amp;amp; Edge
                </a>
                <a href="/download" class="nav-tab download-nav-btn">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4"/><polyline points="7 10 12 15 17 10"/><line x1="12" y1="15" x2="12" y2="3"/></svg>
                    Download Native
```

```
                </a>
            </nav>
        </header>

        <!-- SPECS MAIN -->
        <main class="doc-container">
            <div class="doc-hero card-glass">
                <div class="hero-badge">CANONICAL LANGUAGE SPECIFICATION</div>
                <h1>The OmniCode Programming Language</h1>
                <p class="hero-lead">
                    An ontological, dual-readable programming language engineered by <strong>CreativeWorkzStudio
LLC</strong> from first principles. Operates with bare-metal low-level precision while natively expressing balanced-ternary quad-
state state machines, causal laws, and ethical invariants.
                </p>
            </div>

            <div class="doc-grid">
                <!-- 4-BLOCK ARCHITECTURE -->
                <div class="doc-card card-glass">
                    <div class="doc-card-header">
                        <span class="doc-icon">🏛️</span>
                        <h2>The 4-Block Architecture</h2>
                    </div>
                    <p>Every valid OmniCode unit must be structured into four sequential compilation blocks:</p>
                    <ol class="doc-list">
                        <li><strong>Block 1: Metadata Block (Manifest)</strong> — Declares ontological identity, covenant
anchor, and boundary classifications.</li>
                        <li><strong>Block 2: Opening Block (Opening { ... })</strong> — Import libraries, assert
immutable preconditions, and establish initial state vectors.</li>
                        <li><strong>Block 3: Body Block (Body [ ... ])</strong> — Execute causal logic, balanced ternary
rolls, and iterative computations.</li>
                        <li><strong>Block 4: Closing Block (Closing { ... })</strong> — Verify post-conditions, reconcile
invariants, and permanently seal transaction state.</li>
                    </ol>
                </div>

                <!-- BALANCED TERNARY RADIX-3 -->
                <div class="doc-card card-glass">
                    <div class="doc-card-header">
                        <span class="doc-icon">⚖️</span>
                        <h2>Balanced Ternary Quad-State Logic</h2>
                    </div>
                    <p>Unlike binary logic (0 or 1), OmniCode operates on Balanced Radix-3 with a terminal Null state
(T4):</p>
                    <ul class="spec-list">
                        <li><strong>+1 (Proceed):</strong> Affirmative forward vector (<code>--&gt;</code>). Commits
state mutations and advances the Turing reel.</li>
                        <li><strong>0 (Await):</strong> Standby, pass-through equilibrium, or asynchronous yield.</li>
                        <li><strong>-1 (Rollback):</strong> Negation and boundary breach. Automatically unwinds
retroactive vectors (<code>&lt;--</code>).</li>
                        <li><strong>NULL (Halt):</strong> Absorbing bottom type (&perp;). Traps unrecoverable faults and
terminates process.</li>
                    </ul>
                </div>

                <!-- BOUNDARIES -->
                <div class="doc-card card-glass">
                    <div class="doc-card-header">
                        <span class="doc-icon">🛡️</span>
                        <h2>The 6 Boundary Types</h2>
                    </div>
                    <ul class="spec-list">
                        <li><code>&lt; Invariant &gt;</code> — Absolute unalterable boundary. Immutable across all
scopes.</li>
                        <li><code>&lt; Finite &gt;</code> — Strictly bounded range, buffer capacity, or count.</li>
                        <li><code>&lt; Indefinite &gt;</code> — Elastic state awaiting runtime determination.</li>
                        <li><code>&lt; Continuous &gt;</code> — Streaming uninterrupted sequence or real-time feed.</li>
                        <li><code>&lt; Absorb &gt;</code> — Trap boundary that swallows violations into safe fallbacks.
</li>
                        <li><code>&lt; Oscillate &gt;</code> — Alternating dual-phase harmonic boundary.</li>
                    </ul>
                </div>

                <!-- FORMAL ASIDES -->
                <div class="doc-card card-glass">
                    <div class="doc-card-header">
                        <span class="doc-icon">💬</span>
                        <h2>Formal Asides (Dual Readability)</h2>
```

```
                    </div>
                    <p>Parentheses <code>( ... )</code> in OmniCode are not inert comments—they are <strong>ontological semantic anchors</strong> parsed by the compiler:</p>
                    <pre class="code-snippet"><code>roll_integrity [ health &gt; 0 ] ( Vital signs active and confirmed )</code></pre>
                    <p class="subtext">The inner text informs the AICortex and linter of human moral intent while the expression enforces rigorous machine truth.</p>
                </div>
            </div>
        </main>

        <!-- FOOTER -->
        <footer class="app-footer">
            <div class="footer-left">
                <span class="footer-brand">Ω OMNICODE</span>
                <span class="footer-sep">&bull;</span>
                <span class="footer-copy">&copy; 2026 CreativeWorkzStudio LLC. All rights reserved.</span>
            </div>
            <div class="footer-center">
                <span>Formal Dual-Readable Specifications &bull; Rust Core 2024</span>
            </div>
            <div class="footer-right">
                <span class="footer-pill">Language Specs</span>
                <span class="footer-pill">v0.1.0</span>
            </div>
        </footer>
    </div>
</body>
</html>
```

<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/gui/studio.html

HTML | 136 LINES | 610 WORDS | 7.7 KB

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="OmniCode AI Prompt Studio & Code Synthesis — CreativeWorkzStudio LLC.">
    <title>OmniCode AI Studio — CreativeWorkzStudio LLC</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?
family=Fira+Code:wght@400;500;600;700&family=Outfit:wght@300;400;500;600;700;800&family=Cinzel:wght@500;700;800&display=swap"
rel="stylesheet">
    <link rel="stylesheet" href="/style.css">
</head>
<body>
    <div class="app-layout">
        <!-- GLOBAL NAVIGATION HEADER -->
        <header class="app-header" id="app-header">
            <div class="header-left">
                <a href="/studio" class="brand-container">
                    <span class="brand-symbol">Ω</span>
                    <div class="brand-text">
                        <span class="brand-title">OMNICODE</span>
                        <span class="brand-subtitle">CREATIVEWORKZSTUDIO LLC</span>
                    </div>
                </a>
                <div class="status-pill" id="status-pill" title="Engine runs 100% offline with zero cloud telemetry">
                    <span class="status-dot"></span>
                    <span class="status-text">OFFLINE COVENANT ACTIVE</span>
                </div>
            </div>

            <nav class="app-nav" aria-label="Main Navigation">
                <a href="/studio" class="nav-tab active">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
<path d="M12 2L2 7l10 5 10-5-10-5zM2 17l10 5 10-5M2 12l10 5 10-5"/></svg>
                    AI Studio
                </a>
                <a href="/terminal" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
<polyline points="4 17 10 11 4 5"/><line x1="12" y1="19" x2="20" y2="19"/></svg>
                    CLI Terminal
                </a>
                <a href="/specs" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
<path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8z"/><polyline points="14 2 14 8 20 8"/></svg>
                    Language Specs
                </a>
                <a href="/model" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
<path d="M12 2a8 8 0 0 0-8 8c0 3.3 2 6.2 5 7.4V20a2 2 0 0 0 2 2h2a2 2 0 0 0 2-2v-2.6c3-1.2 5-4.1 5-7.4a8 8 0 0 0-8-8z"/></svg>
                    AI Model
                </a>
                <a href="/lexicon" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
<path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20"/><path d="M6.5 2H20v20H6.5A2.5 2.5 0 0 1 4 19.5v-15A2.5 2.5 0 0 1 6.5 2z"/></svg>
                    Covenant Lexicon
                </a>
                <a href="/scrolls" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
<path d="M19 21l-7-5-7 5V5a2 2 0 0 1 2-2h10a2 2 0 0 1 2 2z"/></svg>
                    Scrolls
                </a>
                <a href="/cloud" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
<path d="M18 10h-1.26A8 8 0 1 0 9 20h9a5 5 0 0 0 0-10z"/></svg>
                    Cloud &amp;amp; Edge
                </a>
                <a href="/download" class="nav-tab download-nav-btn">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2">
<path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4"/><polyline points="7 10 12 15 17 10"/><line x1="12" y1="15" x2="12" y2="3"/>
</svg>
                    Download Native
                </a>
```

```
            </nav>
        </header>

        <!-- STUDIO MAIN -->
        <main class="studio-container">
            <div class="studio-grid">
                <!-- LEFT: PROMPT INPUT & INTENT FORM -->
                <div class="card-glass prompt-card">
                    <div class="panel-header">
                        <h2>Natural Language Intent Prompt</h2>
                        <span class="offline-note">🔒 Local execution &bull; 0% Cloud Telemetry</span>
                    </div>
                    <p class="panel-desc">
                        Provide a natural language description. The neural-symbolic transducer aligns it with moral
invariants and generates verified 4-block OmniCode.
                    </p>
                    <textarea id="prompt-input" class="prompt-textarea" rows="6" placeholder="Describe the program you
wish to create... (e.g. 'Build an immutable treasury ledger that verifies balanced transactions and rejects unauthorized
debits')"></textarea>

                    <div class="prompt-actions">
                        <button class="btn btn-accent" id="btn-generate">
                            <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2"><polygon points="5 3 19 12 5 21 5 3"/></svg>
                            Synthesize OmniCode
                        </button>
                        <button class="btn btn-secondary" id="btn-sample-prompt">Sample Intent</button>
                    </div>

                    <!-- RESPONSE STREAM -->
                    <div class="ai-response-box" id="ai-response-box">
                        <h3>AICortex Deliberation Summary</h3>
                        <div id="ai-response-text" class="response-text">Awaiting input prompt...</div>
                    </div>
                </div>

                <!-- RIGHT: SYNTHESIZED OMNICODE CODE VIEWER -->
                <div class="card-glass code-card">
                    <div class="panel-header">
                        <h2>Synthesized 4-Block OmniCode</h2>
                        <div class="badge-group">
                            <span class="badge badge-proceed" id="code-state-badge">+1:PROCEED</span>
                            <button class="btn btn-sm btn-secondary" id="btn-copy-code">Copy</button>
                        </div>
                    </div>
                    <pre class="code-editor" id="code-output"><code id="code-output-inner"># 
================================================================================
# BLOCK 1: METADATA BLOCK
# ================================================================================
Manifest &lt; Treasury.Ledger &gt; ( Moral Equilibrium ) {
    author: &lt; \"CreativeWorkzStudio LLC\" &gt;,
    resonance: &lt; +1:PROCEED &gt;
}
</code></pre>
                </div>
            </div>
        </main>

        <!-- FOOTER -->
        <footer class="app-footer">
            <div class="footer-left">
                <span class="footer-brand">Ω OMNICODE</span>
                <span class="footer-sep">&bull;</span>
                <span class="footer-copy">&copy; 2026 CreativeWorkzStudio LLC. All rights reserved.</span>
            </div>
            <div class="footer-center">
                <span>Dual-Readable Ontological Programming Language &amp;amp; Cognitive AI Model</span>
            </div>
            <div class="footer-right">
                <span class="footer-pill">Rust Core Edition 2024</span>
                <span class="footer-pill">v0.1.0</span>
            </div>
        </footer>
    </div>
    <script src="/app.js"></script>
</body>
</html>
```

<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/gui/style.css

CSS | 1747 LINES | 3804 WORDS | 37.7 KB

```css
/* ========================================================================
   OMNICODE STUDIO & COGNITIVE TERMINAL STYLESHEET
   Aesthetics: Deep Obsidian, Radiant Neon Glassmorphism, Dual Monospace Code
   ======================================================================== */

:root {
    /* Color Palette */
    --bg-base: #07090e;
    --bg-surface: #0d121d;
    --bg-elevated: #131a29;
    --border-subtle: rgba(255, 255, 255, 0.08);
    --border-bright: rgba(0, 240, 255, 0.3);

    /* Text Colors */
    --text-primary: #f0f4fc;
    --text-secondary: #94a3b8;
    --text-muted: #64748b;
    --text-highlight: #ffffff;

    /* Theme Jewel Accents */
    --cyan: #00f0ff;
    --cyan-glow: rgba(0, 240, 255, 0.25);
    --gold: #ffd700;
    --gold-glow: rgba(255, 215, 0, 0.25);

    /* Balanced Ternary Quad-State Colors */
    --state-proceed: #00e676;
    --state-proceed-glow: rgba(0, 230, 118, 0.3);
    --state-await: #ffaa00;
    --state-await-glow: rgba(255, 170, 0, 0.3);
    --state-rollback: #ff3d71;
    --state-rollback-glow: rgba(255, 61, 113, 0.3);
    --state-halt: #9d4edd;
    --state-halt-glow: rgba(157, 78, 221, 0.3);

    /* Typography */
    --font-sans: 'Outfit', -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
    --font-mono: 'Fira Code', Consolas, Monaco, monospace;
    --font-serif: 'Cinzel', Georgia, serif;

    /* Shadows & Radii */
    --radius-sm: 6px;
    --radius-md: 10px;
    --radius-lg: 16px;
    --shadow-glass: 0 8px 32px 0 rgba(0, 0, 0, 0.45);
    --glass-bg: rgba(13, 18, 29, 0.7);
    --glass-border: 1px solid rgba(255, 255, 255, 0.08);
}

/* Base Reset */
*, *::before, *::after {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
}

body {
    background-color: var(--bg-base);
    color: var(--text-primary);
    font-family: var(--font-sans);
    line-height: 1.6;
    overflow-x: hidden;
    min-height: 100vh;
    background-image:
        radial-gradient(circle at 15% 10%, rgba(0, 240, 255, 0.05) 0%, transparent 40%),
        radial-gradient(circle at 85% 80%, rgba(255, 215, 0, 0.04) 0%, transparent 45%),
        radial-gradient(circle at 50% 50%, rgba(157, 78, 221, 0.03) 0%, transparent 60%);
    background-attachment: fixed;
}

/* App Layout */
.app-layout {
    display: flex;
    flex-direction: column;
```

```
    min-height: 100vh;
}

/* ------------------------------------------------------------------------
   HEADER & NAVIGATION
   ------------------------------------------------------------------------ */
.app-header {
    display: flex;
    align-items: center;
    justify-content: space-between;
    padding: 1rem 2rem;
    background: rgba(7, 9, 14, 0.85);
    backdrop-filter: blur(20px);
    -webkit-backdrop-filter: blur(20px);
    border-bottom: var(--glass-border);
    position: sticky;
    top: 0;
    z-index: 100;
}

.header-left {
    display: flex;
    align-items: center;
    gap: 1.5rem;
}

.brand-container {
    display: flex;
    align-items: center;
    gap: 0.75rem;
}

.brand-symbol {
    font-size: 1.8rem;
    font-weight: 800;
    color: var(--cyan);
    text-shadow: 0 0 16px var(--cyan-glow);
    font-family: var(--font-serif);
}

.brand-text {
    display: flex;
    flex-direction: column;
}

.brand-title {
    font-size: 1.15rem;
    font-weight: 800;
    letter-spacing: 0.15em;
    color: var(--text-highlight);
}

.brand-subtitle {
    font-size: 0.65rem;
    letter-spacing: 0.2em;
    color: var(--gold);
    font-weight: 600;
}

.status-pill {
    display: flex;
    align-items: center;
    gap: 0.5rem;
    padding: 0.35rem 0.85rem;
    background: rgba(0, 230, 118, 0.08);
    border: 1px solid rgba(0, 230, 118, 0.3);
    border-radius: 999px;
    font-size: 0.72rem;
    font-weight: 600;
    color: var(--state-proceed);
    letter-spacing: 0.05em;
}

.status-dot {
    width: 7px;
    height: 7px;
    border-radius: 50%;
    background: var(--state-proceed);
```

```
    box-shadow: 0 0 8px var(--state-proceed);
    animation: pulse-dot 2s infinite ease-in-out;
}

@keyframes pulse-dot {
    0%, 100% { opacity: 1; transform: scale(1); }
    50% { opacity: 0.4; transform: scale(0.85); }
}

.app-nav {
    display: flex;
    gap: 0.5rem;
}

.nav-tab {
    display: flex;
    align-items: center;
    gap: 0.5rem;
    padding: 0.6rem 1.1rem;
    background: transparent;
    border: 1px solid transparent;
    border-radius: var(--radius-md);
    color: var(--text-secondary);
    font-family: var(--font-sans);
    font-size: 0.9rem;
    font-weight: 500;
    cursor: pointer;
    transition: all 0.25s ease;
}

.nav-tab:hover {
    color: var(--text-primary);
    background: rgba(255, 255, 255, 0.04);
}

.nav-tab.active {
    color: var(--cyan);
    background: rgba(0, 240, 255, 0.08);
    border-color: rgba(0, 240, 255, 0.35);
    box-shadow: 0 0 15px rgba(0, 240, 255, 0.15);
}

.ternary-quad-badge {
    padding: 0.4rem 0.9rem;
    background: var(--bg-surface);
    border: 1px solid var(--border-subtle);
    border-radius: var(--radius-sm);
    font-family: var(--font-mono);
    font-size: 0.75rem;
    font-weight: 600;
    color: var(--gold);
    letter-spacing: 0.05em;
}

/* ------------------------------------------------------------------------
   MAIN & VIEW PANELS
   ------------------------------------------------------------------------ */
.app-main {
    flex: 1;
    padding: 2rem;
    max-width: 1600px;
    margin: 0 auto;
    width: 100%;
}

.view-panel {
    display: none;
    animation: fadeIn 0.3s ease;
}

.view-panel.active {
    display: block;
}

@keyframes fadeIn {
    from { opacity: 0; transform: translateY(6px); }
    to { opacity: 1; transform: translateY(0); }
}
```

```
231
232 /* Glassmorphism Cards */
233 .card-glass {
234     background: var(--glass-bg);
235     backdrop-filter: blur(16px);
236     -webkit-backdrop-filter: blur(16px);
237     border: var(--glass-border);
238     border-radius: var(--radius-lg);
239     box-shadow: var(--shadow-glass);
240     padding: 1.5rem;
241 }
242
243 .panel-header {
244     display: flex;
245     justify-content: space-between;
246     align-items: flex-start;
247     margin-bottom: 1.25rem;
248     gap: 1rem;
249 }
250
251 .panel-header-title h1 {
252     font-size: 1.5rem;
253     font-weight: 700;
254     color: var(--text-highlight);
255     margin-bottom: 0.35rem;
256 }
257
258 .panel-desc {
259     font-size: 0.88rem;
260     color: var(--text-secondary);
261     max-width: 850px;
262 }
263
264 /* ------------------------------------------------------------------------
265    VIEW 1: PROMPT WORKBENCH
266    ------------------------------------------------------------------------ */
267 .quadstate-indicator {
268     display: flex;
269     align-items: center;
270     gap: 0.6rem;
271     padding: 0.5rem 1.1rem;
272     background: rgba(255, 170, 0, 0.1);
273     border: 1px solid var(--state-await);
274     border-radius: var(--radius-md);
275     color: var(--state-await);
276     font-family: var(--font-mono);
277     font-size: 0.85rem;
278     font-weight: 700;
279     transition: all 0.3s ease;
280 }
281
282 .quadstate-indicator.proceed {
283     background: rgba(0, 230, 118, 0.1);
284     border-color: var(--state-proceed);
285     color: var(--state-proceed);
286     box-shadow: 0 0 15px var(--state-proceed-glow);
287 }
288
289 .quadstate-indicator.rollback {
290     background: rgba(255, 61, 113, 0.1);
291     border-color: var(--state-rollback);
292     color: var(--state-rollback);
293     box-shadow: 0 0 15px var(--state-rollback-glow);
294 }
295
296 .quadstate-indicator.halt {
297     background: rgba(157, 78, 221, 0.1);
298     border-color: var(--state-halt);
299     color: var(--state-halt);
300     box-shadow: 0 0 15px var(--state-halt-glow);
301 }
302
303 .prompt-chips-container {
304     display: flex;
305     flex-wrap: wrap;
306     align-items: center;
307     gap: 0.5rem;
308     margin-bottom: 1.25rem;
```

```
}

.chip-label {
    font-size: 0.8rem;
    font-weight: 600;
    color: var(--text-muted);
    text-transform: uppercase;
    letter-spacing: 0.05em;
}

.prompt-chip, .term-btn {
    background: rgba(255, 255, 255, 0.04);
    border: 1px solid var(--border-subtle);
    border-radius: 999px;
    padding: 0.35rem 0.85rem;
    color: var(--text-secondary);
    font-size: 0.82rem;
    cursor: pointer;
    transition: all 0.2s ease;
}

.prompt-chip:hover, .term-btn:hover {
    background: rgba(0, 240, 255, 0.12);
    border-color: var(--cyan);
    color: var(--text-highlight);
    transform: translateY(-1px);
}

.prompt-chip.chip-warning {
    border-color: rgba(255, 61, 113, 0.3);
    color: #ff7597;
}

.prompt-chip.chip-warning:hover {
    background: rgba(255, 61, 113, 0.15);
    border-color: var(--state-rollback);
}

.prompt-form textarea {
    width: 100%;
    background: var(--bg-surface);
    border: 1px solid var(--border-subtle);
    border-radius: var(--radius-md);
    padding: 1rem;
    color: var(--text-primary);
    font-family: var(--font-sans);
    font-size: 1rem;
    resize: vertical;
    outline: none;
    transition: border-color 0.2s ease, box-shadow 0.2s ease;
}

.prompt-form textarea:focus {
    border-color: var(--cyan);
    box-shadow: 0 0 15px var(--cyan-glow);
}

.form-actions {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-top: 0.85rem;
}

.offline-note {
    font-size: 0.78rem;
    color: var(--text-muted);
}

.btn {
    display: inline-flex;
    align-items: center;
    gap: 0.5rem;
    padding: 0.65rem 1.35rem;
    border-radius: var(--radius-md);
    font-family: var(--font-sans);
    font-size: 0.92rem;
    font-weight: 600;
```

```
    cursor: pointer;
    transition: all 0.25s ease;
    border: none;
}

.btn-primary {
    background: linear-gradient(135deg, #00f0ff 0%, #0088ff 100%);
    color: #050811;
    box-shadow: 0 4px 15px rgba(0, 240, 255, 0.3);
}

.btn-primary:hover {
    filter: brightness(1.15);
    transform: translateY(-1px);
    box-shadow: 0 6px 20px rgba(0, 240, 255, 0.45);
}

.btn-secondary {
    background: rgba(255, 255, 255, 0.06);
    border: 1px solid var(--border-subtle);
    color: var(--text-secondary);
}

.btn-secondary:hover {
    background: rgba(255, 255, 255, 0.1);
    color: var(--text-primary);
}

/* Studio Output Split Grid */
.studio-output-grid {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 1.5rem;
    margin-top: 1.5rem;
}

.card-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 1rem;
    border-bottom: 1px solid rgba(255, 255, 255, 0.05);
    padding-bottom: 0.75rem;
}

.card-title-group {
    display: flex;
    align-items: center;
    gap: 0.5rem;
}

.card-title-group h2 {
    font-size: 1.05rem;
    font-weight: 600;
    color: var(--text-highlight);
}

.session-badge {
    font-family: var(--font-mono);
    font-size: 0.75rem;
    color: var(--cyan);
    background: rgba(0, 240, 255, 0.08);
    padding: 0.25rem 0.6rem;
    border-radius: var(--radius-sm);
}

.response-body {
    min-height: 250px;
    font-size: 0.95rem;
    line-height: 1.7;
    white-space: pre-wrap;
    color: #e2e8f0;
}

.placeholder-text {
    color: var(--text-muted);
    font-style: italic;
    padding-top: 2rem;
```

```
}

.card-footer {
    display: flex;
    flex-wrap: wrap;
    gap: 1rem;
    margin-top: 1rem;
    padding-top: 0.75rem;
    border-top: 1px solid rgba(255, 255, 255, 0.05);
    font-family: var(--font-mono);
    font-size: 0.75rem;
    color: var(--text-muted);
}

.code-body {
    background: #05070c;
    border-radius: var(--radius-md);
    padding: 1rem;
    max-height: 480px;
    overflow-y: auto;
    border: 1px solid rgba(255, 255, 255, 0.05);
}

.code-body pre code {
    font-family: var(--font-mono);
    font-size: 0.85rem;
    line-height: 1.5;
    color: #a5b4fc;
}

.btn-icon {
    background: rgba(255, 255, 255, 0.05);
    border: 1px solid var(--border-subtle);
    color: var(--text-secondary);
    border-radius: var(--radius-sm);
    padding: 0.3rem 0.7rem;
    font-size: 0.75rem;
    cursor: pointer;
}

.btn-icon:hover {
    color: var(--text-highlight);
    background: rgba(255, 255, 255, 0.1);
}

/* ------------------------------------------------------------------------
   VIEW 2: CLI TERMINAL
   ------------------------------------------------------------------------ */
.terminal-shortcuts {
    display: flex;
    flex-wrap: wrap;
    align-items: center;
    gap: 0.5rem;
    margin-bottom: 1rem;
}

.term-btn {
    font-family: var(--font-mono);
    font-size: 0.75rem;
}

.terminal-window {
    background: #04060a;
    border: 1px solid rgba(0, 240, 255, 0.2);
    border-radius: var(--radius-md);
    padding: 1.25rem;
    min-height: 480px;
    max-height: 600px;
    overflow-y: auto;
    font-family: var(--font-mono);
    font-size: 0.88rem;
    line-height: 1.6;
    color: #4ade80;
    box-shadow: inset 0 0 30px rgba(0, 0, 0, 0.8);
}

.terminal-banner pre {
    color: var(--cyan);
```

```
    margin-bottom: 1rem;
    font-family: inherit;
    white-space: pre-wrap;
}

.terminal-stream {
    white-space: pre-wrap;
    word-break: break-all;
}

.term-line-cmd {
    color: var(--gold);
    font-weight: 600;
    margin-top: 0.75rem;
}

.term-line-stdout {
    color: #e2e8f0;
}

.term-line-stderr {
    color: var(--state-rollback);
}

.terminal-input-bar {
    display: flex;
    align-items: center;
    gap: 0.75rem;
    margin-top: 1rem;
    background: var(--bg-surface);
    border: 1px solid var(--border-subtle);
    border-radius: var(--radius-md);
    padding: 0.5rem 1rem;
}

.prompt-prefix {
    font-family: var(--font-mono);
    font-size: 0.9rem;
    font-weight: 700;
    color: var(--cyan);
}

.terminal-input-bar input {
    flex: 1;
    background: transparent;
    border: none;
    color: var(--text-highlight);
    font-family: var(--font-mono);
    font-size: 0.92rem;
    outline: none;
}

/* ------------------------------------------------------------------------
   VIEW 3: COVENANT & LEXICON
   ------------------------------------------------------------------------ */
.lexicon-container {
    display: grid;
    grid-template-columns: 1fr 1.35fr;
    gap: 1.5rem;
}

.column-badge {
    font-size: 0.65rem;
    letter-spacing: 0.15em;
    font-weight: 700;
    color: var(--gold);
    text-transform: uppercase;
}

.column-header h2 {
    font-size: 1.35rem;
    color: var(--text-highlight);
    margin: 0.25rem 0 0.5rem 0;
}

.column-desc {
    font-size: 0.85rem;
    color: var(--text-secondary);
```

```
    margin-bottom: 1.25rem;
}

.laws-list {
    display: flex;
    flex-direction: column;
    gap: 0.85rem;
    max-height: 750px;
    overflow-y: auto;
    padding-right: 0.5rem;
}

.law-card {
    background: var(--bg-surface);
    border: 1px solid var(--border-subtle);
    border-radius: var(--radius-md);
    padding: 1rem;
    transition: transform 0.2s ease, border-color 0.2s ease;
}

.law-card:hover {
    border-color: rgba(255, 215, 0, 0.35);
    transform: translateX(3px);
}

.law-title-row {
    display: flex;
    align-items: center;
    gap: 0.75rem;
    margin-bottom: 0.5rem;
}

.law-num {
    font-family: var(--font-serif);
    font-size: 0.9rem;
    font-weight: 700;
    color: var(--gold);
    width: 24px;
}

.law-name {
    flex: 1;
    font-size: 0.95rem;
    font-weight: 600;
    color: var(--text-highlight);
}

.state-chip {
    font-family: var(--font-mono);
    font-size: 0.68rem;
    font-weight: 700;
    padding: 0.2rem 0.55rem;
    border-radius: var(--radius-sm);
}

.state-proceed {
    background: rgba(0, 230, 118, 0.12);
    color: var(--state-proceed);
    border: 1px solid rgba(0, 230, 118, 0.3);
}

.state-await {
    background: rgba(255, 170, 0, 0.12);
    color: var(--state-await);
    border: 1px solid rgba(255, 170, 0, 0.3);
}

.state-halt {
    background: rgba(157, 78, 221, 0.12);
    color: var(--state-halt);
    border: 1px solid rgba(157, 78, 221, 0.3);
}

.law-quote {
    font-size: 0.82rem;
    font-style: italic;
    color: #cbd5e1;
    margin-bottom: 0.5rem;
```

```
    line-height: 1.5;
}

.law-rule {
    font-size: 0.75rem;
    font-weight: 600;
    color: var(--cyan);
}

.lexicon-controls {
    margin-bottom: 1.25rem;
}

#lexicon-search-input {
    width: 100%;
    background: var(--bg-surface);
    border: 1px solid var(--border-subtle);
    border-radius: var(--radius-md);
    padding: 0.75rem 1rem;
    color: var(--text-primary);
    font-size: 0.9rem;
    outline: none;
    margin-bottom: 0.75rem;
}

#lexicon-search-input:focus {
    border-color: var(--cyan);
}

.domain-filters {
    display: flex;
    flex-wrap: wrap;
    gap: 0.4rem;
}

.filter-btn {
    background: rgba(255, 255, 255, 0.04);
    border: 1px solid var(--border-subtle);
    border-radius: var(--radius-sm);
    padding: 0.3rem 0.75rem;
    color: var(--text-secondary);
    font-size: 0.75rem;
    cursor: pointer;
    transition: all 0.2s ease;
}

.filter-btn:hover {
    color: var(--text-primary);
    background: rgba(255, 255, 255, 0.08);
}

.filter-btn.active {
    background: rgba(0, 240, 255, 0.12);
    border-color: var(--cyan);
    color: var(--cyan);
    font-weight: 600;
}

.lexicon-entries-grid {
    display: flex;
    flex-direction: column;
    gap: 0.85rem;
    max-height: 650px;
    overflow-y: auto;
    padding-right: 0.5rem;
}

.entry-card {
    background: var(--bg-surface);
    border: 1px solid var(--border-subtle);
    border-radius: var(--radius-md);
    padding: 1rem;
    transition: all 0.2s ease;
}

.entry-card:hover {
    border-color: rgba(0, 240, 255, 0.3);
}
```

```
777
778 .entry-header {
779     display: flex;
780     justify-content: space-between;
781     align-items: center;
782     margin-bottom: 0.5rem;
783 }
784
785 .entry-term {
786     font-size: 1.05rem;
787     font-weight: 700;
788     color: var(--cyan);
789     text-transform: uppercase;
790     letter-spacing: 0.05em;
791 }
792
793 .entry-domain {
794     font-size: 0.72rem;
795     color: var(--gold);
796     background: rgba(255, 215, 0, 0.08);
797     padding: 0.2rem 0.5rem;
798     border-radius: var(--radius-sm);
799 }
800
801 .entry-def {
802     font-size: 0.85rem;
803     color: #e2e8f0;
804     margin-bottom: 0.65rem;
805 }
806
807 .entry-witnesses {
808     background: #05070d;
809     border-radius: var(--radius-sm);
810     padding: 0.6rem 0.8rem;
811     font-size: 0.78rem;
812     line-height: 1.5;
813     color: #94a3b8;
814     margin-bottom: 0.5rem;
815 }
816
817 .witness-title {
818     font-weight: 700;
819     color: var(--text-highlight);
820     margin-right: 0.35rem;
821 }
822
823 .entry-guidance {
824     font-size: 0.78rem;
825     color: var(--state-proceed);
826     font-weight: 500;
827 }
828
829 /* ------------------------------------------------------------------------
830    VIEW 4: EPISODIC SCROLLS
831    ------------------------------------------------------------------------ */
832 .metrics-grid {
833     display: grid;
834     grid-template-columns: repeat(6, 1fr);
835     gap: 1rem;
836     margin-bottom: 1.5rem;
837 }
838
839 .metric-card {
840     background: var(--bg-surface);
841     border: 1px solid var(--border-subtle);
842     border-radius: var(--radius-md);
843     padding: 1rem;
844     display: flex;
845     flex-direction: column;
846     align-items: center;
847     text-align: center;
848 }
849
850 .metric-num {
851     font-size: 1.75rem;
852     font-weight: 800;
853     font-family: var(--font-mono);
854     color: var(--text-highlight);
```

```
}

.metric-label {
    font-size: 0.72rem;
    color: var(--text-muted);
    text-transform: uppercase;
    letter-spacing: 0.05em;
    margin-top: 0.25rem;
}

.card-green .metric-num { color: var(--state-proceed); }
.card-amber .metric-num { color: var(--state-await); }
.card-red .metric-num { color: var(--state-rollback); }
.card-purple .metric-num { color: var(--state-halt); }

.episodes-table-wrapper {
    background: #05070c;
    border: 1px solid var(--border-subtle);
    border-radius: var(--radius-md);
    overflow-x: auto;
    max-height: 520px;
}

.episodes-table {
    width: 100%;
    border-collapse: collapse;
    font-size: 0.85rem;
    text-align: left;
}

.episodes-table th {
    background: rgba(255, 255, 255, 0.03);
    padding: 0.85rem 1rem;
    font-weight: 600;
    color: var(--text-muted);
    border-bottom: 1px solid var(--border-subtle);
    text-transform: uppercase;
    font-size: 0.72rem;
    letter-spacing: 0.05em;
}

.episodes-table td {
    padding: 0.85rem 1rem;
    border-bottom: 1px solid rgba(255, 255, 255, 0.03);
    color: #cbd5e1;
}

.episodes-table tr:hover td {
    background: rgba(0, 240, 255, 0.02);
}

/* Responsive */
@media (max-width: 1100px) {
    .studio-output-grid,
    .lexicon-container {
        grid-template-columns: 1fr;
    }
    .metrics-grid {
        grid-template-columns: repeat(3, 1fr);
    }
}

@media (max-width: 768px) {
    .app-header {
        flex-direction: column;
        gap: 1rem;
        padding: 1rem;
    }
    .metrics-grid {
        grid-template-columns: repeat(2, 1fr);
    }
}

/* ------------------------------------------------------------------------
   CREATIVEWORKZSTUDIO LLC BRANDING & DOCUMENTATION STYLES
   ------------------------------------------------------------------------ */
.cws-badge {
    background: linear-gradient(135deg, rgba(255, 215, 0, 0.15), rgba(255, 170, 0, 0.08));
```

```
    border: 1px solid rgba(255, 215, 0, 0.35);
    color: var(--gold);
    font-size: 0.75rem;
    font-weight: 700;
    padding: 0.35rem 0.85rem;
    border-radius: 9999px;
    letter-spacing: 0.04em;
    text-transform: uppercase;
    display: inline-flex;
    align-items: center;
    gap: 0.4rem;
    box-shadow: 0 0 12px rgba(255, 215, 0, 0.1);
}

.terminal-ctrl-group {
    display: flex;
    gap: 0.5rem;
    align-items: center;
}

/* Documentation Layout */
.doc-container {
    display: flex;
    flex-direction: column;
    gap: 1.5rem;
    max-width: 1300px;
    margin: 0 auto;
    width: 100%;
}

.doc-hero {
    padding: 2.2rem;
    border-radius: var(--radius-lg);
    background: linear-gradient(135deg, rgba(13, 18, 29, 0.85) 0%, rgba(20, 28, 45, 0.7) 100%);
    border: var(--glass-border);
    position: relative;
    overflow: hidden;
}

.doc-hero::before {
    content: '';
    position: absolute;
    top: 0;
    right: 0;
    width: 320px;
    height: 100%;
    background: radial-gradient(circle at center, rgba(0, 240, 255, 0.08) 0%, transparent 70%);
    pointer-events: none;
}

.hero-badge {
    display: inline-block;
    font-size: 0.72rem;
    font-weight: 800;
    letter-spacing: 0.1em;
    text-transform: uppercase;
    color: var(--cyan);
    margin-bottom: 0.6rem;
}

.doc-hero h1 {
    font-size: 2.2rem;
    font-weight: 800;
    color: #ffffff;
    margin-bottom: 0.8rem;
    letter-spacing: -0.02em;
}

.hero-lead {
    font-size: 1.05rem;
    color: var(--text-secondary);
    line-height: 1.7;
    max-width: 950px;
}

.doc-grid {
    display: grid;
    grid-template-columns: repeat(2, 1fr);
```

```
    gap: 1.5rem;
}

@media (max-width: 1024px) {
    .doc-grid {
        grid-template-columns: 1fr;
    }
}

.doc-card {
    padding: 1.8rem;
    border-radius: var(--radius-md);
    display: flex;
    flex-direction: column;
    gap: 1rem;
}

.doc-card-header {
    display: flex;
    align-items: center;
    gap: 0.75rem;
    border-bottom: 1px solid var(--border-subtle);
    padding-bottom: 0.8rem;
}

.doc-icon {
    font-size: 1.5rem;
}

.doc-card-header h2 {
    font-size: 1.2rem;
    font-weight: 700;
    color: #ffffff;
}

/* 4-Block Timeline */
.four-block-timeline {
    display: flex;
    flex-direction: column;
    gap: 0.6rem;
    margin: 0.5rem 0;
}

.timeline-step {
    display: flex;
    align-items: flex-start;
    gap: 0.8rem;
    background: rgba(255, 255, 255, 0.02);
    padding: 0.6rem 0.8rem;
    border-radius: var(--radius-sm);
    border-left: 3px solid var(--cyan);
}

.step-num {
    background: var(--cyan);
    color: #000000;
    font-weight: 800;
    font-size: 0.75rem;
    width: 20px;
    height: 20px;
    border-radius: 50%;
    display: flex;
    align-items: center;
    justify-content: center;
    flex-shrink: 0;
}

.step-content {
    font-size: 0.88rem;
    color: var(--text-primary);
}

.code-snippet-box {
    background: #05070b;
    border: 1px solid var(--border-subtle);
    border-radius: var(--radius-sm);
    padding: 1rem;
    font-family: var(--font-mono);
```

```
    font-size: 0.82rem;
    overflow-x: auto;
    color: var(--cyan);
}

/* Spec Tables */
.table-responsive {
    overflow-x: auto;
    width: 100%;
}

.spec-table {
    width: 100%;
    border-collapse: collapse;
    font-size: 0.85rem;
}

.spec-table th {
    background: rgba(255, 255, 255, 0.04);
    padding: 0.7rem 0.85rem;
    color: var(--text-muted);
    font-size: 0.72rem;
    text-transform: uppercase;
    letter-spacing: 0.05em;
    text-align: left;
    border-bottom: 1px solid var(--border-subtle);
}

.spec-table td {
    padding: 0.7rem 0.85rem;
    border-bottom: 1px solid rgba(255, 255, 255, 0.03);
    color: #cbd5e1;
}

.spec-table tr:hover td {
    background: rgba(0, 240, 255, 0.02);
}

/* Boundary Badges */
.badge-rigid { color: #58a6ff; font-weight: 700; }
.badge-fluid { color: #38bdf8; font-weight: 700; }
.badge-omni { color: #d29922; font-weight: 700; }
.badge-sink { color: #f85149; font-weight: 700; }
.badge-source { color: #3fb950; font-weight: 700; }
.badge-aside { color: #bc8cff; font-weight: 700; }

/* Quad-State Showcase */
.quad-state-showcase {
    display: grid;
    grid-template-columns: repeat(2, 1fr);
    gap: 0.75rem;
    margin: 0.5rem 0;
}

.quad-item {
    padding: 0.8rem;
    border-radius: var(--radius-sm);
    display: flex;
    flex-direction: column;
    gap: 0.3rem;
    border: 1px solid rgba(255, 255, 255, 0.05);
}

.quad-item.quad-proceed { background: rgba(0, 230, 118, 0.06); border-color: rgba(0, 230, 118, 0.25); }
.quad-item.quad-await { background: rgba(255, 170, 0, 0.06); border-color: rgba(255, 170, 0, 0.25); }
.quad-item.quad-rollback { background: rgba(255, 61, 113, 0.06); border-color: rgba(255, 61, 113, 0.25); }
.quad-item.quad-halt { background: rgba(157, 78, 221, 0.06); border-color: rgba(157, 78, 221, 0.25); }

.quad-code {
    font-family: var(--font-mono);
    font-weight: 800;
    font-size: 0.85rem;
}
.quad-proceed .quad-code { color: var(--state-proceed); }
.quad-await .quad-code { color: var(--state-await); }
.quad-rollback .quad-code { color: var(--state-rollback); }
.quad-halt .quad-code { color: var(--state-halt); }

```

```
.quad-meaning {
    font-size: 0.8rem;
    color: var(--text-secondary);
}

/* AI Pipeline */
.pipeline-diagram {
    display: flex;
    flex-direction: column;
    align-items: center;
    gap: 0.4rem;
    margin: 0.5rem 0;
}

.pipe-node {
    width: 100%;
    background: rgba(255, 255, 255, 0.03);
    border: 1px solid var(--border-subtle);
    padding: 0.65rem 1rem;
    border-radius: var(--radius-sm);
    text-align: center;
    font-size: 0.85rem;
    font-weight: 600;
    color: var(--text-primary);
}

.pipe-node.highlight {
    background: rgba(0, 240, 255, 0.08);
    border-color: var(--border-bright);
    color: var(--cyan);
}

.pipe-arrow {
    color: var(--text-muted);
    font-size: 0.9rem;
}

.mini-laws-grid {
    display: flex;
    flex-direction: column;
    gap: 0.4rem;
    font-size: 0.85rem;
}

.mini-law {
    background: rgba(255, 255, 255, 0.02);
    padding: 0.5rem 0.8rem;
    border-radius: var(--radius-sm);
    color: var(--text-secondary);
}

.mini-law strong {
    color: #ffffff;
}

.doc-subcard {
    background: rgba(255, 255, 255, 0.02);
    border: 1px solid var(--border-subtle);
    border-radius: var(--radius-sm);
    padding: 1rem;
    margin-top: 0.5rem;
}

.doc-subcard h3 {
    font-size: 0.95rem;
    color: var(--text-primary);
    margin-bottom: 0.4rem;
}

.doc-subcard p {
    font-size: 0.85rem;
    color: var(--text-secondary);
    margin-bottom: 0.5rem;
}

.doc-subcard code {
    display: block;
    background: #05070b;
```

```
    padding: 0.5rem 0.8rem;
    border-radius: var(--radius-sm);
    font-family: var(--font-mono);
    font-size: 0.8rem;
    color: var(--cyan);
}

.spec-list {
    list-style: none;
    display: flex;
    flex-direction: column;
    gap: 0.6rem;
    font-size: 0.88rem;
    color: var(--text-secondary);
}

.spec-list li strong {
    color: #ffffff;
}

/* ------------------------------------------------------------------------
   FOOTER
   ------------------------------------------------------------------------ */
.app-footer {
    display: flex;
    align-items: center;
    justify-content: space-between;
    padding: 1.2rem 2rem;
    background: rgba(7, 9, 14, 0.9);
    border-top: var(--glass-border);
    font-size: 0.82rem;
    color: var(--text-muted);
    margin-top: 2rem;
}

.footer-left {
    display: flex;
    align-items: center;
    gap: 0.75rem;
}

.footer-brand {
    font-family: var(--font-serif);
    font-weight: 700;
    color: var(--cyan);
}

.footer-center {
    color: var(--text-secondary);
}

.footer-right {
    display: flex;
    align-items: center;
    gap: 0.5rem;
}

.footer-pill {
    background: rgba(255, 255, 255, 0.05);
    padding: 0.2rem 0.6rem;
    border-radius: 9999px;
    font-size: 0.72rem;
    color: var(--text-secondary);
}

/* ------------------------------------------------------------------------
   INTERACTIVE TERMINAL WORKSPACE & SCHEDULER STYLES
   ------------------------------------------------------------------------ */
.terminal-body {
    overflow: hidden;
}
.terminal-page-layout {
    height: 100vh;
    display: flex;
    flex-direction: column;
}
.terminal-workspace {
    flex: 1;
```

```
    display: flex;
    flex-direction: column;
    padding: 1rem 1.5rem;
    gap: 0.8rem;
    overflow: hidden;
}
.terminal-toolbar {
    display: flex;
    align-items: center;
    justify-content: space-between;
    padding: 0.6rem 1rem;
    background: rgba(13, 17, 23, 0.8);
    border: 1px solid rgba(255, 255, 255, 0.08);
    border-radius: 8px;
}
.toolbar-left {
    display: flex;
    align-items: center;
    gap: 0.8rem;
}
.terminal-title-badge {
    font-family: var(--font-mono);
    font-size: 0.78rem;
    font-weight: 700;
    color: var(--cyan);
    letter-spacing: 0.05em;
}
.sched-pill {
    background: rgba(0, 240, 255, 0.1);
    border: 1px solid rgba(0, 240, 255, 0.25);
    color: var(--cyan);
    font-size: 0.72rem;
    padding: 0.25rem 0.6rem;
    border-radius: 9999px;
    display: flex;
    align-items: center;
    gap: 0.4rem;
}
.mem-pill {
    background: rgba(0, 255, 136, 0.1);
    border: 1px solid rgba(0, 255, 136, 0.25);
    color: var(--green);
    font-size: 0.72rem;
    padding: 0.25rem 0.6rem;
    border-radius: 9999px;
}
.toolbar-actions {
    display: flex;
    gap: 0.4rem;
}
.btn-sm {
    padding: 0.35rem 0.7rem;
    font-size: 0.75rem;
}
.btn-xs {
    padding: 0.2rem 0.45rem;
    font-size: 0.68rem;
}
.btn-danger-ghost {
    background: transparent;
    color: #ff5555;
    border: 1px solid rgba(255, 85, 85, 0.25);
}
.btn-danger-ghost:hover {
    background: rgba(255, 85, 85, 0.15);
}

.terminal-grid {
    flex: 1;
    display: grid;
    grid-template-columns: 1fr 380px;
    gap: 1rem;
    min-height: 0;
}
.terminal-console-pane {
    display: flex;
    flex-direction: column;
    background: rgba(10, 12, 16, 0.95);
```

```
    border: 1px solid rgba(0, 240, 255, 0.2);
    border-radius: 8px;
    box-shadow: 0 8px 32px rgba(0, 0, 0, 0.5);
    min-height: 0;
    overflow: hidden;
}
.console-screen {
    flex: 1;
    padding: 1.2rem;
    overflow-y: auto;
    font-family: var(--font-mono);
    font-size: 0.86rem;
    line-height: 1.5;
    color: #e6edf3;
    outline: none;
}
.term-intro {
    color: var(--cyan);
    white-space: pre-wrap;
    margin-bottom: 1rem;
    font-size: 0.78rem;
    opacity: 0.9;
}
.term-entry {
    margin-bottom: 0.8rem;
}
.term-command-line {
    display: flex;
    align-items: center;
    gap: 0.5rem;
    color: #ffffff;
    font-weight: 600;
}
.term-prompt {
    color: var(--gold);
    font-family: var(--font-mono);
    font-weight: 700;
}
.term-cmd-text {
    color: #00f0ff;
}
.term-result-box {
    margin-top: 0.3rem;
    padding-left: 0.8rem;
    border-left: 2px solid rgba(0, 240, 255, 0.3);
}
.term-meta-bar {
    display: flex;
    align-items: center;
    gap: 0.6rem;
    margin-bottom: 0.25rem;
    font-size: 0.72rem;
}
.term-output-text {
    font-family: var(--font-mono);
    font-size: 0.82rem;
    white-space: pre-wrap;
    word-break: break-word;
    color: #d1d5db;
    margin: 0;
}
.console-input-row {
    display: flex;
    align-items: center;
    gap: 0.6rem;
    padding: 0.8rem 1.2rem;
    background: rgba(13, 17, 23, 0.95);
    border-top: 1px solid rgba(255, 255, 255, 0.08);
}
.term-input {
    flex: 1;
    background: transparent;
    border: none;
    outline: none;
    color: #ffffff;
    font-family: var(--font-mono);
    font-size: 0.9rem;
}
```

```
1479
1480 /* Scheduler Right Pane */
1481 .scheduler-pane {
1482     display: flex;
1483     flex-direction: column;
1484     background: rgba(13, 17, 23, 0.85);
1485     border: 1px solid rgba(255, 255, 255, 0.08);
1486     border-radius: 8px;
1487     padding: 1rem;
1488     gap: 0.8rem;
1489     min-height: 0;
1490     overflow: hidden;
1491 }
1492 .scheduler-pane-header {
1493     display: flex;
1494     justify-content: space-between;
1495     align-items: center;
1496 }
1497 .sched-title-group h2 {
1498     font-size: 0.95rem;
1499     margin: 0;
1500     color: #ffffff;
1501 }
1502 .sched-title-group .subtext {
1503     font-size: 0.72rem;
1504     color: var(--text-muted);
1505     margin: 0;
1506 }
1507 .sched-spawn-box {
1508     display: grid;
1509     grid-template-columns: 1fr 1fr;
1510     gap: 0.4rem;
1511 }
1512 .sched-spawn-box button {
1513     grid-column: span 2;
1514 }
1515 .form-input-sm, .form-select-sm {
1516     background: rgba(255, 255, 255, 0.05);
1517     border: 1px solid rgba(255, 255, 255, 0.1);
1518     color: #ffffff;
1519     padding: 0.35rem 0.5rem;
1520     font-size: 0.75rem;
1521     border-radius: 4px;
1522     outline: none;
1523 }
1524 .task-table-wrapper {
1525     flex: 1;
1526     overflow-y: auto;
1527     border: 1px solid rgba(255, 255, 255, 0.05);
1528     border-radius: 4px;
1529 }
1530 .task-table {
1531     width: 100%;
1532     border-collapse: collapse;
1533     font-size: 0.75rem;
1534 }
1535 .task-table th, .task-table td {
1536     padding: 0.45rem 0.6rem;
1537     text-align: left;
1538     border-bottom: 1px solid rgba(255, 255, 255, 0.05);
1539 }
1540 .task-table th {
1541     background: rgba(255, 255, 255, 0.03);
1542     color: var(--text-muted);
1543     font-weight: 600;
1544 }
1545 .action-cell {
1546     display: flex;
1547     gap: 0.2rem;
1548 }
1549 .session-vars-box {
1550     background: rgba(0, 0, 0, 0.3);
1551     border: 1px solid rgba(255, 255, 255, 0.05);
1552     border-radius: 4px;
1553     padding: 0.6rem;
1554     max-height: 120px;
1555     overflow-y: auto;
1556 }
```

```
.vars-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 0.3rem;
}
.vars-header h3 {
    font-size: 0.75rem;
    margin: 0;
    color: var(--gold);
}
.vars-content {
    font-family: var(--font-mono);
    font-size: 0.72rem;
    color: #e6edf3;
}

/* Download Page Styles */
.download-container {
    max-width: 1100px;
    margin: 2rem auto;
    padding: 0 1.5rem;
}
.download-hero {
    text-align: center;
    padding: 3rem 2rem;
    margin-bottom: 2rem;
}
.download-btn-row {
    margin: 2rem 0 1rem;
}
.download-cta {
    background: linear-gradient(135deg, #00f0ff, #0099ff);
    color: #000000;
    font-weight: 800;
    font-size: 1.05rem;
    padding: 0.9rem 2.2rem;
    border-radius: 8px;
    box-shadow: 0 0 25px rgba(0, 240, 255, 0.35);
    display: inline-flex;
    align-items: center;
    gap: 0.8rem;
    text-decoration: none;
    transition: transform 0.2s, box-shadow 0.2s;
}
.download-cta:hover {
    transform: translateY(-2px);
    box-shadow: 0 0 35px rgba(0, 240, 255, 0.5);
}
.hero-subtext {
    font-size: 0.82rem;
    color: var(--text-muted);
}
.download-grid {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 1.5rem;
}
.download-card {
    padding: 1.8rem;
}
.code-copy-box {
    background: rgba(0, 0, 0, 0.5);
    border: 1px solid rgba(0, 240, 255, 0.2);
    border-radius: 6px;
    padding: 0.9rem;
    margin: 1rem 0;
    font-family: var(--font-mono);
    font-size: 0.82rem;
    color: var(--cyan);
    word-break: break-all;
}
.download-nav-btn {
    color: var(--cyan) !important;
    border-color: rgba(0, 240, 255, 0.3) !important;
}

/* ------------------------------------------------------------------------
```

```
   OMNICODE 3-TIER COMPOSITE HOVERCARD & DOCSTRING INSPECTOR
   ------------------------------------------------------------------------- */
.omni-hover-card {
    position: absolute;
    z-index: 10000;
    max-width: 480px;
    background: rgba(10, 15, 29, 0.94);
    backdrop-filter: blur(20px);
    -webkit-backdrop-filter: blur(20px);
    border: 1px solid rgba(0, 240, 255, 0.35);
    border-radius: 10px;
    box-shadow: 0 12px 40px rgba(0, 0, 0, 0.8), 0 0 20px rgba(0, 240, 255, 0.15);
    padding: 1.1rem;
    color: #e2e8f0;
    font-family: var(--font-sans, -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif);
    font-size: 0.85rem;
    line-height: 1.5;
    pointer-events: auto;
    transition: opacity 0.15s ease, transform 0.15s ease;
}

.omni-hover-header {
    display: flex;
    align-items: center;
    justify-content: space-between;
    border-bottom: 1px solid rgba(255, 255, 255, 0.1);
    padding-bottom: 0.6rem;
    margin-bottom: 0.8rem;
}

.omni-hover-symbol {
    font-family: var(--font-mono, monospace);
    font-size: 1.05rem;
    font-weight: 700;
    color: #00f0ff;
    letter-spacing: 0.5px;
}

.omni-hover-rank-badge {
    display: inline-block;
    padding: 0.2rem 0.6rem;
    font-size: 0.72rem;
    font-weight: 600;
    border-radius: 4px;
    background: rgba(0, 240, 255, 0.15);
    color: #38bdf8;
    border: 1px solid rgba(56, 189, 248, 0.3);
    text-transform: uppercase;
}

.omni-hover-rank-badge.rank-1 {
    background: rgba(234, 179, 8, 0.18);
    color: #facc15;
    border-color: rgba(250, 204, 21, 0.4);
}

.omni-hover-tier {
    margin-bottom: 0.8rem;
    padding-bottom: 0.7rem;
    border-bottom: 1px solid rgba(255, 255, 255, 0.06);
}

.omni-hover-tier:last-child {
    margin-bottom: 0;
    padding-bottom: 0;
    border-bottom: none;
}

.omni-hover-tier-title {
    font-size: 0.75rem;
    font-weight: 700;
    text-transform: uppercase;
    letter-spacing: 0.8px;
    color: #94a3b8;
    margin-bottom: 0.4rem;
    display: flex;
    align-items: center;
    gap: 0.4rem;
```

```
}

.omni-hover-tier-title.tier-tape { color: #fbbf24; }
.omni-hover-tier-title.tier-omni { color: #34d399; }
.omni-hover-tier-title.tier-silicon { color: #c084fc; }

.omni-hover-prop {
    display: flex;
    margin-bottom: 0.25rem;
    font-size: 0.8rem;
}

.omni-hover-label {
    color: #64748b;
    width: 130px;
    flex-shrink: 0;
}

.omni-hover-val {
    color: #f1f5f9;
    font-family: var(--font-mono, monospace);
    word-break: break-word;
}

.omni-hover-aside {
    background: rgba(255, 255, 255, 0.04);
    border-left: 2px solid #38bdf8;
    padding: 0.35rem 0.6rem;
    margin: 0.4rem 0;
    font-style: italic;
    color: #cbd5e1;
    font-size: 0.78rem;
}

```

<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/gui/terminal.html

HTML | 191 LINES | 782 WORDS | 10.8 KB

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="OmniCode Interactive Native Terminal & OS Process Scheduler — CreativeWorkzStudio LLC.">
    <title>OmniCode Interactive Terminal &amp;amp; Task Scheduler — CreativeWorkzStudio LLC</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Fira+Code:wght@400;500;600;700&family=Outfit:wght@300;400;500;600;700;800&family=Cinzel:wght@500;700;800&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="/style.css">
</head>
<body class="terminal-body">
    <div class="app-layout terminal-page-layout">
        <!-- GLOBAL NAVIGATION HEADER -->
        <header class="app-header" id="app-header">
            <div class="header-left">
                <a href="/studio" class="brand-container">
                    <span class="brand-symbol">Ω</span>
                    <div class="brand-text">
                        <span class="brand-title">OMNICODE</span>
                        <span class="brand-subtitle">CREATIVEWORKZSTUDIO LLC</span>
                    </div>
                </a>
                <div class="status-pill" id="status-pill" title="Engine runs 100% offline with zero cloud telemetry">
                    <span class="status-dot"></span>
                    <span class="status-text">OFFLINE COVENANT ACTIVE</span>
                </div>
            </div>

            <nav class="app-nav" aria-label="Main Navigation">
                <a href="/studio" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M12 2L2 7l10 5 10-5-10-5zM2 17l10 5 10-5M2 12l10 5 10-5"/></svg>
                    AI Studio
                </a>
                <a href="/terminal" class="nav-tab active">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><polyline points="4 17 10 11 4 5"/><line x1="12" y1="19" x2="20" y2="19"/></svg>
                    CLI Terminal
                </a>
                <a href="/specs" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8z"/><polyline points="14 2 14 8 20 8"/></svg>
                    Language Specs
                </a>
                <a href="/model" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M12 2a8 8 0 0 0-8 8c0 3.3 2 6.2 5 7.4V20a2 2 0 0 0 2 2h2a2 2 0 0 0 2-2v-2.6c3-1.2 5-4.1 5-7.4a8 8 0 0 0-8-8z"/></svg>
                    AI Model
                </a>
                <a href="/lexicon" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20"/><path d="M6.5 2H20v20H6.5A2.5 2.5 0 0 1 4 19.5v-15A2.5 2.5 0 0 1 6.5 2z"/></svg>
                    Covenant Lexicon
                </a>
                <a href="/scrolls" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M19 21l-7-5-7 5V5a2 2 0 0 1 2-2h10a2 2 0 0 1 2 2z"/></svg>
                    Scrolls
                </a>
                <a href="/cloud" class="nav-tab">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M18 10h-1.26A8 8 0 1 0 9 20h9a5 5 0 0 0 0-10z"/></svg>
                    Cloud &amp;amp; Edge
                </a>
                <a href="/download" class="nav-tab download-nav-btn">
                    <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4"/><polyline points="7 10 12 15 17 10"/><line x1="12" y1="15" x2="12" y2="3"/></svg>
```

```
                    Download Native
                </a>
            </nav>
        </header>

        <!-- TERMINAL WORKSPACE MAIN -->
        <main class="terminal-workspace">
            <!-- TERMINAL CONTROL TOOLBAR -->
            <div class="terminal-toolbar card-glass">
                <div class="toolbar-left">
                    <span class="terminal-title-badge">Ω NATIVE USERSPACE TERMINAL</span>
                    <span class="sched-pill" id="sched-status-pill">
                        <span class="status-dot"></span>
                        <span id="sched-pill-text">Scheduler: Online (3 tasks)</span>
                    </span>
                    <span class="mem-pill" id="mem-pill">State: +1:PROCEED</span>
                </div>
                <div class="toolbar-actions">
                    <button class="btn btn-sm btn-ghost" id="btn-quick-demo">🧪 Demo</button>
                    <button class="btn btn-sm btn-ghost" id="btn-quick-lint">📋 Lint</button>
                    <button class="btn btn-sm btn-ghost" id="btn-quick-restore">🩹 Restore</button>
                    <button class="btn btn-sm btn-ghost" id="btn-quick-tasks">📊 Tasks (ps)</button>
                    <button class="btn btn-sm btn-ghost" id="btn-quick-tick">⚡ Tick</button>
                    <button class="btn btn-sm btn-danger-ghost" id="btn-clear-term">🧹 Clear (Ctrl+L)</button>
                </div>
            </div>

            <!-- DUAL VIEWPORT: TERMINAL SCREEN & SCHEDULER MONITOR -->
            <div class="terminal-grid">
                <!-- LEFT: LIVE INTERACTIVE CONSOLE -->
                <div class="terminal-console-pane card-glass">
                    <div class="console-screen" id="console-screen" tabindex="0">
                        <div class="term-intro">
 ================================================================================
   Ω OMNICODE INTERACTIVE USERSYSTEM SHELL &amp;amp; TASK SCHEDULER
   CreativeWorkzStudio LLC — Autonomous Neural-Symbolic Intelligence
   Engine: Rust Core 2024 (Balanced Radix-3 Quad-State Physics)
 ================================================================================
 * Grounded in Biblical Moral Invariants &amp;amp; Symmetrical Covenant Bedrock
 * Type 'help' for terminal commands, 'ps' for scheduler tasks, 'tick' to advance clock.
 * Keyboard: Up/Down for command history | Tab for autocompletion | Ctrl+L to clear screen.
                        </div>
                        <div id="term-history-log"></div>
                    </div>

                    <!-- INTERACTIVE INPUT BAR -->
                    <div class="console-input-row" id="input-container">
                        <span class="term-prompt">omnicode&amp;gt;</span>
                        <input type="text" id="term-input" class="term-input" autocomplete="off" autofocus
spellcheck="false" placeholder="Type command (e.g. ps, spawn 'worker', let x = 42, omnicode demo, help)...">
                        <button class="btn btn-sm btn-accent" id="btn-term-send">Execute</button>
                    </div>
                </div>

                <!-- RIGHT: LIVE OS TASK SCHEDULER & PROCESS LEDGER -->
                <div class="scheduler-pane card-glass">
                    <div class="scheduler-pane-header">
                        <div class="sched-title-group">
                            <h2>Cooperative Task Scheduler</h2>
                            <p class="subtext">Active process table grounded in Balanced Radix-3 states</p>
                        </div>
                        <button class="btn btn-sm btn-secondary" id="btn-refresh-tasks">Refresh</button>
                    </div>

                    <!-- TASK SPAWN ROW -->
                    <div class="sched-spawn-box">
                        <input type="text" id="spawn-name-input" class="form-input-sm" placeholder="Task Name (e.g.
InvariantWatcher)">
                        <input type="text" id="spawn-cmd-input" class="form-input-sm" placeholder="Command (e.g. omnicode
lint manifest.omni)">
                        <select id="spawn-priority-select" class="form-select-sm">
                            <option value="NORMAL">Normal Priority</option>
                            <option value="HIGH">High Priority</option>
                            <option value="LOW">Low Priority</option>
                            <option value="BACKGROUND">Background</option>
                        </select>
                        <button class="btn btn-sm btn-accent" id="btn-spawn-task">+ Spawn</button>
                    </div>
```

```
                    <!-- TASK TABLE -->
                    <div class="task-table-wrapper">
                        <table class="task-table">
                            <thead>
                                <tr>
                                    <th>PID</th>
                                    <th>Task Name</th>
                                    <th>Priority</th>
                                    <th>Quad-State</th>
                                    <th>Ticks</th>
                                    <th>Actions</th>
                                </tr>
                            </thead>
                            <tbody id="task-tbody">
                                <!-- Populated dynamically via JS -->
                            </tbody>
                        </table>
                    </div>

                    <!-- IN-MEMORY SESSION VARIABLES -->
                    <div class="session-vars-box">
                        <div class="vars-header">
                            <h3>In-Memory Session Variables</h3>
                            <button class="btn btn-xs btn-ghost" id="btn-refresh-vars">Refresh Vars</button>
                        </div>
                        <div class="vars-content" id="session-vars-content">
                            <span class="dim-text">No session variables declared yet. Use 'let &lt;name&gt; =
&lt;value&gt;'</span>
                        </div>
                    </div>
                </div>
            </div>
        </main>

        <!-- FOOTER -->
        <footer class="app-footer">
            <div class="footer-left">
                <span class="footer-brand">Ω OMNICODE</span>
                <span class="footer-sep">&bull;</span>
                <span class="footer-copy">&copy; 2026 CreativeWorkzStudio LLC. All rights reserved.</span>
            </div>
            <div class="footer-center">
                <span>Native Userspace Shell &amp; Cooperative Quad-State Process Scheduler</span>
            </div>
            <div class="footer-right">
                <span class="footer-pill">Local Ring-3 Shell</span>
                <span class="footer-pill">v0.1.0</span>
            </div>
        </footer>
    </div>

    <script src="/terminal.js"></script>
</body>
</html>
```

<TRACK 3> : {Applications & OmniStudio Interfaces}/apps/gui/terminal.js

JS | 305 LINES | 885 WORDS | 11.9 KB

```
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( OmniCode Native Interactive Terminal & Task Scheduler JavaScript Engine )
// ================================================================================
/**
 * OmniCode Terminal & Scheduler Frontend Engine
 * CreativeWorkzStudio LLC
 * Zero external dependencies. Connects via standard HTTP JSON endpoints:
 *   - POST /api/terminal          -> Dispatches command to session & scheduler
 *   - GET  /api/scheduler         -> Retrieves live process ledger
 *   - POST /api/scheduler/spawn   -> Spawns new task
 *   - POST /api/scheduler/action  -> Mutates task state (pause, resume, kill)
 */

document.addEventListener("DOMContentLoaded", () => {
    const consoleScreen = document.getElementById("console-screen");
    const historyLog = document.getElementById("term-history-log");
    const termInput = document.getElementById("term-input");
    const btnSend = document.getElementById("btn-term-send");
    const btnClear = document.getElementById("btn-clear-term");
    const taskTbody = document.getElementById("task-tbody");
    const schedPillText = document.getElementById("sched-pill-text");
    const varsContent = document.getElementById("session-vars-content");

    // Command History & Autocomplete State
    const history = JSON.parse(localStorage.getItem("omnicode_terminal_history") || "[]");
    let historyIndex = history.length;

    const AUTOCOMPLETE_LIST = [
        "ps", "tasks", "spawn", "pause", "resume", "kill", "halt", "tick", "vars", "env",
        "let", "help", "clear",
        "omnicode demo",
        "omnicode lint manifest.omni",
        "omnicode restore manifest.omni",
        "omnicode manifest",
        "omnicode scrolls",
        "omnicode lexicon truth",
        "omnicode ai \"What is the covenant?\""
    ];

    // Focus input on console click
    consoleScreen.addEventListener("click", () => {
        termInput.focus();
    });

    // --------------------------------------------------------------------------
    // EXECUTION DISPATCHER
    // --------------------------------------------------------------------------
    async function executeCommand(cmd) {
        const clean = cmd.trim();
        if (!clean) return;

        // Save to history
        if (history.length === 0 || history[history.length - 1] !== clean) {
            history.push(clean);
            if (history.length > 100) history.shift();
            localStorage.setItem("omnicode_terminal_history", JSON.stringify(history));
        }
        historyIndex = history.length;

        // Handle local 'clear'
        if (clean.toLowerCase() === "clear") {
            historyLog.innerHTML = "";
            termInput.value = "";
            return;
        }

        // Render command prompt entry
        const entryEl = document.createElement("div");
        entryEl.className = "term-entry";
        entryEl.innerHTML = `
            <div class="term-command-line">
                <span class="term-prompt">omnicode&gt;</span>
                <span class="term-cmd-text">${escapeHtml(clean)}</span>
```

```
            </div>
            <div class="term-loading-spinner dim-text">Deliberating &amp;amp; evaluating...</div>
        `;
        historyLog.appendChild(entryEl);
        consoleScreen.scrollTop = consoleScreen.scrollHeight;
        termInput.value = "";

        const spinner = entryEl.querySelector(".term-loading-spinner");

        try {
            const resp = await fetch("/api/terminal", {
                method: "POST",
                headers: { "Content-Type": "application/json" },
                body: JSON.stringify({ command: clean })
            });

            const data = await resp.json();
            spinner.remove();

            const resultEl = document.createElement("div");
            resultEl.className = "term-result-box";

            const stateClass = getStateClass(data.state || "+1:PROCEED");
            const durationMs = (data.duration_us / 1000).toFixed(2);

            resultEl.innerHTML = `
                <div class="term-meta-bar">
                    <span class="badge ${stateClass}">${escapeHtml(data.state || "+1:PROCEED")}</span>
                    <span class="dim-text">${durationMs} ms</span>
                </div>
                <pre class="term-output-text">${formatTerminalOutput(data.stdout || "(Completed with zero output)")}
</pre>
            `;
            entryEl.appendChild(resultEl);
        } catch (err) {
            spinner.remove();
            const errEl = document.createElement("div");
            errEl.className = "term-error-box";
            errEl.innerHTML = `<pre class="text-red">✗ Error dispatching command: ${escapeHtml(err.message)}</pre>`;
            entryEl.appendChild(errEl);
        }

        consoleScreen.scrollTop = consoleScreen.scrollHeight;
        refreshSchedulerTasks();
        refreshVariables();
    }

    // ---------------------------------------------------------------------------
    // SCHEDULER TASK POLLING & ACTIONS
    // ---------------------------------------------------------------------------
    async function refreshSchedulerTasks() {
        try {
            const resp = await fetch("/api/scheduler");
            if (!resp.ok) return;
            const tasks = await resp.json();

            taskTbody.innerHTML = "";
            let activeCount = 0;

            tasks.forEach(t => {
                if (t.state === "+1:PROCEED") activeCount++;
                const tr = document.createElement("tr");
                const stateClass = getStateClass(t.state);

                tr.innerHTML = `
                    <td class="mono font-bold">${t.id}</td>
                    <td title="${escapeHtml(t.command)}"><strong>${escapeHtml(t.name)}</strong></td>
                    <td><span class="badge badge-subtle">${escapeHtml(t.priority)}</span></td>
                    <td><span class="badge ${stateClass}">${escapeHtml(t.state)}</span></td>
                    <td class="mono">${t.ticks}</td>
                    <td class="action-cell">
                        ${t.state === "+1:PROCEED"
                            ? `<button class="btn btn-xs btn-ghost btn-pause" data-id="${t.id}" title="Pause
(0:AWAIT)">⏸</button>`
                            : `<button class="btn btn-xs btn-ghost btn-resume" data-id="${t.id}" title="Resume
(+1:PROCEED)">▶</button>`}
                        ${t.state !== "NULL:HALT"
                            ? `<button class="btn btn-xs btn-danger-ghost btn-halt" data-id="${t.id}" title="Terminate
```

```
(NULL:HALT)">■</button>`
                            : `<span class="dim-text">Ended</span>`}
                    </td>
                `;
                taskTbody.appendChild(tr);
            });

            schedPillText.textContent = `Scheduler: Online (${tasks.length} tasks, ${activeCount} active)`;

            // Attach action listeners
            document.querySelectorAll(".btn-pause").forEach(b => {
                b.onclick = () => mutateTask(b.dataset.id, "pause");
            });
            document.querySelectorAll(".btn-resume").forEach(b => {
                b.onclick = () => mutateTask(b.dataset.id, "resume");
            });
            document.querySelectorAll(".btn-halt").forEach(b => {
                b.onclick = () => mutateTask(b.dataset.id, "halt");
            });
        } catch (e) {
            console.error("Scheduler fetch error:", e);
        }
    }

    async function mutateTask(id, action) {
        await executeCommand(`${action} ${id}`);
    }

    async function refreshVariables() {
        try {
            const resp = await fetch("/api/terminal", {
                method: "POST",
                headers: { "Content-Type": "application/json" },
                body: JSON.stringify({ command: "vars" })
            });
            const data = await resp.json();
            varsContent.innerHTML = `<pre class="mono-code">${escapeHtml(data.stdout || "No session variables")}</pre>`;
        } catch (e) {
            console.error(e);
        }
    }

    // ---------------------------------------------------------------------------
    // KEYBOARD NAVIGATION & SHORTCUTS
    // ---------------------------------------------------------------------------
    termInput.addEventListener("keydown", (e) => {
        // Enter to execute
        if (e.key === "Enter") {
            e.preventDefault();
            executeCommand(termInput.value);
            return;
        }

        // Up arrow (history backward)
        if (e.key === "ArrowUp") {
            e.preventDefault();
            if (historyIndex > 0) {
                historyIndex--;
                termInput.value = history[historyIndex] || "";
            }
            return;
        }

        // Down arrow (history forward)
        if (e.key === "ArrowDown") {
            e.preventDefault();
            if (historyIndex < history.length - 1) {
                historyIndex++;
                termInput.value = history[historyIndex] || "";
            } else {
                historyIndex = history.length;
                termInput.value = "";
            }
            return;
        }

        // Tab autocomplete
        if (e.key === "Tab") {
```

```
            e.preventDefault();
            const val = termInput.value.trim().toLowerCase();
            if (!val) return;
            const match = AUTOCOMPLETE_LIST.find(item => item.toLowerCase().startsWith(val));
            if (match) {
                termInput.value = match;
            }
            return;
        }

        // Ctrl + L (Clear)
        if (e.ctrlKey && e.key.toLowerCase() === "l") {
            e.preventDefault();
            historyLog.innerHTML = "";
            return;
        }
    });

    btnSend.addEventListener("click", () => {
        executeCommand(termInput.value);
    });

    btnClear.addEventListener("click", () => {
        historyLog.innerHTML = "";
        termInput.focus();
    });

    // Toolbar Quick Action Buttons
    document.getElementById("btn-quick-demo")?.addEventListener("click", () => executeCommand("omnicode demo"));
    document.getElementById("btn-quick-lint")?.addEventListener("click", () => executeCommand("omnicode lint
manifest.omni"));
    document.getElementById("btn-quick-restore")?.addEventListener("click", () => executeCommand("omnicode restore
manifest.omni"));
    document.getElementById("btn-quick-tasks")?.addEventListener("click", () => executeCommand("ps"));
    document.getElementById("btn-quick-tick")?.addEventListener("click", () => executeCommand("tick"));
    document.getElementById("btn-refresh-tasks")?.addEventListener("click", refreshSchedulerTasks);
    document.getElementById("btn-refresh-vars")?.addEventListener("click", refreshVariables);

    // Spawn Task Button
    document.getElementById("btn-spawn-task")?.addEventListener("click", () => {
        const nameInput = document.getElementById("spawn-name-input");
        const cmdInput = document.getElementById("spawn-cmd-input");
        const prioSelect = document.getElementById("spawn-priority-select");

        const cmd = cmdInput.value.trim();
        if (!cmd) return;

        executeCommand(`spawn ${cmd}`);
        cmdInput.value = "";
        nameInput.value = "";
    });

    // Helper functions
    function getStateClass(state) {
        if (!state) return "badge-proceed";
        if (state.includes("PROCEED")) return "badge-proceed";
        if (state.includes("AWAIT")) return "badge-await";
        if (state.includes("ROLLBACK")) return "badge-rollback";
        if (state.includes("HALT")) return "badge-halt";
        return "badge-proceed";
    }

    function escapeHtml(str) {
        return (str || "")
            .replace(/&/g, "&amp;")
            .replace(/</g, "&lt;")
            .replace(/>/g, "&gt;")
            .replace(/"/g, "&quot;")
            .replace(/'/g, "&#039;");
    }

    function formatTerminalOutput(text) {
        return escapeHtml(text);
    }

    // Initial scheduler poll & timer
    refreshSchedulerTasks();
    refreshVariables();
```

```
    setInterval(refreshSchedulerTasks, 4000);
});
```

<TRACK 5> : {Infrastructure, Operations & Archives}/TRACK.md MD 12 LINES 57 WORDS 0.5 KB

```
# <TRACK 5> : {Infrastructure, Operations & Archives}

## Scope & Invariant

Operations, containerization, DNS synchronization, packaging, and daemon management.

- `infra/dist/`: Standalone distribution packages (`.tar.gz`).
- `infra/dns/`: Dynamic DNS synchronization scripts & zone records.
- `infra/docker/`: Multi-architecture containerization and composition topologies.
- `infra/scripts/`: Toolchain packaging, cloud synchronization, and deployment automation.
- `infra/systemd/`: Production daemon service definitions.
```

<TRACK 5> : {Infrastructure, Operations & Archives}/infra/systemd/omnicode-web.service

SERVICE | 22 LINES | 34 WORDS | 0.5 KB

```
[Unit]
Description=OmniCode Cognitive Studio Web & REST API Server (CreativeWorkzStudio LLC)
After=network.target

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/home/ubuntu/OmniCode
ExecStart=/home/ubuntu/OmniCode/target/release/omnicode gui --port 8080
Restart=always
RestartSec=5s
Environment=RUST_LOG=info
Environment=OMNICODE_OFFLINE=1
Environment=OMNICODE_HOST=0.0.0.0

# Security and resource sandboxing
LimitNOFILE=65536
NoNewPrivileges=true

[Install]
WantedBy=multi-user.target
```

<TRACK 5> : {Infrastructure, Operations & Archives}/infra/docker/Dockerfile

TEXT | 178 LINES | 947 WORDS | 7.8 KB

```
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( OmniCode Bare-Metal OS & Multi-Architecture Toolchain Specification )
# ================================================================================
# Image:       omnicode-toolchain:latest
# Base OS:     Debian 12 (Bookworm) via official Rust 1.85 slim image
# Maintainer:  ProfessorSeanEX
# Architecture: Multi-target (x86_64, aarch64, riscv64gc)
#
# PURPOSE & COGNITIVE RATIONALE:
#   This container establishes a completely reproducible, hermetic compilation,
#   emulation, and cognitive execution environment for the OmniCode language.
#   It serves two distinct computational tiers:
#     1. Low-Level Bare-Metal Systems: no_std cross-compilation, bootloader ISO
#        packaging, NASM assembly, and multi-architecture QEMU hardware emulation.
#     2. High-Level Cognitive & Language Runtime: The Rust 2024 core engine,
#        interactive REPL, AI Cortex, Episodic Scroll logger, and self-healing Restorator.
#
# WHY EXTENSIVE COMMENTING MATTERS FOR AI & DEVELOPERS:
#   - For AI Agents (e.g., AICortex, Pair-Programming Assistants):
#     Code comments serve as semantic invariant anchors. When an AI parses this
#     Dockerfile, comments provide explicit causal rationales ("Why is xorriso here?
#     Because GRUB El Torito ISOs require it without loop mounts"). This prevents
#     the AI from hallucinating dependencies or accidentally stripping essential
#     cross-compilers during automated refactoring.
#   - For Human Engineers:
#     Systems programming toolchains conflate dozens of specialized CLI utilities.
#     Detailed comments eliminate guesswork, clarify port assignments (e.g., 1234
#     for QEMU GDB), and explain target triples for bare-metal silicon.
# ================================================================================

FROM rust:1.85-slim-bookworm AS toolchain-base

LABEL maintainer="ProfessorSeanEX"
LABEL description="OmniCode Bare-Metal Toolchain: Rust Core, LLVM/Clang, QEMU Emulation, and ISO Bootloaders"
LABEL version="0.1.0"

# Non-interactive frontend prevents apt from prompting for timezone/keyboard input during build
ENV DEBIAN_FRONTEND=noninteractive

# Standardize Cargo and Rustup directories into global system locations
ENV CARGO_HOME=/usr/local/cargo
ENV RUSTUP_HOME=/usr/local/rustup
ENV PATH="${CARGO_HOME}/bin:${PATH}"

# OmniCode runtime environment defaults
ENV RUST_BACKTRACE=1
ENV OMNICODE_SCROLL_DIR=/workspace/scrolls
ENV OMNICODE_LIB_DIR=/workspace/lib


# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( System Dependencies, Cross-Compilers, Assemblers & Emulation Prerequisites )
# ================================================================================
# RATIONALE FOR PACKAGE GROUPINGS:
#   1. build-essential, clang, lld, llvm:
#      - Clang/LLVM provides the primary C ABI transpilation and LLVM IR compilation backend.
#      - LLD is LLVM's ultra-fast linker, indispensable for bare-metal custom memory linker scripts.
#   2. nasm, yasm:
#      - Assemblers for 16-bit real mode, 32-bit protected mode bootloaders, and 64-bit entry stubs.
#   3. qemu-system-x86, qemu-system-arm, qemu-system-misc:
#      - Hardware emulators covering all three tier-1 OmniCode targets:
#        * qemu-system-x86: Intel/AMD x86_64 PC hardware (Q35/i440FX chipsets).
#        * qemu-system-arm: ARMv8-A Cortex bare-metal boards (virt, raspi).
#        * qemu-system-misc: RISC-V 64-bit hardware (virt board, RV64GC).
#   4. xorriso, grub-pc-bin, grub-common, mtools:
#      - Bootloader ISO image authoring. Allows generating bootable hybrid El Torito ISOs
#        and Multiboot2/UEFI media without needing root loop-device privileges.
#   5. gdb-multiarch:
#      - Cross-architecture debugger capable of stepping through x86_64, aarch64, and
#        riscv64 instructions when attached to QEMU's GDB stub on port 1234.
#   6. ripgrep, jq, file, tree, less:
```

```
#       - Diagnostic inspection tools. `file` inspects ELF kernel binaries; `jq` and `ripgrep`
#         parse JSON AST dumps and episodic trace telemetry.
#   7. nodejs, npm:
#       - Required for building, linting, and packaging the OmniCode VS Code extension
#         via `@vscode/vsce` in editors/vscode/.
# ================================================================================

RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    clang \
    lld \
    llvm \
    nasm \
    yasm \
    qemu-system-x86 \
    qemu-system-arm \
    qemu-system-misc \
    xorriso \
    grub-pc-bin \
    grub-common \
    mtools \
    gdb-multiarch \
    curl \
    git \
    pkg-config \
    ca-certificates \
    ripgrep \
    jq \
    file \
    tree \
    less \
    nodejs \
    npm \
    && rm -rf /var/lib/apt/lists/*


# ================================================================================
# BLOCK 3: BODY BLOCK
# ( Bare-Metal Rust Targets, LLVM Tools, and Global Binary Compilation )
# ================================================================================
# RATIONALE FOR RUSTUP COMPONENTS & TARGETS:
#   1. rust-src:
#       - Contains the source code of the Rust standard library (core, alloc, compiler_builtins).
#       - Required by cargo `-Z build-std` when compiling custom bare-metal OS kernels.
#   2. llvm-tools-preview:
#       - Ships llvm-objdump, llvm-objcopy, llvm-nm, and llvm-size for binary introspection.
#   3. Bare-Metal Target Triples:
#       - `x86_64-unknown-none`: 64-bit x86 bare-metal kernel target (no OS, no libc).
#       - `aarch64-unknown-none`: 64-bit ARM bare-metal target (no OS, no libc).
#       - `riscv64gc-unknown-none-elf`: 64-bit RISC-V with General Purpose + Compressed instructions.
# ================================================================================

RUN rustup component add rust-src llvm-tools-preview rustfmt clippy && \
    rustup target add x86_64-unknown-none && \
    rustup target add aarch64-unknown-none && \
    rustup target add riscv64gc-unknown-none-elf

# Establish standardized container workspace directory
WORKDIR /workspace

# Copy manifest, source tree, libraries, and tests to pre-compile the release binary
# into /usr/local/cargo/bin so `omnicode` is immediately available in the system PATH
COPY Cargo.toml Cargo.lock /workspace/
COPY src/ /workspace/src/
COPY lib/ /workspace/lib/
COPY examples/ /workspace/examples/
COPY tests/ /workspace/tests/

# Build and install the canonical OmniCode binary globally into the image
RUN cargo build --release && \
    cargo test --release && \
    cp /workspace/target/release/omnicode /usr/local/bin/omnicode && \
    chmod +x /usr/local/bin/omnicode

# Comprehensive toolchain verification during build
RUN echo "=== Verifying OmniCode Multi-Target Toolchain ===" && \
    omnicode --version && \
    rustc --version && \
```

```
    cargo --version && \
    clang --version && \
    lld --version && \
    nasm -v && \
    qemu-system-x86_64 --version && \
    qemu-system-aarch64 --version && \
    qemu-system-riscv64 --version && \
    gdb-multiarch --version && \
    node --version && \
    npm --version


# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ( Runtime Verification, Environment Seal & Default Entrypoint )
# ================================================================================
# ENTRYPOINT & INVARIANTS:
#   - Default command drops into an interactive bash shell where developers or
#     AI subagents can immediately invoke `omnicode repl`, `omnicode run`,
#     `omnicode ai`, `omnicode scrolls`, or run QEMU bare-metal kernel sessions.
#   - In docker-compose, this workspace is overlaid with the host volume mount
#     to provide live two-way code synchronization.
# ================================================================================

# Default command: interactive bash shell with full toolchain availability
CMD ["bash"]
```

<TRACK 5> : {Infrastructure, Operations & Archives}/infra/docker/docker-compose.yml

YML | 113 LINES | 650 WORDS | 5.7 KB

```yaml
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Container Orchestration Identity, Infrastructure Manifest & Cognitive Intent )
# ================================================================================
# Project:      OmniCode Language Engine & Bare-Metal OS
# Compose File: docker-compose.yml
# Service:      omnicode-toolchain
# Purpose:      Multi-tier development, bare-metal hardware emulation, and cognitive execution
#
# WHY EXTENSIVE COMMENTING MATTERS FOR AI & DEVELOPERS:
#   - For AI Autonomous Agents:
#     When an AI inspects orchestration files, bare configuration directives (e.g.
#     `1234:1234`) lack intent. Detailed comments explain the semantic contract:
#     Port 1234 connects to QEMU's GDB stub; modifying it breaks kernel debugging.
#     Named volume `target-cache` prevents cross-compilation thrashing between the
#     host and Linux container. These comments serve as immutable guardrails.
#   - For Human Collaborators:
#     Eliminates onboarding friction. Developers immediately know how to attach an
#     interactive REPL, connect a debugger to a bootloader, or run the AI cognitive loop.
# ================================================================================

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( Service Topology, Container Preconditions & Interactive Shell Configuration )
# ================================================================================
services:
  omnicode-toolchain:
    build:
      context: .
      dockerfile: Dockerfile
    image: omnicode-toolchain:latest
    container_name: omnicode-toolchain

    # Interactive TTY configuration:
    # `stdin_open: true` (-i) and `tty: true` (-t) keep STDIN open and allocate
    # a pseudo-TTY, enabling interactive `omnicode repl` and terminal debugging sessions.
    stdin_open: true
    tty: true

    # Execution root directory inside the container
    working_dir: /workspace

# ================================================================================
# BLOCK 3: BODY BLOCK
# ( Volume Topology, Network Port Routing & Environment Invariant Configuration )
# ================================================================================
    # VOLUME MOUNTING STRATEGY & EXPLANATION:
    #   1. `.:/workspace`:
    #      Two-way live bind mount between host project root and container.
    #      Edits in host IDEs (VS Code, Cursor, Antigravity) appear immediately inside
    #      the container, and generated `.scroll` episodic memory logs are saved to host.
    #   2. `cargo-cache:/usr/local/cargo/registry`:
    #      Named persistent volume caching downloaded crates.io index and packages,
    #      preventing redundant downloads across container restarts.
    #   3. `cargo-git:/usr/local/cargo/git`:
    #      Named persistent volume caching git dependencies fetched by Cargo.
    #   4. `target-cache:/workspace/target`:
    #      Named persistent volume isolating Linux ELF build artifacts from the host OS.
    #      CRITICAL: Prevents target directory collisions if the host machine is macOS
    #      or Windows, and dramatically speeds up compilation by avoiding 9p/FUSE overhead.
    volumes:
      - .:/workspace
      - cargo-cache:/usr/local/cargo/registry
      - cargo-git:/usr/local/cargo/git
      - target-cache:/workspace/target

    # PORT MAPPING TOPOLOGY:
    #   - 1234: QEMU Remote GDB Debugging Stub.
    #           When launching a bare-metal kernel with `qemu-system-x86_64 -s -S`,
    #           QEMU halts execution at the first instruction and listens on TCP port 1234.
    #           Attach from host or container with `gdb-multiarch -ex "target remote localhost:1234"`.
    #   - 5900: QEMU VNC Graphical Framebuffer.
    #           Bare-metal graphical displays rendered via VESA/GOP or virtio-gpu are
```

```
    #           streamed over VNC. Connect any VNC viewer to `localhost:5900`.
    #   - 8080: OmniCode Cognitive Daemon & Language Server.
    #           Reserved for future HTTP/WebSocket language server or telemetry visualization.
    ports:
      - "1234:1234" # QEMU GDB remote debugging socket
      - "5900:5900" # QEMU VNC graphical framebuffer display
      - "8080:8080" # OmniCode Cognitive Daemon / Web Inspector

    # RUNTIME ENVIRONMENT INVARIANTS:
    #   - RUST_BACKTRACE=1: Produces full symbolic stack traces upon error or panic.
    #   - OMNICODE_SCROLL_DIR: Directs episodic cognitive logs to the shared workspace.
    #   - OMNICODE_LIB_DIR: Resolves built-in standard library modules (Core.Arithmetic, etc.).
    environment:
      - RUST_BACKTRACE=1
      - OMNICODE_SCROLL_DIR=/workspace/scrolls
      - OMNICODE_LIB_DIR=/workspace/lib

    # Default command drops into an interactive bash shell ready for commands:
    # e.g., `omnicode repl`, `omnicode run <file>`, `omnicode ai "<prompt>"`, or `./scripts/check_toolchain.sh`
    command: bash

# ============================================================================
# BLOCK 4: CLOSING BLOCK
# ( Volume Persistence Declarations & Invariant Storage Lifecycle )
# ============================================================================
# NAMED VOLUME DECLARATIONS:
#   These volumes persist independently of container destruction:
#   - Run `docker compose down` to stop the container without losing caches.
#   - Run `docker compose down -v` to explicitly purge build caches if clean builds are required.
volumes:
  cargo-cache:
    labels:
      description: "Persistent cache for Cargo crates.io package registry"
  cargo-git:
    labels:
      description: "Persistent cache for Cargo git repositories"
  target-cache:
    labels:
      description: "Isolated persistent build cache for target/ binary compilation"
```

<TRACK 5> : {Infrastructure, Operations & Archives}/infra/scripts/check_toolchain.sh

SH | 57 LINES | 227 WORDS | 2.5 KB

```sh
#!/usr/bin/env bash
# ================================================================================
# BLOCK 1: METADATA BLOCK
# ( Toolchain Diagnostics Identity & Environment Manifest )
# ================================================================================
set -e

echo -e "\x1b[1;36m=============================================================================================\x1b[0m"
echo -e "\x1b[1;36m  OmniCode Toolchain Diagnostics & Verification Harness\x1b[0m"
echo -e "\x1b[1;36m=============================================================================================\x1b[0m\n"

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ( Dependency Check Function & Invariant Preconditions )
# ================================================================================
check_tool() {
    local name="$1"
    local cmd="$2"
    if command -v "$cmd" >/dev/null 2>&1; then
        echo -e "  \x1b[1;32m✓\x1b[0m $name: \x1b[33m$($cmd --version 2>&1 | head -n 1)\x1b[0m"
    else
        echo -e "  \x1b[1;31m✗\x1b[0m $name is NOT installed ($cmd missing)"
        return 1
    fi
}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ( Core Compilers, Assemblers, Emulators & Bare-Metal Verification )
# ================================================================================
echo -e "\x1b[1;34m[1/3] Core Compilers & Assemblers:\x1b[0m"
check_tool "Rust Compiler" rustc
check_tool "Cargo" cargo
check_tool "Clang" clang
check_tool "LLD Linker" lld
check_tool "NASM Assembler" nasm

echo -e "\n\x1b[1;34m[2/3] Hardware Emulation & Bootloaders:\x1b[0m"
check_tool "QEMU x86_64" qemu-system-x86_64
check_tool "Xorriso" xorriso

echo -e "\n\x1b[1;34m[3/3] Rust Bare-Metal Target Verification:\x1b[0m"
TARGETS=$(rustup target list --installed)
for target in "x86_64-unknown-none" "aarch64-unknown-none" "riscv64gc-unknown-none-elf"; do
    if echo "$TARGETS" | grep -q "$target"; then
        echo -e "  \x1b[1;32m✓\x1b[0m Installed target: \x1b[33m$target\x1b[0m"
    else
        echo -e "  \x1b[1;33m!\x1b[0m Missing bare-metal target: $target (Install with 'rustup target add $target')"
    fi
done

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ( Final Status Summary & Exit Verification )
# ================================================================================
echo -e "\n\x1b[1;32m✓ Toolchain verification complete.\x1b[0m"
```

<TRACK 5> : {Infrastructure, Operations & Archives}/infra/scripts/deploy_oracle_cloud.sh

SH | 160 LINES | 721 WORDS | 7.0 KB

```sh
#!/usr/bin/env bash
# ================================================================================
# CreativeWorkzStudio LLC - OmniCode Platform
# Oracle Cloud Ubuntu VM Turnkey Provisioning & Deployment Script
# ================================================================================
# Turnkey setup script for deploying OmniCode 24/7 on an Oracle Cloud Compute Instance:
# 1. Installs system build prerequisites, Rust/Cargo compiler, and git.
# 2. Configures OS firewall & iptables to allow Port 8080 (Web Studio & Cognitive API).
# 3. Builds the optimized OmniCode release binary (`cargo build --release`).
# 4. Installs and registers the systemd 24/7 service (`omnicode-web.service`).
# 5. Verifies service health and edge availability.
#
# Usage (run as root or with sudo on the Oracle Cloud VM):
#   sudo ./scripts/deploy_oracle_cloud.sh
# ================================================================================

set -e

# Visual styling
BOLD="\033[1m"
GREEN="\033[0;32m"
BLUE="\033[0;34m"
CYAN="\033[0;36m"
YELLOW="\033[1;33m"
RED="\033[0;31m"
NC="\033[0m"

echo -e "${BOLD}${CYAN}================================================================================${NC}"
echo -e "${BOLD}${CYAN}  🌌 CreativeWorkzStudio LLC — OmniCode Platform                              ${NC}"
echo -e "${BOLD}${CYAN}  Oracle Cloud VM Turnkey Deployment & Systemd Provisioner                     ${NC}"
echo -e "${BOLD}${CYAN}================================================================================${NC}"

if [ "$EUID" -ne 0 ]; then
    echo -e "${RED}[!] Error: This script must be run as root or via sudo.${NC}"
    echo -e "    Please run: sudo $0"
    exit 1
fi

REAL_USER="${SUDO_USER:-$USER}"
REAL_HOME="$(eval echo "~$REAL_USER")"
SCRIPT_DIR="$(cd -P "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
BASE_DIR="$(cd "$SCRIPT_DIR/.." && pwd)"

echo -e "${BLUE}[*] Provisioning for host user: ${GREEN}$REAL_USER${NC}"
echo -e "${BLUE}[*] OmniCode base directory:    ${GREEN}$BASE_DIR${NC}"

# Detect Architecture
ARCH="$(uname -m)"
echo -e "${BLUE}[*] Target CPU architecture:    ${GREEN}$ARCH${NC}"

# --------------------------------------------------------------------------------
# STEP 1: System Packages and Build Essentials
# --------------------------------------------------------------------------------
echo ""
echo -e "${BOLD}${BLUE}[1/5] Updating packages and installing build prerequisites...${NC}"
export DEBIAN_FRONTEND=noninteractive
apt-get update -y
apt-get install -y \
    build-essential \
    curl \
    git \
    pkg-config \
    libssl-dev \
    iptables \
    iptables-persistent \
    netfilter-persistent \
    ca-certificates

# Ensure Rust & Cargo are installed for the target user
if ! sudo -u "$REAL_USER" command -v cargo >/dev/null 2>&1; then
    echo -e "${BLUE}[*] Rust compiler not found. Installing Rust toolchain via rustup...${NC}"
    sudo -u "$REAL_USER" curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sudo -u "$REAL_USER" sh -s -- -y
    echo -e "${GREEN}[+] Rust toolchain installed.${NC}"
```

```
else
    echo -e "${GREEN}[+] Rust toolchain already available: $(sudo -u "$REAL_USER" cargo --version)${NC}"
fi

# ------------------------------------------------------------------------------
# STEP 2: Configure Oracle Cloud Firewall & iptables
# ------------------------------------------------------------------------------
echo ""
echo -e "${BOLD}${BLUE}[2/5] Configuring OS Firewall & iptables for Port 8080...${NC}"
PORTS=(22 8080)

for PORT in "${PORTS[@]}"; do
    if ! iptables -C INPUT -p tcp --dport "$PORT" -j ACCEPT 2>/dev/null; then
        echo -e "${BLUE}[*] Opening TCP Port $PORT in iptables...${NC}"
        iptables -I INPUT 1 -p tcp --dport "$PORT" -j ACCEPT
    else
        echo -e "${GREEN}[+] TCP Port $PORT already allowed in iptables.${NC}"
    fi
done

# Persist firewall rules
netfilter-persistent save || true

# Check UFW if active
if command -v ufw >/dev/null 2>&1 && ufw status 2>/dev/null | grep -q "Status: active"; then
    echo -e "${BLUE}[*] UFW firewall active. Permitting port 8080...${NC}"
    ufw allow 22/tcp
    ufw allow 8080/tcp
    ufw reload
fi
echo -e "${GREEN}[+] OS firewall configured for OmniCode Web & Cognitive Studio (Port 8080).${NC}"

# ------------------------------------------------------------------------------
# STEP 3: Compile OmniCode Release Binary
# ------------------------------------------------------------------------------
echo ""
echo -e "${BOLD}${BLUE}[3/5] Compiling OmniCode release binary with Cargo...${NC}"
export PATH="$REAL_HOME/.cargo/bin:$PATH"
sudo -u "$REAL_USER" env PATH="$REAL_HOME/.cargo/bin:$PATH" cargo build --release --manifest-path "$BASE_DIR/Cargo.toml"
echo -e "${GREEN}[+] OmniCode release binary built at: $BASE_DIR/target/release/omnicode${NC}"

# ------------------------------------------------------------------------------
# STEP 4: Install and Enable Systemd Service
# ------------------------------------------------------------------------------
echo ""
echo -e "${BOLD}${BLUE}[4/5] Registering and enabling systemd 24/7 service...${NC}"
SERVICE_SRC="$BASE_DIR/systemd/omnicode-web.service"
SERVICE_DEST="/etc/systemd/system/omnicode-web.service"

sed -e "s|/home/ubuntu/OmniCode|$BASE_DIR|g" \
    -e "s|User=ubuntu|User=$REAL_USER|g" \
    "$SERVICE_SRC" > "$SERVICE_DEST"

systemctl daemon-reload
systemctl enable omnicode-web.service
systemctl restart omnicode-web.service
echo -e "${GREEN}[+] omnicode-web.service installed and restarted.${NC}"

# ------------------------------------------------------------------------------
# STEP 5: Verification and Endpoint Summary
# ------------------------------------------------------------------------------
echo ""
echo -e "${BOLD}${BLUE}[5/5] Verifying service health and endpoints...${NC}"
sleep 2

STATUS=$(systemctl is-active omnicode-web.service || echo "failed")
PUBLIC_IP="$(curl -s -4 https://ifconfig.me || curl -s -4 https://api.ipify.org || echo "ORACLE_VM_PUBLIC_IP")"

echo ""
echo -e "${BOLD}${GREEN}================================================================================${NC}"
echo -e "${BOLD}${GREEN}  🎉 OMNICODE ORACLE CLOUD 24/7 DEPLOYMENT COMPLETE!                ${NC}"
echo -e "${BOLD}${GREEN}  CreativeWorkzStudio LLC                                           ${NC}"
echo -e "${BOLD}${GREEN}================================================================================${NC}"
echo -e "Service Status:         ${BOLD}${GREEN}$STATUS${NC}"
echo -e "Public Server IP:       ${BOLD}${YELLOW}$PUBLIC_IP${NC}"
echo -e "Web Studio & API:       ${BOLD}${CYAN}http://$PUBLIC_IP:8080${NC}"
echo ""
echo -e "${BOLD}Management Commands:${NC}"
```

```
echo -e "  Check service status:  ${BLUE}sudo systemctl status omnicode-web${NC}"
echo -e "  Restart service:       ${BLUE}sudo systemctl restart omnicode-web${NC}"
echo -e "  View live logs:        ${BLUE}sudo journalctl -u omnicode-web -f${NC}"
echo ""
echo -e "${BOLD}Cloudflare Integration:${NC}"
echo -e "  To route edge traffic through Cloudflare, launch the tunnel connector:"
echo -e "  ${BLUE}$BASE_DIR/scripts/setup_cloudflare_tunnel.sh --quick${NC}"
echo -e "${BOLD}${GREEN}================================================================================${NC}"
```

<TRACK 5> : {Infrastructure, Operations & Archives}/infra/scripts/oci_security_list_helper.sh

SH | 87 LINES | 474 WORDS | 4.1 KB

```sh
#!/usr/bin/env bash
# ============================================================================
# CreativeWorkzStudio LLC - OmniCode Platform
# Oracle Cloud VCN Ingress/Egress & Firewall Configuration Helper
# ============================================================================
# Helper script to manage VM OS firewall (iptables) and OCI VCN Security List
# for OmniCode Studio (port 8080).
#
# Requirements:
#   1. VM OS Firewall: iptables + netfilter-persistent (Already configured)
#   2. OCI Cloud Firewall: OCI CLI (installed at /usr/local/bin/oci) or OCI Console
# ============================================================================

set -e

BOLD="\033[1m"
GREEN="\033[0;32m"
BLUE="\033[0;34m"
CYAN="\033[0;36m"
YELLOW="\033[1;33m"
NC="\033[0m"

echo -e "${BOLD}${CYAN}============================================================================${NC}"
echo -e "${BOLD}${CYAN}  🛡️  Oracle Cloud VM Firewall & Ingress/Egress Helper                ${NC}"
echo -e "${BOLD}${CYAN}  CreativeWorkzStudio LLC — OmniCode Platform (Port 8080)             ${NC}"
echo -e "${BOLD}${CYAN}============================================================================${NC}"

# 1. Verify OS-level iptables rules
echo -e "\n${BLUE}[1/3] Checking OS-level iptables firewall rules for Port 8080...${NC}"
if sudo iptables -L INPUT -n | grep -q "dpt:8080"; then
    echo -e "${GREEN}[✓] Port 8080 is ACCEPT in iptables INPUT chain.${NC}"
else
    echo -e "${YELLOW}[*] Adding Port 8080 to iptables INPUT chain...${NC}"
    sudo iptables -I INPUT 6 -p tcp --dport 8080 -j ACCEPT
fi

# Ensure rules are saved persistently
if command -v netfilter-persistent >/dev/null 2>&1; then
    sudo netfilter-persistent save >/dev/null 2>&1
    echo -e "${GREEN}[✓] iptables rules saved persistently via netfilter-persistent.${NC}"
fi

# 2. Check OCI CLI installation
echo -e "\n${BLUE}[2/3] Checking Oracle Cloud Infrastructure CLI (oci-cli)...${NC}"
if command -v oci >/dev/null 2>&1; then
    OCI_VERSION=$(oci --version)
    echo -e "${GREEN}[✓] OCI CLI is installed at: $(which oci) (version: $OCI_VERSION)${NC}"
else
    echo -e "${YELLOW}[!] OCI CLI not found. Run setup on VM: sudo pip3 install oci-cli${NC}"
fi

# 3. Guidance for OCI VCN Security List (Cloud Layer)
echo -e "\n${BLUE}[3/3] Oracle Cloud VCN Ingress Rule Instructions (Cloud Hypervisor Layer):${NC}"
cat << 'EOF'

In Oracle Cloud Infrastructure (OCI), incoming traffic must pass through TWO layers:
  Layer 1: Linux OS Firewall (iptables) -> ALREADY CONFIGURED & PERSISTED (Port 8080 OPEN)
  Layer 2: OCI VCN Security List        -> Requires Cloud Ingress Rule

Option A: Via OCI Web Console (1-Minute Setup)
  1. Open Oracle Cloud Console (https://cloud.oracle.com)
  2. Navigate to: Networking > Virtual Cloud Networks > Select your VCN
  3. Under Resources, click "Security Lists" > Click "Default Security List for [VCN]"
  4. Click "Add Ingress Rules":
     - Source CIDR:            0.0.0.0/0
     - IP Protocol:            TCP
     - Destination Port Range: 8080
     - Description:           OmniCode Studio & Interactive Terminal GUI
  5. Click "Add Ingress Rules" to save.

Option B: Via OCI CLI (Command Line on VM)
  If you have configured OCI API credentials (`oci setup config`):
  Run:
```

```
    oci network security-list update \
      --security-list-id <YOUR_SECURITY_LIST_OCID> \
      --ingress-security-rules file://<(echo '[{"source":"0.0.0.0/0","protocol":"6","tcpOptions":{"destinationPortRange":
{"max":8080,"min":8080}},"description":"OmniCode Studio"}]')

Option C: Via Cloudflare Zero Trust Tunnel (Zero Open Ports Required)
  Because cloudflared is already running on the VM, you can route:
  - Hostname: studio.omnicodestudio.com -> HTTP://localhost:8080
  This routes traffic securely via Cloudflare edge without needing any open ports in OCI VCN!
EOF

echo -e "\n${BOLD}${GREEN}================================================================================${NC}"
echo -e "${BOLD}${GREEN}  ✓ Firewall verification & guidance completed.                      ${NC}"
echo -e "${BOLD}${GREEN}================================================================================${NC}"
```

<TRACK 5> : {Infrastructure, Operations & Archives}/infra/scripts/omnicode_gui.sh

SH | 16 LINES | 53 WORDS | 0.5 KB

```sh
#!/usr/bin/env bash
# ============================================================================
# CreativeWorkzStudio LLC — OmniCode Desktop GUI Studio Launcher
# ============================================================================
set -e

SCRIPT_DIR="$(cd -P "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
BASE_DIR="$(cd "$SCRIPT_DIR/.." && pwd)"

if command -v python3 >/dev/null 2>&1; then
    exec python3 "$BASE_DIR/gui/desktop_app.py" "$@"
else
    echo "Python 3 is required to launch the OmniCode Desktop GUI."
    exit 1
fi
```

<TRACK 5> : {Infrastructure, Operations & Archives}/infra/scripts/package_native.sh

SH | 125 LINES | 395 WORDS | 4.2 KB

```sh
#!/usr/bin/env bash
# ================================================================================
# CreativeWorkzStudio LLC - OmniCode Platform
# Automated Native Terminal & Language OS Distribution Packager
# ================================================================================
# Compiles the optimized release binary and bundles a standalone native
# distribution archive with zero runtime dependencies.
#
# Output:
#   dist/omnicode-terminal-linux-x86_64.tar.gz
#   dist/omnicode-terminal-latest.tar.gz
# ================================================================================

set -e

SCRIPT_DIR="$(cd -P "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
if [ -f "$SCRIPT_DIR/../../Cargo.toml" ]; then
    BASE_DIR="$(cd "$SCRIPT_DIR/../.." && pwd)"
else
    BASE_DIR="$(cd "$SCRIPT_DIR/.." && pwd)"
fi
DIST_DIR="$BASE_DIR/infra/dist"
STAGING_DIR="/tmp/omnicode_dist_staging"

# ANSI styling
BOLD="\033[1m"
GREEN="\033[0;32m"
BLUE="\033[0;34m"
CYAN="\033[0;36m"
NC="\033[0m"

echo -e "${BOLD}${CYAN}================================================================================${NC}"
echo -e "${BOLD}${CYAN}  📦 OmniCode Native Distribution Packager                                      ${NC}"
echo -e "${BOLD}${CYAN}  CreativeWorkzStudio LLC — Standalone Native OS Shell                          ${NC}"
echo -e "${BOLD}${CYAN}================================================================================${NC}"

# 1. Compile release binary locally
echo -e "\n${BLUE}[1/4] Compiling optimized release binary with cargo...${NC}"
(cd "$BASE_DIR" && cargo build --release)

# 2. Prepare staging directory
echo -e "\n${BLUE}[2/4] Assembling standalone distribution files...${NC}"
rm -rf "$STAGING_DIR"
mkdir -p "$STAGING_DIR/bin"
mkdir -p "$STAGING_DIR/lib/core"
mkdir -p "$STAGING_DIR/examples"
mkdir -p "$STAGING_DIR/editors"
mkdir -p "$DIST_DIR"

# Copy binary
cp "$BASE_DIR/target/release/omnicode" "$STAGING_DIR/bin/omnicode"
chmod +x "$STAGING_DIR/bin/omnicode"

# Copy standard libraries and manifest
cp -r "$BASE_DIR/lib/core/"*.omni "$STAGING_DIR/lib/core/"
cp "$BASE_DIR/manifest.omni" "$STAGING_DIR/"
cp -r "$BASE_DIR/examples/"*.omni "$STAGING_DIR/examples/"

# Copy VS Code extension if available
if [ -d "$BASE_DIR/apps/editors/vscode" ]; then
    cp "$BASE_DIR/apps/editors/vscode/"*.vsix "$STAGING_DIR/editors/" 2>/dev/null || true
fi

# Create turnkey install script
cat << 'EOF' > "$STAGING_DIR/install.sh"
#!/usr/bin/env bash
# CreativeWorkzStudio LLC - OmniCode Standalone Installer
set -e

DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"

echo "Installing OmniCode Native Terminal..."

```

```
# Target install directory
if [ "$EUID" -eq 0 ]; then
    INSTALL_DIR="/usr/local/bin"
else
    INSTALL_DIR="$HOME/.local/bin"
    mkdir -p "$INSTALL_DIR"
fi

cp "$DIR/bin/omnicode" "$INSTALL_DIR/omnicode"
chmod +x "$INSTALL_DIR/omnicode"

echo "✓ OmniCode installed successfully to $INSTALL_DIR/omnicode"
echo "Testing installation:"
"$INSTALL_DIR/omnicode" demo

echo ""
echo "🎉 OmniCode Native Shell is ready! Type 'omnicode' to enter interactive terminal."
EOF
chmod +x "$STAGING_DIR/install.sh"

# Create Distribution README
cat << 'EOF' > "$STAGING_DIR/README.md"
# OmniCode Native Terminal & Language Core
CreativeWorkzStudio LLC — Autonomous Neural-Symbolic Intelligence

## Quick Start
Run turnkey installer:
  ./install.sh

Or run binary directly:
  ./bin/omnicode

Launch interactive userspace shell:
  ./bin/omnicode terminal
EOF

# 3. Create compressed archives
echo -e "\n${BLUE}[3/4] Creating standalone distribution archives in dist/...${NC}"
ARCHIVE_NAME="omnicode-terminal-linux-x86_64.tar.gz"
(cd "/tmp" && tar -czf "$DIST_DIR/$ARCHIVE_NAME" -C "$STAGING_DIR" .)
cp "$DIST_DIR/$ARCHIVE_NAME" "$DIST_DIR/omnicode-terminal-latest.tar.gz"

# 4. Report
echo -e "\n${BLUE}[4/4] Package Verification:${NC}"
ls -lh "$DIST_DIR/$ARCHIVE_NAME"
sha256sum "$DIST_DIR/$ARCHIVE_NAME"

echo -e "\n${BOLD}${GREEN}================================================================================${NC}"
echo -e "${BOLD}${GREEN}  ✅ Standalone Native Distribution Package Created Successfully!     ${NC}"
echo -e "${BOLD}${GREEN}  Output: $DIST_DIR/$ARCHIVE_NAME                                     ${NC}"
echo -e "${BOLD}${GREEN}================================================================================${NC}"
```

<TRACK 5> : {Infrastructure, Operations & Archives}/infra/scripts/setup_cloudflare_tunnel.sh

SH | 194 LINES | 713 WORDS | 6.8 KB

```sh
#!/usr/bin/env bash
# ============================================================================
# CreativeWorkzStudio LLC - OmniCode Platform
# Cloudflare Zero Trust Secure HTTPS Tunnel Connector
# ============================================================================
# Provides secure edge HTTPS routing to the local OmniCode Studio:
# 1. Quick Tunnel (Free, zero-config ephemeral HTTPS endpoint via trycloudflare.com):
#    Useful for local staging, development, and remote access.
# 2. Named Tunnel (Custom domain via Cloudflare Zero Trust Tunnel Token):
#    Production 24/7 tunnel mapped to your custom domain.
#
# Usage:
#   ./scripts/setup_cloudflare_tunnel.sh            # Auto-detects token from .env or starts quick
#   ./scripts/setup_cloudflare_tunnel.sh --quick    # Starts instant ephemeral quick tunnel
#   ./scripts/setup_cloudflare_tunnel.sh --token <TOKEN> # Starts named tunnel with specified token
#   ./scripts/setup_cloudflare_tunnel.sh --port <PORT>   # Specify local port (default: 8080)
# ============================================================================

set -e

# Visual formatting constants
BOLD="\033[1m"
GREEN="\033[0;32m"
BLUE="\033[0;34m"
CYAN="\033[0;36m"
YELLOW="\033[1;33m"
RED="\033[0;31m"
NC="\033[0m"

# Canonical directory resolution
SCRIPT_DIR="$(cd -P "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
if [ -f "$SCRIPT_DIR/../../Cargo.toml" ]; then
    BASE_DIR="$(cd "$SCRIPT_DIR/../.." && pwd)"
else
    BASE_DIR="$(cd "$SCRIPT_DIR/.." && pwd)"
fi
ENV_FILE="$BASE_DIR/.env"

PORT="8080"
MODE="auto"
CLI_TOKEN=""

echo -e "${BOLD}${CYAN}============================================================================${NC}"
echo -e "${BOLD}${CYAN}  🌐 OmniCode Cloudflare Tunnel Connector                                  ${NC}"
echo -e "${BOLD}${CYAN}  CreativeWorkzStudio LLC — Edge Security & Gateway                       ${NC}"
echo -e "${BOLD}${CYAN}============================================================================${NC}"

# Ensure cloudflared is installed
ensure_cloudflared() {
    if command -v cloudflared >/dev/null 2>&1; then
        echo -e "${GREEN}[+] cloudflared binary verified: $(cloudflared --version | head -n 1)${NC}"
        return 0
    fi

    echo -e "${BLUE}[*] cloudflared not found in PATH. Checking system architecture...${NC}"
    ARCH="$(uname -m)"
    case "$ARCH" in
        x86_64)
            CLOUDFLARED_URL="https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-amd64"
            ;;
        aarch64|arm64)
            CLOUDFLARED_URL="https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-arm64"
            ;;
        armv7l)
            CLOUDFLARED_URL="https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-arm"
            ;;
        *)
            echo -e "${RED}[-] Unsupported architecture: $ARCH. Please install cloudflared manually.${NC}"
            exit 1
            ;;
    esac

    echo -e "${BLUE}[*] Downloading cloudflared binary for $ARCH...${NC}"
```

```
    TMP_BIN="/tmp/cloudflared"
    curl -fsSL "$CLOUDFLARED_URL" -o "$TMP_BIN"
    chmod +x "$TMP_BIN"

    if [ "$EUID" -eq 0 ]; then
        mv "$TMP_BIN" /usr/local/bin/cloudflared
    elif sudo -n true 2>/dev/null; then
        sudo mv "$TMP_BIN" /usr/local/bin/cloudflared
    else
        mkdir -p "$BASE_DIR/bin"
        mv "$TMP_BIN" "$BASE_DIR/bin/cloudflared"
        export PATH="$BASE_DIR/bin:$PATH"
    fi

    echo -e "${GREEN}[+] cloudflared successfully installed.${NC}"
}

# Parse CLI arguments
while [ $# -gt 0 ]; do
    case "$1" in
        --quick)
            MODE="quick"
            shift
            ;;
        --token)
            MODE="token"
            CLI_TOKEN="$2"
            shift 2
            ;;
        --port)
            PORT="$2"
            shift 2
            ;;
        *)
            shift
            ;;
    esac
done

ensure_cloudflared

# Resolve Token from .env if auto
ENV_TOKEN=""
if [ -f "$ENV_FILE" ]; then
    ENV_TOKEN=$(grep -E '^CLOUDFLARE_TUNNEL_TOKEN=' "$ENV_FILE" 2>/dev/null | cut -d '=' -f 2- | tr -d '"' | tr -d "'" || true)
fi

# Fallback: check yugioh-server .env if available
if [ -z "$ENV_TOKEN" ] && [ -f "/home/professorseanex/yugioh-server/.env" ]; then
    ENV_TOKEN=$(grep -E '^CLOUDFLARE_TUNNEL_TOKEN=' "/home/professorseanex/yugioh-server/.env" 2>/dev/null | cut -d '=' -f 2- | tr -d '"' | tr -d "'" || true)
fi

if [ "$MODE" = "token" ] && [ -n "$CLI_TOKEN" ]; then
    ACTIVE_TOKEN="$CLI_TOKEN"
elif [ "$MODE" = "auto" ] && [ -n "$ENV_TOKEN" ] && [ "$ENV_TOKEN" != "your_cloudflare_tunnel_token_here" ]; then
    ACTIVE_TOKEN="$ENV_TOKEN"
    MODE="token"
else
    MODE="quick"
fi

# Execution
if [ "$MODE" = "token" ]; then
    echo -e "${BLUE}[*] Starting Cloudflare Named Tunnel using configured token...${NC}"
    echo -e "    Local Target  : http://localhost:$PORT (OmniCode Studio & API)"
    echo -e "    Token         : ${ACTIVE_TOKEN:0:15}... (Truncated for security)"
    echo ""
    echo -e "${YELLOW}[!] Tunnel running in foreground. Press Ctrl+C to terminate.${NC}"
    echo ""
    exec cloudflared tunnel --no-autoupdate run --token "$ACTIVE_TOKEN"
else
    echo -e "${BLUE}[*] Starting Cloudflare Quick Tunnel (Free, zero-config HTTPS edge)...${NC}"
    echo -e "    Local Target : http://localhost:$PORT"
    echo ""

    LOG_FILE="/tmp/omnicode_cloudflared_quick.log"
```

```
    rm -f "$LOG_FILE"

    cloudflared tunnel --url "http://localhost:$PORT" > "$LOG_FILE" 2>&1 &
    CF_PID=$!

    cleanup() {
        echo ""
        echo -e "${YELLOW}[*] Shutting down Cloudflare tunnel (PID: $CF_PID)...${NC}"
        kill "$CF_PID" 2>/dev/null || true
        exit 0
    }
    trap cleanup SIGINT SIGTERM EXIT

    echo -e "${BLUE}[*] Provisioning edge tunnel and acquiring public HTTPS URL...${NC}"
    URL=""
    for i in {1..30}; do
        if [ -f "$LOG_FILE" ]; then
            URL=$(grep -o 'https://[-a-zA-Z0-9\.]*\.trycloudflare\.com' "$LOG_FILE" | head -n 1 || true)
            if [ -n "$URL" ]; then
                break
            fi
        fi
        sleep 1
    done

    if [ -z "$URL" ]; then
        echo -e "${RED}[-] Timeout: Failed to obtain public tunnel URL within 30 seconds.${NC}"
        echo -e "    Log output:"
        cat "$LOG_FILE"
        exit 1
    fi

    echo ""
    echo -e "${BOLD}${GREEN}================================================================================${NC}"
    echo -e "${BOLD}${GREEN}  🎉 OMNICODE CLOUDFLARE HTTPS TUNNEL IS LIVE!                                  ${NC}"
    echo -e "${BOLD}${GREEN}================================================================================${NC}"
    echo -e "  Public HTTPS URL:  ${BOLD}${CYAN}${URL}${NC}"
    echo -e "  Local Destination: http://localhost:$PORT"
    echo -e "  Organization:      CreativeWorkzStudio LLC"
    echo ""
    echo -e "${YELLOW}[!] Keep this terminal open. Tunnel will close when stopped.${NC}"
    echo -e "    Press Ctrl+C to terminate."
    wait "$CF_PID"
fi
```

<TRACK 5> : {Infrastructure, Operations & Archives}/infra/scripts/sync_oracle_vm.sh

SH | 133 LINES | 577 WORDS | 5.6 KB

```sh
#!/usr/bin/env bash
# ==============================================================================
# CreativeWorkzStudio LLC - OmniCode Platform
# Oracle Cloud VM Automated Synchronization Script
# ==============================================================================
# Synchronizes the local OmniCode repository to the Oracle Cloud VM:
# 1. Connects to Oracle Cloud VM (147.224.147.30 / Host 'oracle') via SSH.
# 2. Synchronizes repository source files to /home/ubuntu/OmniCode using rsync.
# 3. Compiles the optimized release binary with cargo on the remote host.
# 4. Restarts systemd service: omnicode-web.service.
# 5. Queries and reports remote service health.
#
# Usage:
#   ./scripts/sync_oracle_vm.sh [optional commit or sync note]
# ==============================================================================

set -e

# ANSI Styling
GREEN="\033[0;32m"
BLUE="\033[0;34m"
CYAN="\033[0;36m"
YELLOW="\033[1;33m"
RED="\033[0;31m"
BOLD="\033[1m"
NC="\033[0m"

SCRIPT_DIR="$(cd -P "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
if [ -f "$SCRIPT_DIR/../../Cargo.toml" ]; then
    BASE_DIR="$(cd "$SCRIPT_DIR/../.." && pwd)"
else
    BASE_DIR="$(cd "$SCRIPT_DIR/.." && pwd)"
fi

# Target Configuration
ORACLE_HOST="${ORACLE_VM_HOST:-147.224.147.30}"
ORACLE_USER="${ORACLE_VM_USER:-ubuntu}"
SSH_KEY="${ORACLE_SSH_KEY:-$HOME/.ssh/oracle_key.pem}"
REMOTE_DIR="/home/ubuntu/OmniCode"

echo -e "${BOLD}${CYAN}================================================================================${NC}"
echo -e "${BOLD}${CYAN}  🌌 CreativeWorkzStudio LLC — OmniCode Platform                                ${NC}"
echo -e "${BOLD}${CYAN}  Oracle Cloud VM Automated Synchronization Gateway                              ${NC}"
echo -e "${BOLD}${CYAN}  Target: ${ORACLE_USER}@${ORACLE_HOST} (${REMOTE_DIR})                          ${NC}"
echo -e "${BOLD}${CYAN}================================================================================${NC}"

# Check SSH Key
if [ ! -f "$SSH_KEY" ]; then
    echo -e "${RED}[!] SSH Private Key not found at: ${SSH_KEY}${NC}"
    echo -e "    Please specify valid key path via ORACLE_SSH_KEY env var."
    exit 1
fi
chmod 600 "$SSH_KEY" 2>/dev/null || true

# Test SSH Connection
echo -e "\n${BLUE}[1/4] Verifying SSH connectivity to Oracle VM...${NC}"
if ! ssh -i "$SSH_KEY" -o ConnectTimeout=8 -o StrictHostKeyChecking=accept-new "${ORACLE_USER}@${ORACLE_HOST}" "echo 'Connected successfully'" >/dev/null 2>&1; then
    echo -e "${RED}[!] Failed to establish SSH connection to ${ORACLE_USER}@${ORACLE_HOST}.${NC}"
    exit 1
fi
echo -e "${GREEN}[+] Remote Oracle VM host reachable.${NC}"

# Ensure Remote Directory Exists
echo -e "\n${BLUE}[2/4] Ensuring remote workspace directory exists...${NC}"
ssh -i "$SSH_KEY" "${ORACLE_USER}@${ORACLE_HOST}" "mkdir -p $REMOTE_DIR"

# Rsync Files to Oracle VM (excluding target/ and .git/ large artifacts for speed)
echo -e "\n${BLUE}[3/4] Synchronizing repository files via rsync...${NC}"
rsync -avz --delete \
    -e "ssh -i $SSH_KEY -o StrictHostKeyChecking=accept-new" \
    --exclude "target/" \
    --exclude ".git/" \
```

```
    --exclude "node_modules/" \
    --exclude "*.log" \
    "$BASE_DIR/" "${ORACLE_USER}@${ORACLE_HOST}:${REMOTE_DIR}/"
echo -e "${GREEN}[+] Repository source synchronization complete.${NC}"

# Ensure release binary exists locally and transfer it
if [ ! -f "$BASE_DIR/target/release/omnicode" ]; then
    echo -e "${BLUE}[*] Compiling local release binary with cargo...${NC}"
    (cd "$BASE_DIR" && cargo build --release)
fi

echo -e "\n${BLUE}[*] Deploying optimized release binary to Oracle VM...${NC}"
ssh -i "$SSH_KEY" "${ORACLE_USER}@${ORACLE_HOST}" "mkdir -p ${REMOTE_DIR}/target/release"
scp -i "$SSH_KEY" "$BASE_DIR/target/release/omnicode" "${ORACLE_USER}@${ORACLE_HOST}:/tmp/omnicode.new"
ssh -i "$SSH_KEY" "${ORACLE_USER}@${ORACLE_HOST}" "mv -f /tmp/omnicode.new ${REMOTE_DIR}/target/release/omnicode && chmod
+x ${REMOTE_DIR}/target/release/omnicode"
echo -e "${GREEN}[+] Binary deployment complete.${NC}"

# Remote Service Configuration & Restart
echo -e "\n${BLUE}[4/4] Configuring firewall and restarting omnicode-web.service on Oracle VM...${NC}"
SSH_CMD="ssh -i \"$SSH_KEY\" \"${ORACLE_USER}@${ORACLE_HOST}\""

eval $SSH_CMD << 'EOF'
set -e
GREEN="\033[0;32m"
BLUE="\033[0;34m"
NC="\033[0m"

cd /home/ubuntu/OmniCode

# Install / update systemd service
SERVICE_FILE="infra/systemd/omnicode-web.service"
if [ ! -f "$SERVICE_FILE" ]; then
    SERVICE_FILE="systemd/omnicode-web.service"
fi
if [ -f "$SERVICE_FILE" ]; then
    echo -e "${BLUE}[*] Installing/updating omnicode-web.service from $SERVICE_FILE...${NC}"
    sudo cp "$SERVICE_FILE" /etc/systemd/system/omnicode-web.service
    sudo systemctl daemon-reload
    sudo systemctl enable omnicode-web.service
    sudo systemctl restart omnicode-web.service
fi

# Ensure TCP port 8080 is permitted in iptables
if ! sudo iptables -L INPUT -n | grep -q "dpt:8080"; then
    echo -e "${BLUE}[*] Opening port 8080 in iptables...${NC}"
    sudo iptables -I INPUT 6 -p tcp --dport 8080 -j ACCEPT
    if command -v netfilter-persistent >/dev/null 2>&1; then
        sudo netfilter-persistent save >/dev/null 2>&1 || true
    fi
fi

sleep 2
STATUS=$(sudo systemctl is-active omnicode-web.service || echo "inactive")
echo -e " • omnicode-web.service status: [${STATUS}]"
EOF

echo -e "\n${BOLD}${GREEN}================================================================================${NC}"
echo -e "${BOLD}${GREEN}  ✅ OmniCode Oracle Cloud VM Synchronization Complete!              ${NC}"
echo -e "${BOLD}${GREEN}  CreativeWorkzStudio LLC                                           ${NC}"
echo -e "${BOLD}${GREEN}================================================================================${NC}"
```

<TRACK 5> : {Infrastructure, Operations & Archives}/infra/dns/README.md MD 71 LINES 380 WORDS 2.9 KB

```
# 🌐 OmniCode DNS Zone Infrastructure

Authoritative BIND RFC 1035 DNS zone configuration files for **OmniCode Studio & CreativeWorkzStudio LLC**, designed for 1-click import into Cloudflare DNS.

---

## 📁 File Manifest

| File | Standard | Description |
| :--- | :--- | :--- |
| [`omnicodestudio.com.zone`](file:///home/professorseanex/OmniCode/dns/omnicodestudio.com.zone) | RFC 1035 / BIND 9 | Master DNS zone file compatible with Cloudflare DNS Import |
| [`zone_manager.py`](file:///home/professorseanex/OmniCode/dns/zone_manager.py) | Python 3 | CLI generator, syntax validator, and record inspector |

---

## 🚀 How to Import into Cloudflare (Step-by-Step)

1. Log into your [Cloudflare Dashboard](https://dash.cloudflare.com/).
2. Select your domain (e.g. `omnicodestudio.com` or your company domain).
3. In the left navigation sidebar, click **DNS** → **Records**.
4. Scroll to the **Import and Export** card and click **Import**.
5. Select [`omnicodestudio.com.zone`](file:///home/professorseanex/OmniCode/dns/omnicodestudio.com.zone).
6. Click **Import**. Cloudflare will parse and create all records automatically.

---

## 🗺️ Architectural Routing Map

```
Internet Request
       |
       ├── HTTPS Web Studio (Port 8080) ──→ Cloudflare Tunnel (c346aa27...) ──→ Oracle VM / Local Port 8080
       |     (studio.omnicodestudio.com)
       |
       ├── DuckDNS Dynamic Host ──────────→ omnicodestudio.duckdns.org ──────→ 71.11.99.168 (CWS Gateway)
       |     (duck.omnicodestudio.com)
       |
       └── Yu-Gi-Oh! Game Simulator (7911) → play.omnicodestudio.com ────────→ 147.224.147.30 (Oracle VM)
             (Raw TCP - Grey Cloud Only!)
```

⚙️ Proxy Status Rules (Crucial!)

Record	Type	Target	Cloudflare Proxy Status	Why
`@`, `www`	`CNAME`	`c346aa27...cfargotunnel.com.`	**Proxied (Orange Cloud)**	Free edge SSL/TLS & DDoS mitigation
`studio`, `api`	`CNAME`	`c346aa27...cfargotunnel.com.`	**Proxied (Orange Cloud)**	Direct edge routing to OmniCode
`local`, `gateway`	`A`	`71.11.99.168`	**DNS-Only (Grey Cloud)**	CreativeWorkzStudio LLC WAN IP
`cloud`, `vm`	`A`	`147.224.147.30`	**DNS-Only (Grey Cloud)**	Direct Oracle VM SSH / Administration
`duck`, `edge`	`CNAME`	`omnicodestudio.duckdns.org.`	**DNS-Only (Grey Cloud)**	Dynamic IP synchronization
`play`, `sim`	`A`	`147.224.147.30`	**DNS-Only (Grey Cloud)**	**MUST BE GREY CLOUD**: Raw TCP port 7911

🛠️ CLI Zone Management

```bash
cd /home/professorseanex/OmniCode

# Validate zone file syntax
python3 dns/zone_manager.py validate

# Inspect all configured records
python3 dns/zone_manager.py inspect

# Re-render with a custom domain
```
```

```
python3 dns/zone_manager.py render --domain yourcompany.com --out dns/yourcompany.com.zone
```

<TRACK 5> : {Infrastructure, Operations & Archives}/infra/dns/omnicodestudio.com.zone

ZONE | 49 LINES | 249 WORDS | 2.6 KB

```
; ================================================================================
; BIND RFC 1035 Zone File for omnicodestudio.com
; CreativeWorkzStudio LLC — OmniCode Platform Infrastructure
; Compatible with Cloudflare DNS Zone File Import (DNS -> Records -> Import)
; ================================================================================
$ORIGIN omnicodestudio.com.
$TTL 3600

; ================================================================================
; 1. PRIMARY WEB STUDIO & REST API (HTTP/HTTPS - Port 8080)
; ================================================================================
; Option A: Cloudflare Zero Trust Named Tunnel (Recommended)
; Tunnel ID: c346aa27-b4c1-41ba-815c-f66b2b84243b
; Benefits: Free SSL certificates, zero open firewall ports, DDoS mitigation.
@       IN  CNAME   c346aa27-b4c1-41ba-815c-f66b2b84243b.cfargotunnel.com.
www     IN  CNAME   c346aa27-b4c1-41ba-815c-f66b2b84243b.cfargotunnel.com.
studio  IN  CNAME   c346aa27-b4c1-41ba-815c-f66b2b84243b.cfargotunnel.com.
api     IN  CNAME   c346aa27-b4c1-41ba-815c-f66b2b84243b.cfargotunnel.com.

; Option B: Direct Oracle Cloud Compute 24/7 Instance (IP: 147.224.147.30)
; If routing directly to the Oracle VM rather than Cloudflare Tunnel, use these:
; @       IN  A       147.224.147.30
; www     IN  CNAME   omnicodestudio.com.
cloud   IN  A       147.224.147.30
vm      IN  A       147.224.147.30

; ================================================================================
; 2. CREATIVEWORKZSTUDIO LLC LOCAL GATEWAY & WORKSTATION (IP: 71.11.99.168)
; ================================================================================
; Points to CreativeWorkzStudio LLC primary gateway / local staging engine
local   IN  A       71.11.99.168
gateway IN  A       71.11.99.168

; ================================================================================
; 3. DYNAMIC DNS (DuckDNS) FALLBACK ALIASES
; ================================================================================
; Dynamic CNAME mappings to licensed DuckDNS host (omnicodestudio.duckdns.org)
duck    IN  CNAME   omnicodestudio.duckdns.org.
edge    IN  CNAME   omnicodestudio.duckdns.org.

; ================================================================================
; 4. UNIFIED PLATFORM CROSS-LINKS (Yu-Gi-Oh! Server & Simulator)
; ================================================================================
; Live duel simulator on Oracle VM (Raw TCP Port 7911 / HTTP Port 7922)
; NOTE: In Cloudflare Dashboard, play/sim MUST remain DNS-Only (Grey Cloud) for TCP
cards   IN  CNAME   c346aa27-b4c1-41ba-815c-f66b2b84243b.cfargotunnel.com.
play    IN  A       147.224.147.30
sim     IN  A       147.224.147.30
```

<TRACK 5> : {Infrastructure, Operations & Archives}/infra/dns/zone_manager.py

PY | 269 LINES | 925 WORDS | 10.7 KB

```python
#!/usr/bin/env python3
# ==============================================================================
# BLOCK 1: METADATA BLOCK
# ==============================================================================
"""
Module: dns.zone_manager
Architecture: Systems Engineering & Network Edge Routing (OmniCode Platform)
Organization: CreativeWorkzStudio LLC
Description:
    Authoritative BIND RFC 1035 Zone File Generator, Validator, and Synchronizer.
    Manages DNS routing rules for Cloudflare Zero Trust Tunnels, Oracle Cloud
    Compute VM (147.224.147.30), Local Gateway (71.11.99.168), and DuckDNS endpoints.

Exported CLI & API:
    - python3 dns/zone_manager.py validate [file.zone]
    - python3 dns/zone_manager.py inspect [file.zone]
    - python3 dns/zone_manager.py render [--domain DOMAIN] [--out PATH]
"""

import os
import re
import sys
from dataclasses import dataclass
from typing import Dict, List, Optional, Tuple

_CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))
_PROJECT_ROOT = os.path.dirname(_CURRENT_DIR)

DEFAULT_ZONE_FILE_PATH: str = os.path.join(_CURRENT_DIR, "omnicodestudio.com.zone")
ENV_FILE_PATH: str = os.path.join(_PROJECT_ROOT, ".env")


# ==============================================================================
# BLOCK 2: OPENING BLOCK (Data Structures & Models)
# ==============================================================================

@dataclass
class DNSZoneRecord:
    name: str
    record_type: str
    target: str
    ttl: int = 3600
    comment: str = ""

    def to_bind_line(self) -> str:
        base = f"{self.name:<8} IN  {self.record_type:<7} {self.target}"
        if self.comment:
            return f"{base:<50} ; {self.comment}"
        return base


def parse_env_file(path: str) -> Dict[str, str]:
    values = {}
    if not os.path.isfile(path):
        return values
    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if not line or line.startswith("#"):
                continue
            if "=" in line:
                k, v = line.split("=", 1)
                values[k.strip()] = v.strip().strip("\"'")
    return values


# ==============================================================================
# BLOCK 3: BODY BLOCK (Renderer, Validator & Inspector)
# ==============================================================================

def render_zone_file(
    domain: str = "omnicodestudio.com",
    oracle_ip: str = "147.224.147.30",
```

```
    local_ip: str = "71.11.99.168",
    duckdns_host: str = "omnicodestudio.duckdns.org",
    tunnel_id: str = "c346aa27-b4c1-41ba-815c-f66b2b84243b",
    ttl: int = 3600,
) -> str:
    """Renders a standard RFC 1035 / BIND 9 zone file string for Cloudflare import."""
    tunnel_target = f"{tunnel_id}.cfargotunnel.com."
    duck_target = f"{duckdns_host.rstrip('.') }."

    lines = [
        "; ================================================================================",
        f"; BIND RFC 1035 Zone File for {domain}",
        "; CreativeWorkzStudio LLC — OmniCode Platform Infrastructure",
        "; Compatible with Cloudflare DNS Zone File Import (DNS -> Records -> Import)",
        "; ================================================================================",
        f"$ORIGIN {domain.rstrip('.')}.",
        f"$TTL {ttl}",
        "",
        "; ================================================================================",
        "; 1. PRIMARY WEB STUDIO & REST API (HTTP/HTTPS - Port 8080)",
        "; ================================================================================",
        "; Option A: Cloudflare Zero Trust Named Tunnel (Recommended)",
        f"; Tunnel ID: {tunnel_id}",
        DNSZoneRecord("@", "CNAME", tunnel_target, ttl, "Root Apex -> Cloudflare Tunnel").to_bind_line(),
        DNSZoneRecord("www", "CNAME", tunnel_target, ttl, "WWW Subdomain -> Cloudflare Tunnel").to_bind_line(),
        DNSZoneRecord("studio", "CNAME", tunnel_target, ttl, "OmniCode Studio Web Interface").to_bind_line(),
        DNSZoneRecord("api", "CNAME", tunnel_target, ttl, "OmniCode REST API Endpoint").to_bind_line(),
        "",
        "; Option B: Oracle Cloud Compute 24/7 Instance",
        DNSZoneRecord("cloud", "A", oracle_ip, ttl, "Oracle Cloud VM Direct Ingress").to_bind_line(),
        DNSZoneRecord("vm", "A", oracle_ip, ttl, "Oracle Cloud VM Alias").to_bind_line(),
        "",
        "; ================================================================================",
        "; 2. CREATIVEWORKZSTUDIO LLC LOCAL GATEWAY (WORKSTATION IP)",
        "; ================================================================================",
        DNSZoneRecord("local", "A", local_ip, ttl, "CreativeWorkzStudio LLC Workstation WAN").to_bind_line(),
        DNSZoneRecord("gateway", "A", local_ip, ttl, "CreativeWorkzStudio LLC Gateway IP").to_bind_line(),
        "",
        "; ================================================================================",
        "; 3. DYNAMIC DNS (DuckDNS) FALLBACK ALIASES",
        "; ================================================================================",
        DNSZoneRecord("duck", "CNAME", duck_target, ttl, "DuckDNS Dynamic Host Endpoint").to_bind_line(),
        DNSZoneRecord("edge", "CNAME", duck_target, ttl, "DuckDNS Edge Alias").to_bind_line(),
        "",
        "; ================================================================================",
        "; 4. UNIFIED PLATFORM CROSS-LINKS (Yu-Gi-Oh! Server & Simulator)",
        "; ================================================================================",
        DNSZoneRecord("cards", "CNAME", tunnel_target, ttl, "Card Catalog & Lore Web Dashboard").to_bind_line(),
        DNSZoneRecord("play", "A", oracle_ip, ttl, "Live Duel Simulator (Raw TCP 7911 - Grey Cloud)").to_bind_line(),
        DNSZoneRecord("sim", "A", oracle_ip, ttl, "Game Room Manager (Port 7922)").to_bind_line(),
        "",
    ]
    return "\n".join(lines)


def validate_zone_file(file_path: str) -> Tuple[bool, List[str]]:
    """Validates zone file syntax and record definitions."""
    errors = []
    if not os.path.isfile(file_path):
        return False, [f"File not found: {file_path}"]

    with open(file_path, "r", encoding="utf-8") as f:
        content = f.read()

    has_origin = bool(re.search(r"^\$ORIGIN\s+[\w\.\-]+\.", content, re.MULTILINE))
    if not has_origin:
        errors.append("Missing required directive: $ORIGIN <domain>.")

    has_ttl = bool(re.search(r"^\$TTL\s+\d+", content, re.MULTILINE))
    if not has_ttl:
        errors.append("Missing required directive: $TTL <seconds>")

    record_pattern = re.compile(r"^\s*([\@\w\.\-]+)\s+(?:IN\s+)?(A|AAAA|CNAME|TXT|MX|NS)\s+([^\s;]+)", re.MULTILINE)
    matches = record_pattern.findall(content)
    if not matches:
        errors.append("No valid DNS resource records found in file.")

    return len(errors) == 0, errors
```

```
152
153
154 def inspect_zone_file(file_path: str):
155     """Parses and prints a formatted summary table of the DNS zone records."""
156     if not os.path.isfile(file_path):
157         print(f"[!] Error: File '{file_path}' does not exist.")
158         sys.exit(1)
159
160     print("=" * 70)
161     print(f"  🌐 DNS Zone Inspector: {os.path.basename(file_path)}")
162     print("  CreativeWorkzStudio LLC — OmniCode Platform")
163     print("=" * 70)
164
165     with open(file_path, "r", encoding="utf-8") as f:
166         lines = f.readlines()
167
168     origin = "unknown"
169     ttl = "unknown"
170     records = []
171
172     record_re = re.compile(r"^\s*([\@\w\.\-]+)\s+(?:IN\s+)?(A|AAAA|CNAME|TXT|MX|NS)\s+([^\s;]+)(?:\s*;\s*(.*))?")
173
174     for line in lines:
175         stripped = line.strip()
176         if stripped.startswith("$ORIGIN"):
177             origin = stripped.split()[1]
178         elif stripped.startswith("$TTL"):
179             ttl = stripped.split()[1]
180         elif stripped and not stripped.startswith(";"):
181             m = record_re.match(line)
182             if m:
183                 records.append({
184                     "name": m.group(1),
185                     "type": m.group(2),
186                     "target": m.group(3),
187                     "comment": m.group(4) or ""
188                 })
189
190     print(f"• Origin Domain : {origin}")
191     print(f"• Default TTL   : {ttl} seconds")
192     print(f"• Record Count  : {len(records)}\n")
193     print(f"{'NAME':<10} {'TYPE':<8} {'TARGET':<45} {'NOTE'}")
194     print("-" * 75)
195     for r in records:
196         print(f"{r['name']:<10} {r['type']:<8} {r['target']:<45} {r['comment']}")
197     print("-" * 75)
198
199
200 # ==============================================================================
201 # BLOCK 4: CLOSING BLOCK (CLI Dispatcher)
202 # ==============================================================================
203
204 def main():
205     args = sys.argv[1:]
206     command = args[0] if args else "inspect"
207
208     target_file = DEFAULT_ZONE_FILE_PATH
209     env = parse_env_file(ENV_FILE_PATH)
210
211     if command == "inspect":
212         file_to_inspect = args[1] if len(args) > 1 else target_file
213         inspect_zone_file(file_to_inspect)
214
215     elif command == "validate":
216         file_to_val = args[1] if len(args) > 1 else target_file
217         valid, errors = validate_zone_file(file_to_val)
218         if valid:
219             print(f"[✓] Zone file '{file_to_val}' is syntactically valid RFC 1035 format.")
220             sys.exit(0)
221         else:
222             print(f"[✗] Zone file validation failed with {len(errors)} error(s):")
223             for err in errors:
224                 print(f"    - {err}")
225             sys.exit(1)
226
227     elif command == "render":
228         domain = "omnicodestudio.com"
229         out_file = target_file
```

```
        if "--domain" in args:
            idx = args.index("--domain")
            if idx + 1 < len(args):
                domain = args[idx + 1]

        if "--out" in args:
            idx = args.index("--out")
            if idx + 1 < len(args):
                out_file = args[idx + 1]

        oracle_ip = env.get("ORACLE_VM_HOST", "147.224.147.30")
        local_ip = env.get("PUBLIC_WAN_IP", "71.11.99.168")
        duckdns = env.get("PUBLIC_DOMAIN", "omnicodestudio.duckdns.org")
        tunnel_id = env.get("CLOUDFLARE_TUNNEL_ID", "c346aa27-b4c1-41ba-815c-f66b2b84243b")

        rendered = render_zone_file(
            domain=domain,
            oracle_ip=oracle_ip,
            local_ip=local_ip,
            duckdns_host=duckdns,
            tunnel_id=tunnel_id,
        )

        with open(out_file, "w", encoding="utf-8") as f:
            f.write(rendered)

        print(f"[✓] Rendered RFC 1035 zone file to: {out_file}")
        inspect_zone_file(out_file)

    else:
        print("Usage:")
        print("  python3 dns/zone_manager.py inspect [path.zone]")
        print("  python3 dns/zone_manager.py validate [path.zone]")
        print("  python3 dns/zone_manager.py render [--domain DOMAIN] [--out PATH]")


if __name__ == "__main__":
    main()
```